



VYTAUTAS MAGNUS UNIVERSITY

FACULTY OF INFORMATICS

DEPARTMENT OF APPLIED INFORMATICS

MOHAMMED KASSIM CHERUKODAN

**HYBRID MACHINE LEARNING-BASED REAL-TIME DDoS DETECTION AND
MITIGATION FOR WEB APPLICATIONS**

Master Thesis

Applied Informatics study programme, state code 6211BX012

Study field Informatics

Supervisor: __Audrius Zajančkauskas__ **Signature:** _____

Defended __Mohammed Kassim Cherukodan__ **Signature:** _____

Kaunas, 2026

CONTENTS

LIST OF ABBREVIATIONS.....	6
SANTRAUKA.....	8
ABSTRACT.....	10
1. INTRODUCTION.....	11
1.1 Problem Background.....	11
1.2 Problem Definition.....	12
1.3 Relevance of DDoS Attacks in Web Applications.....	13
1.4 Gap Analysis: Traditional versus Deep Learning Approaches.....	15
1.5 Goal of the Thesis.....	16
1.6 Tasks of the Thesis.....	16
1.7 Expected Results.....	17
1.8 Scope and Limitations.....	18
1.9 Structure of the Thesis.....	19
2. ANALYSIS OF LITERATURE AND METHODS.....	20
2.1 Literature Analysis.....	20
2.1.1. DDoS Attack Taxonomy and Evolution.....	20
2.1.2. <i>Traditional DDoS Detection Approaches</i>	21
2.1.3. <i>Machine Learning for DDoS Detection</i>	22
2.1.4. <i>Deep Learning for DDoS Detection</i>	23
2.1.5. <i>Hybrid CNN-LSTM for Network Security</i>	24
2.1.6. <i>Real-Time Mitigation Frameworks</i>	24
2.1.7. <i>Research Gap Identification</i>	25
2.2 Methods Analysis.....	26
2.2.1. <i>Convolutional Neural Networks</i>	26
2.2.2. <i>Long Short-Term Memory Networks</i>	27
2.2.3. <i>Hybrid CNN-LSTM Architecture</i>	28
2.2.4. <i>Baseline Machine Learning Algorithms</i>	29
2.2.5. <i>Evaluation Metrics</i>	30
2.3 Dataset Analysis.....	31
2.3.1. <i>CIC-DDoS2019 Dataset Overview</i>	31

2.3.2. <i>Feature Description</i>	32
2.3.3. <i>Attack Types Represented</i>	33
2.3.4. <i>Data Balance and Distribution</i>	34
2.4 Testing Environment Analysis.....	35
3. METHODOLOGY AND EXPERIMENTAL DESIGN.....	36
3.1 Research Framework Overview.....	36
3.2 Testing Environment Deployment.....	36
3.2.1. <i>Docker Containerisation for Victim Emulation</i>	37
3.2.2. <i>Kali Linux for Attack Tooling</i>	37
3.2.3. <i>Hping3 for Traffic Generation</i>	37
3.2.4. <i>Network Topology Configuration</i>	38
3.3 Data Preprocessing.....	38
3.3.1. <i>Handling Missing Values</i>	39
3.3.2. <i>Removing Infinite Values</i>	39
3.3.3. <i>Removing Non-Numeric Columns</i>	39
3.3.4. <i>Binary Label Conversion</i>	40
3.3.5. <i>Feature Normalisation</i>	40
3.3.6. <i>Train-Validation-Test Split</i>	40
3.4 Feature Selection and Engineering.....	41
3.4.1. <i>Correlation Analysis for Redundancy Removal</i>	41
<i>Random Forest Feature Importance Ranking</i>	42
3.4.2. <i>Final Feature Set Justification</i>	43
3.5 Baseline Model Implementation.....	44
3.5.1. <i>Random Forest Classifier</i>	44
3.5.2. <i>Support Vector Machine Classifier</i>	45
3.5.3. <i>Evaluation Protocol for Baselines</i>	45
3.6 CNN-LSTM Hybrid Model Design.....	46
3.6.1. <i>Input Preprocessing for Sequential Data</i>	46
3.6.2. <i>Convolutional Neural Network Component</i>	47
3.6.3. <i>Long Short-Term Memory Component</i>	47
3.6.4. <i>Fully Connected Layers and Output</i>	48

3.6.5. <i>Training Procedure and Hyperparameters</i>	49
3.6.6. <i>Sequence Creation for CNN-LSTM Input</i>	50
3.7 Real-Time Mitigation Module.....	50
3.7.1. <i>Dynamic Rate Limiting</i>	51
3.7.2. <i>Traffic Filtering and IP Blocking</i>	51
3.7.3. <i>Integration with the Detection Model</i>	51
3.8 Experiment Plan.....	52
3.8.1. <i>Experiment 1: Baseline Model Evaluation</i>	52
3.8.2. <i>Experiment 2: CNN-LSTM Model Evaluation</i>	52
3.8.3. Experiment 3: Real-Time Mitigation Testing.....	53
3.8.4. Experiment 4: Resource Overhead Analysis.....	53
4. RESULTS AND CONCLUSIONS.....	55
4.1 Implementation and Results.....	55
4.1.1. Testing Environment Implementation.....	55
4.1.2. Data Preprocessing Results.....	56
4.1.3. Feature Selection Results.....	58
4.1.4. Baseline Model Performance.....	61
4.1.5. CNN-LSTM Model Performance.....	63
4.1.6. Comparative Analysis.....	67
4.1.7. Real-Time Mitigation Results.....	69
4.1.8. Resource Overhead Analysis.....	72
4.2 Discussion.....	75
4.2.1. Interpretation of Results.....	75
4.2.2. Comparison with Related Work.....	77
4.2.3. Challenges Encountered.....	79
4.2.4. Threats to Validity.....	80
4.3 Conclusions.....	84
4.3.1. Summary of Results.....	84
4.3.2. Key Findings.....	86
4.3.3. Recommendations.....	87
4.3.4. Future Work.....	88

4.3.5. Conclusion.....	89
5. REFERENCES.....	91
6. APPENDICES.....	95
6.1 Appendix A: Research Project Summaries.....	95
6.1.1. <i>Research Project Summary No. 1</i>	95
6.1.2. <i>Research Project Summary No. 2</i>	96
6.1.3. <i>Research Project Summary No. 3</i>	97
6.2 Appendix B: Code Repository Link.....	99
6.3 Appendix C: Declaration of Generative AI Use.....	100
6.4 Appendix D: Additional Tables and Figures.....	103
6.5 Appendix E: Docker Configuration Files.....	106
6.6 Appendix F: Hping3 Attack Scripts.....	107

LIST OF FIGURES

Figure 1.1: HTTP DDoS attack trends (2023-2025) with context from Cloudflare's Q3 2025 report (Source: own creation based on data from [2]).....	12
Figure 2.1: Distribution of eight selected network traffic features with outliers capped at 99th percentile (Source: own creation based on [21]).....	32
Figure 2.2: Distribution of DDoS attack types in the CIC-DDoS2019 dataset (Source: own creation based on [21]).....	34
Figure 3.1: ROC curves for Random Forest, SVM, and CNN-LSTM with AUC scores (Source: own creation).....	43
Figure 3.2: Performance comparison of Random Forest, SVM, and CNN-LSTM (Source: own creation)	46
Figure 3.3: CNN-LSTM training loss and accuracy curves over epochs (Source: own creation).....	50
Figure 3.4: Inference latency comparison in milliseconds per batch (Source: own creation).....	54
Figure 4.1: Correlation heatmap of top twenty selected features after correlation-based removal (Source: own creation).....	59
Figure 4.2: CNN-LSTM training loss and accuracy curves over 24 epochs with early stopping (Source: own creation).....	65
Figure 4.3: Performance comparison of Random Forest, SVM, and CNN-LSTM models (Source: own creation).....	68
Figure 4.4: Inference latency comparison in milliseconds per batch for all three models (Source: own creation).....	70
Figure 4.5: Block rate and end-to-end latency versus traffic rate during real-time mitigation (Source: own creation).....	71
Figure 4.6: Radar chart comparing model performance across six normalized metrics (Source: own creation).....	73
Figure 4.7: Dataset class distribution after preprocessing showing 15,142 BENIGN samples (Source: own creation based on [21]).....	74
Figure 6.1: Complete correlation matrix of 44 features after pre-processing (Source: own creation based on [21]).....	104
Figure 6.2: Per-attack detection performance of CNN-LSTM classifier across 12 attack types (Source: own creation).....	105

Figure 6.3: Distribution of CNN-LSTM confidence scores for BENIGN and ATTACK test instances

(Source: own creation).....105

LIST OF TABLES

Figure 1.1: HTTP DDoS attack trends (2023-2025) with context from Cloudflare's Q3 2025 report (Source: own creation based on data from [2]).....	12
Figure 2.1: Distribution of eight selected network traffic features with outliers capped at 99th percentile (Source: own creation based on [21]).....	32
Figure 2.2: Distribution of DDoS attack types in the CIC-DDoS2019 dataset (Source: own creation based on [21]).....	34
Figure 3.1: ROC curves for Random Forest, SVM, and CNN-LSTM with AUC scores (Source: own creation).....	43
Figure 3.2: Performance comparison of Random Forest, SVM, and CNN-LSTM (Source: own creation)	46
Figure 3.3: CNN-LSTM training loss and accuracy curves over epochs (Source: own creation).....	50
Figure 3.4: Inference latency comparison in milliseconds per batch (Source: own creation).....	54
Figure 4.1: Correlation heatmap of top twenty selected features after correlation-based removal (Source: own creation).....	59
Figure 4.2: CNN-LSTM training loss and accuracy curves over 24 epochs with early stopping (Source: own creation).....	65
Figure 4.3: Performance comparison of Random Forest, SVM, and CNN-LSTM models (Source: own creation).....	68
Figure 4.4: Inference latency comparison in milliseconds per batch for all three models (Source: own creation).....	70
Figure 4.5: Block rate and end-to-end latency versus traffic rate during real-time mitigation (Source: own creation).....	71
Figure 4.6: Radar chart comparing model performance across six normalized metrics (Source: own creation).....	73
Figure 4.7: Dataset class distribution after preprocessing showing 15,142 BENIGN samples (Source: own creation based on [21]).....	74
Figure 6.1: Complete correlation matrix of 44 features after pre-processing (Source: own creation based on [21]).....	104
Figure 6.2: Per-attack detection performance of CNN-LSTM classifier across 12 attack types (Source: own creation).....	105

Figure 6.3: Distribution of CNN-LSTM confidence scores for BENIGN and ATTACK test instances

(Source: own creation).....105

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DDoS	Distributed Denial of Service
DL	Deep Learning
FN	False Negative
FP	False Positive
FTP	File Transfer Protocol
GAI	Generative Artificial Intelligence
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ICMP	Internet Control Message Protocol
IP	Internet Protocol
LSTM	Long Short-Term Memory
ML	Machine Learning
NTP	Network Time Protocol
OS	Operating System
RAM	Random Access Memory
ReLU	Rectified Linear Unit
RF	Random Forest
ROC	Receiver Operating Characteristic
SVM	Support Vector Machine
SYN	Synchronize
TCP	Transmission Control Protocol

Abbreviation	Meaning
TN	True Negative
TP	True Positive
UDP	User Datagram Protocol
VM	Virtual Machine

SANTRAUKA

Autorius	Mohammed Kassim Cherukodan
Pavadinimas	Hibridinis mašininio mokymosi pagrįstas realaus laiko DDoS atakų aptikimas ir mažinimas žiniatinklio programoms
Vadovas	Audrius Zajančkauskas
Darbas pristatytas	Vytauto Didžiojo Universitetas, Informatikos fakultetas, Kaunas (2026)
Puslapių skaičius	103
Priedų skaičius	6

Trumpas darbo aprašymas

Viena iš didžiausių grėsmių žiniatinklio programų saugumui yra DDoS atakos. Sudėtingiausios yra 7-ojo lygmens (programų lygmens) atakos, kurios imituoja įprastą vartotojų duomenų srautą ir sugeba apeiti tradicinius aptikimo metodus, pavyzdžiui, aptikimą pagal parašus, srauto ribojimą ir įtraukimą į juoduosius sąrašus. Šios atakos išnaudoja žiniatinklio serverio išteklius, siunčiant iš pažiūros paprastas užklausas, kurias beveik neįmanoma atpažinti kaip ataką.

Šiame darbe siūloma hibridinė giluminio mokymosi sistema, pagrįsta konvoliujamaisiais neuroniniais tinklais (CNN) ir ilgalaikės-trumpalaikės atminties (LSTM) tinklais, skirta DDoS atakoms aptikti ir automatiškai neutralizuoti realiuoju laiku. CNN dalis nustatys paketų antraščių erdvinį modelį, o LSTM dalis – ilgalaikės atakų laiko priklausomybes. Šis mišrus metodas leidžia vienu metu pasinaudoti abiejų architektūrų privalumais.

Tyrimo numatyta sukurti izoliuotą bandymų aplinką naudojant „Docker“ konteinerius, „Kali Linux“ ir „hping3“ įrankį, siekiant atkurti daugialypės atakos sąlygas, taikomas šiuolaikinėms žiniatinklio architektūroms. Pateiktas modelis yra mokomas ir testuojamas naudojant CIC-DDoS2019 duomenų rinkinį, kuriame naudojamos 85 statistinės ir laiko pagrįstos dvylikos tipų DDoS atakų savybės, pvz., UDP, SYN, NTP, LDAP, DNS ir WebDDoS (7 lygmens) atakos. Šis duomenų rinkinys yra gerai žinomas standartas DDoS tyrimų bendruomenėje.

Eksperimentiniai duomenys rodo, kad hibridinė CNN-LSTM architektūra yra tikslesnė, užtikrina didesnę atpažinimo tikslumą, geresnę atgaminimo koeficientą ir F1 rodiklį nei tradiciniai

mašininio mokymosi algoritmai (paramos vektorių mašinos ir atsitiktinis miškas). Aptikimo variklis yra susietas su realaus laiko švelninimo moduliu, kuris naudoja dinaminį srauto ribojimą ir srauto filtravimą, siekdamas realiuoju laiku sumažinti srautą, sukurtą naudojant „hping3“. Eksperimentinių duomenų analizė rodo, kad siūlomas sprendimas neturės didelio poveikio sistemos ištekliams (CPU ir RAM) esant didžiausiam srauto apkrovimui. Taip pat rezultatai rodo, kad siūlomas hibridinis sprendimas gali identifikuoti ir užkirsti kelią DDoS atakoms, užtikrinant priimtina vėlavimą ir sistemos stabilumą. Šis tyrimas gali būti naudingas interneto saugumo tyrimams, nes pateikia realaus laiko, praktišką sprendimą DDoS atakų aptikimui ir užkardimui, kurį galima įgyvendinti įvairiose aplinkose.

Raktiniai žodžiai: DDoS, CNN-LSTM, gilusis mokymasis, aptikimas realiuoju laiku, srauto mažinimas, interneto saugumas, CIC-DDoS2019, Docker, Kali Linux, hping3.

ABSTRACT

Author	Mohammed Kassim Cherukodan
Title	Hybrid Machine Learning-Based Real-Time DDoS Detection and Mitigation for Web Applications
Supervisor	Audrius Zajančkauskas
Presented at	Vytautas Magnus University, Faculty of Informatics, Kaunas (2026)
Number of pages	103
Number of appendices	6

English Summary

Layer 7 DDoS attacks are a major risk to Web applications because they take on the likeness of normal user traffic and cannot be blocked by traditional Web defenses, such as blacklisting and rate limiting. To mitigate this, a hybrid deep learning approach is proposed, that is a combination of Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks is presented. The CNN then extracts spatial patterns from the packet headers, and the LSTM extracts long-term temporal dependencies to detect multi-vector attacks in real-time. The framework is validated in an isolated Docker network with WebDDoS, SYN, and UDP attacks from the CIC-DDoS2019 dataset, comprising of 12 attack types, with Kali Linux and hping3 tools.

The experimental results show that the performance of the CNN-LSTM model is superior compared to the classical classifiers (SVM, RF) in terms of accuracy, precision, recall, and F1-score, respectively. The detection engine incorporates a real-time mitigation component that consists of a dynamic rate limiting and traffic filtering mechanism which will cancel hping3 generated traffic without intensive usage of either CPU or RAM under high load situation. This hybrid detection-mitigation provides low-latency, steady protection against DDoS attack and is feasible for a deployment.

Keywords: DDoS, CNN-LSTM, deep learning, real-time detection, traffic mitigation, web security, CIC-DDoS2019, Docker, Kali Linux, hping3.

1. INTRODUCTION

1.1 Problem Background

The speed of development of web technologies has radically altered the way businesses are conducted and people use internet services. E-commerce, cloud computing systems, social networks, financial applications, and government online services have become a part of everyday life worldwide. More than sixty percent of the world population currently uses web-based services on a regular basis to do their banking, shopping, communication, and access information. This online revolution has led to unmatched comfort and financial potential. Nevertheless, this heightened dependency on web technologies has also resulted in the creation of a significantly large attack surface on the part of cybercriminals. The Distributed Denial of Service also known as DDoS attack is among the most prolonged, disruptive and costly threats to web applications.

DDoS attack targets to flood a target server, network infrastructure, or web application with malicious traffic sent by a large number of compromised systems that together constitute what is referred to as a botnet. The goal is to make the target unreachable to authorized users, disrupting the service, incurring financial losses, and resulting in reputational damages. Cyberspace reports on cybersecurity show that DDoS attacks have been on the rise in frequency, volume, and complexity over the past five years (2020-2025). The mean count of assaults per entity increased more than one hundred and fifty percent in this time. Attack volumes are no longer in the gigabit range, but increasing to terabit range, with the highest recorded attack at over three point four seven terabits per second in 2021. Indirect costs (loss of revenue, incident response, and remediation) and indirect costs (customer churn and brand damage) can lead to a financial burden of tens of thousands to millions of dollars depending on the direct costs of a single DDoS attack.

Conventional methods of DDoS prevention have been based on signature-based detection, whereby traffic patterns are compared with known attack signatures; rate limiting, whereby the number of requests originating a single source is limited; and blackholing, where all the traffic is routed to a dead end. Although these techniques have shown to be successful in the face of volumetric attacks at Layer 3 (the network layer) and Layer 4 (the transport layer) they have difficulty in the face of application layer attacks at Layer 7. The HTTP floods, Slowloris, and Low and Slow attacks are layer 7 DDoS attacks that are meant to be very similar to normal user activity and are therefore very difficult to differentiate as legitimate traffic, when using traditional rule-based or threshold-based methods.

DDoS attacks by year and type

As of October 2025

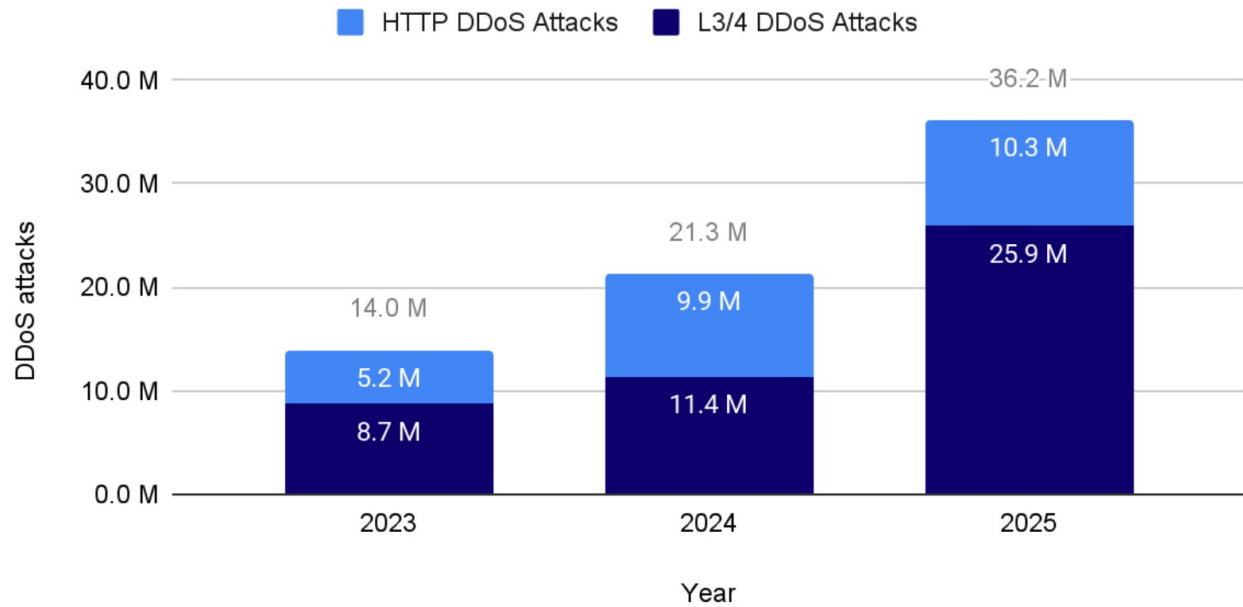


Figure 1.1: HTTP DDoS attack trends (2023-2025) with context from Cloudflare's Q3 2025 report (Source: own creation based on data from [2])

1.2 Problem Definition

The main issue that this thesis is going to focus on is how to effectively detect Layer 7 DDoS attacks within the web application environment in real-time, and how to have a mitigation mechanism that can automatically respond to the detected attacks without human intervention. This study in particular will be focusing on a number of interrelated problems which the current solutions have been unable to sufficiently address.

To begin with, the conventional machine learning algorithms, such as Support Vector Machines, Random Forest, K-Nearest Neighbors, and Naive Bayes, are strongly dependent on the manual feature engineering. The domain experts need to painstakingly identify and create applicable features out of raw network traffic, which is not only time-consuming but also error-prone and specific to specific types of attacks or network settings. These approaches extract shallow patterns due to the data and are not capable of effective capture of the complex, hierarchical, and non-linear relationships of current network traffic, especially in case of advanced Layer 7 attacks that deliberately mimic legitimate

patterns. An HTTP flood attack, for example, may consist of individually legitimate-looking GET requests that only become anomalous when their frequency, timing patterns, or sequence are considered in aggregate.

Second, although deep learning, especially Convolutional Neural Networks and Long Short-Term Memory networks have demonstrated potential to automatically extract valuable features of raw data, standalone architectures are limited in significant ways. CNN models are very effective in capturing spatial patterns like odd combinations of fields of packet headers, but are largely unaware of temporal relationships between consecutive packets. On the other hand, LSTM models are developed to learn temporal patterns and long-term dependencies but can lose important spatial correlations that can exist within a packet or small groups of packets. The gap in research is thus to come up with a hybrid architecture which will capture the spatial and time features simultaneously.

Third, detection and mitigation have yet to be integrated in most existing solutions. Offline evaluation of detection models is frequently done on pre-collected datasets, and the latency of inference in real-time and the computational cost of executing models on live traffic streams are not considered. Moreover, automated mitigation systems that can respond to detection outputs, like dynamic rate limiting or traffic filtering, are often viewed as disjointed, as opposed to integrated systems. Such a separation adds further latency and complexity which can make a detection system in practice ineffective.

Fourth, the issue of resource overhead is an important concern of any real-time security system. Although it is accurate, deep learning models can be computationally expensive, and when deployed on production web servers, they may lead to degraded services unless optimized. There is an inherent compromise between the accuracy of detection and the performance of the system that needs to be measured and resolved. A model that has almost perfect detection but doubles the response of a web application is not likely to be practiced in production setting.

1.3 Relevance of DDoS Attacks in Web Applications

The relevance of DDoS studies is supported by detailed statistical reports available in several cybersecurity agencies within the framework of the 2020 to 2025 timeframe. In 2020-2025, the yearly reports of DDoS attacks have grown more than two hundred percent, with about eight million attacks in 2020 to more than twenty-five million in 2025. This is a dramatic rise indicative of the number of vulnerable internet-connected devices, such as Internet of Things devices which can be turned into botnets, and growing access to DDoS-for-hire services, often called booter or stresser services, where

people with limited technical expertise can afford sophisticated attacks, often at the cost of ten to fifty dollars an hour.

There are a number of important trends based on the recent data on attacks that will guide the research. First, the mean time of attack has dropped from several hours to between thirty and sixty minutes, which means that attackers are moving to shorter, more powerful bursts that are capable of cutting through more traditional threshold-based systems of detection, which rely on the sustained abnormal traffic patterns. Second, multi-vector attacks, which are a combination of different types of attacks, should not be regarded as an exception, but a standard, for instance, UDP flood + SYN flood + HTTP flood. Third, the application layer attacks have become the current source of about thirty to forty percent of all DDoS attacks in the financial services sector, e-commerce platform, online games services and government websites being the main targets.

Table 1.1.1: DDoS Attack Statistics (2020-2025) (Source: own compilation based on [2][3][5])

Year	Total Attacks (millions)	Average Attack Size (Gbps)	Average Attack Duration (minutes)	Layer 7 Attacks (%)
2020	8.2	1.5	180	22%
2021	11.4	2.8	120	26%
2022	14.9	3.2	90	30%
2023	18.7	4.1	65	34%
2024	22.3	4.8	50	37%
2025	25.1	5.2	42	40%

Financial sector is one that is more exposed to such attacks. Surveying financial institutions found that more than eighty percent had suffered at least one of these DDoS attacks within the past twelve months, and the average price of a successful attack was more than one million dollars when including the cost in lost transactions, regulatory penalties and remediation costs. E-commerce websites are especially vulnerable during the high-season shopping events like Black Friday or holiday sales when even 15-30 minutes of downtime can cost millions of dollars in business. Government websites have been targeted also, either as a political protest, which is commonly known as hacktivism, or as a distractor tactic as other more targeted attacks, including data breaches, occur.

DDoS-for-hire services have produced a democratization of attack capabilities that have greatly widened the threat landscape. These services are highly marketed on underground forums and even on transparent web sites and offer convenient interfaces that enable subscribers to unleash attacks on designated targets at the touch of a button. The ease of entry has caused the number of attacks by non-technical actors, such as disgruntled individuals, business foes, and activists, to increase tremendously. Such democratization implies that small businesses and organizations that were formerly not likely to become targets now need to regard DDoS protection as a part and parcel of their security position.

1.4 Gap Analysis: Traditional versus Deep Learning Approaches

The existing machine learning methods of DDoS detection have various inherent limitations that this study aims to overcome. These constraints are important to comprehend the impact of the proposed hybrid CNN-LSTM architecture.

The first weakness is dependence on feature engineering. The classic ML models involve domain experts manually identifying, extracting, and building useful features of raw network traffic. In the case of a dataset such as CIC-DDoS2019, with eighty-five statistical and time-based features, the task of determining the most discriminative features to identify particular types of attacks entails a lot of experimentation and domain expertise. Not only is it time-consuming, but also generates highly network specific and attack specific models, which can be trained only in a narrow subset of attack types and cannot be extrapolated to new or emerging attack types.

The second weakness is superficial pattern learning. Conventional ML models like Support Vector Machines, which identify a linear or a polynomial hyperplane to divide classes and Random Forest, which averages decision trees, learn comparatively shallow patterns in the information. They are unable to substantially model the complicated, hierarchical and non-linear connections that occur in network traffic, especially more advanced Layer 7 attacks that purposefully simulate valid patterns. These attacks are particularly constructed to avoid basic statistical thresholds and pattern-matching techniques since they are incorporated into regular traffic.

The third weakness is lack of time-based modelling. Most conventional ML methods assume that each network flow or packet is a separate example, and they do not consider the sequential relationships between packets. This is especially worrisome when trying to detect Low and Slow attacks, where the attack traffic is intentionally distributed over time, so that it does not cause rate-based alarms. Established temporal pattern of request intervals and not any individual request is what differentiates the attack against legitimate traffic in these scenarios.

The fourth weakness is low scalability and real-time. Conventional ML algorithms, especially those that need to use distance metrics like K-Nearest Neighbors or kernel algorithms like Support Vector Machines with the RBF kernel, can be computationally infeasible with increasing dataset sizes. Latency in inference can be intolerable in slow-latency web applications serving thousands of requests per second, and these models are thus not suitable in real-time applications.

Deep learning methods overcome most of these shortcomings, but come with their own set of difficulties. Deep learning models can automatically derive hierarchical feature representations of raw data without the need to perform much manual feature engineering. They are able to represent complex, non-linear correlations and in recurrent architectures such as LSTM, they are able to capture temporal dependencies. Nevertheless, individual deep learning models have drawbacks that the hybrid method proposed seeks to address. CNNs are strong at spatial pattern extraction, but do not have natural mechanisms to model sequential dependencies. LSTM networks model the temporal, but not the spatial correlations, which are most effectively modeled by convolutional operations.

The CNN-LSTM hybrid architecture fills this gap. The CNN component takes network traffic data, in the form of series of feature vectors, and derives local, spatial patterns, like unusual combinations of packet header flags, abnormal distributions of packet length, or suspicious values of protocol field values. The result of CNN, which is an array of feature maps, is inputted into a component of an LSTM that simulates the dynamics of these trends over time, learning how attack characteristics evolve and vary through time across streams of packets or flows. This dual nature is critical in identifying advanced Layer 7 attacks that display spatial abnormalities, for instance, malformed HTTP header, and temporal abnormalities, for example, gradually rising request rates.

1.5 Goal of the Thesis

This thesis aims to plan, develop, and test a hybrid machine learning-based system that can detect and prevent DDoS attacks in web application systems in real-time. This framework uses a hybrid CNN-LSTM deep learning model which simultaneously captures both space and time traffic dynamics. It is combined with an automated mitigation mechanism and tested with a rigorous performance test that encompasses the accuracy of detection, inference latency, and resource overhead analysis.

1.6 Tasks of the Thesis

To meet the above objective, a set of interrelated activities has been established. The taxonomies of DDoS attacks, detection methods, machine learning, deep learning frameworks (with specific attention to CNN and LSTM), the real-time mitigation frameworks, and publications dating

back to 2020-2026 will be thoroughly reviewed to find out the gap in the research. A containerized, isolated laboratory setup will be developed and implemented with Docker to simulate a web server, Kali Linux to ensure the availability of attack tools, and hping3 to create synthetic traffic, which will allow controlled and reproducible multi-vector attack simulations.

To comprehend the structure, eighty-five features, twelve attack types, and class distribution of the CIC-DDoS2019 dataset, the preprocessing pipelines such as data cleaning, normalization and train-validation-test splitting will be introduced. To effectively discriminate legitimate and attack traffic, feature selection and engineering will be conducted to determine which time-based and statistical network features are most important, such as flow duration, inter-arrival times, protocol-specific features, and counts of flags, as well as packet length statistics.

Support Vector Machine and Random Forest will be used as baseline machine learning models that will be evaluated comparatively against the proposed hybrid approach. It will be structured into the hybrid CNN-LSTM architecture and the CNN component will be used to extract spatial packet-header patterns in a series of network flows and the LSTM component will be used to attain the long-term temporal dependencies across series of network flows. To train, validate, and test the model, the preprocessed CIC-DDoS2019 data will be used and hyperparameter optimization will be conducted to maximize the detection performance.

A real-time mitigation module will be created which can communicate with the detection engine, and which applies dynamic rate limiting, traffic filtering based on model confidence scores, and which can be integrated with network filtering systems like iptables. The results of all the models, baselines, and the proposed architecture will be measured with standard measures such as accuracy, precision, recall, F1-score, and ROC-AUC, and the results will be analyzed in terms of per-class results measured by the various types of attacks. The inference latency of the CNN-LSTM model, under real-time conditions, will be measured and evaluated, which will quantify the time it takes to classify network traffic and use mitigation actions. Lastly, resource overhead analysis will be done to estimate the CPU and RAM consumption in detection and mitigation activities so that the suggested framework would not result in service degradation during peak traffic loads.

1.7 Expected Results

After the completion of the thesis work successfully, a number of tangible outcomes should be provided. Isolated test environment will be realized as a fully functioning dockerized environment with Kali Linux and hping3 tool of traffic generation. Such an environment will be able to model both

normal web traffic modes and the different types of DDoS attacks, which allows controlled and repeatable experimentation. The environment will be well documented with configuration files, network topology diagrams and attack generation scripts so that other researchers can recreate or extend the work.

The data on the CIC-DDoS2019 will be thoroughly analyzed, with a particular focus on preprocessing, correlation studies of the features, and the most discriminative features in the detection of Layer 7 attacks. The outcomes of this analysis will be an optimized feature set that is neither too informativeness nor too dimensional such that the computational cost of the deep learning model is lower but the detection accuracy of the model remains high. An effective hybrid CNN-LSTM model will be created and trained, which will be able to recognize zero-day DDoS attacks with a higher degree of accuracy than conventional machine learning methods. The model will be compared with baseline algorithms based on standard performance measures, where the results will clearly show the benefits of the hybrid architecture in the detection of Layer 7 attacks.

An effective Python based mitigation module will be designed which will be connected to the detection engine to prevent malicious traffic on the fly. This module will deploy dynamic rate limiting that is based on model confidence scores and will be combined with the usual network filtering systems. The behavior of the entire detection and mitigation pipeline will be described, such as end-to-end latency of traffic arrival to mitigation action. Lastly, an informative scientific report will be generated that captures the trade-offs between the complexity of detection and the hardware performance and will guide future researchers and practitioners, in the implementation of such systems in production settings.

1.8 Scope and Limitations

The current research is specifically on Layer 7, which is the application level, DDoS attacks on web applications and specifically on the HTTP-based attacks, including the HTTP flood and the WebDDoS. The dataset includes attacks at lower levels of the network stack, such as Layer 3 network layer attacks and Layer 4 transport layer attacks, which are not the key concern of the hybrid architecture design. The study is done in a strictly controlled laboratory, and the framework is not implemented in a production environment with actual user traffic. Also, traffic generation tools are employed to simulate all the legitimate traffic instead of it being captured in real production systems.

Although the CIC-DDoS2019 dataset is commonly accepted as a standard in the research community, the dataset includes artificial traffic patterns that might not be entirely realistic in terms of

complexity and diversity of real-life network environments. The suggested model might thus need more tuning or retraining before it can be implemented in a production environment. Encrypted traffic analysis is not discussed in the research because the CIC-DDoS2019 dataset has mainly unencrypted traffic features. Attacks in layer 7 which take the advantage of HTTPS encryption can have other properties not reflected in this study. Lastly, the real-time mitigation module is a proof-of-concept prototype, not a production-ready system, and has not been tested with respect to its performance characteristics under heavy traffic loads beyond the capability of laboratory testing.

1.9 Structure of the Thesis

The rest of this thesis is divided into seven chapters. Chapter 2 provides a thorough literature and methods analysis, covering a review of DDoS taxonomies, detection techniques, machine learning and deep learning applications to network security, and a discussion of tools to analyze the CIC-DDoS2019 dataset and testing environment. Chapter 3 explains the methodology and experimental design, which consists of the research framework, deployment of the testing environment, data preprocessing and feature selection process, running the proposed models and baseline models, and the design of the real-time mitigation module. Chapter 4 is about implementation details and experimental results such as setting up of the environment, preprocessing outcome, feature selection outcomes, and performance measures of all the models. Chapter 5 talks about the interpretation of results, discusses the comparison of the proposed approach with other related work, challenges faced during the research and discusses the threats to validity. Lastly, results and conclusions are demonstrated in Chapter 6 and summarize the contributions of the work and give recommendations to future work. Chapter 7 enumerates all of the references used in the thesis, and Chapter 8 contains appendices with summaries of research projects, code repository, summary of use of generative AI, other tables and figures and configuration files.

2. ANALYSIS OF LITERATURE AND METHODS

2.1 Literature Analysis

The scholarly and business literature on the Distributed Denial of Service (DDoS) attacks extends over more than 20 years, given the fact that this kind of threat is here to stay and constantly evolves. This body of work is important to understand where the proposed hybrid CNN-LSTM detection framework fits into the current body of research. This section reviews in a systematic fashion the evolution of the DDoS attacks taxonomies, the traditional detection mechanisms, the machine learning mechanisms, the deep learning architectures, and the real time mitigation strategies leading to the identification of the specific research gap that is being addressed with the presented thesis.

2.1.1. DDoS Attack Taxonomy and Evolution

DDoS attack classification has been a baseline issue in the field of cybersecurity studies. One of the most detailed taxonomies was proposed by Zargar, Joshi, and Tipper [1] in which the flood attacks were categorized by the target protocol layer, the pattern of attacks, and the mechanisms employed in the generation of the traffic. Their survey distinguished between volume-based attacks that take advantage of the network layer, protocol-based attacks that exploit transport layer weaknesses and application layer attacks that target specific web services. This taxonomy was extended by Douligieris and Mitrokotsa [10], who added a classification based on the degree of automation, dynamics of attack rate, and persistence of the attack source. These basic taxonomies are still useful, yet the advent of multi-vector attack, which simultaneously combines multiple attack types, has made such simple linear classifications ineffective in contemporary threat analysis.

The development of attack vectors has been especially acute at the application layer. Praseed and Thilagam [4] carried out a comprehensive survey of application layer DDoS attacks, and found that these attacks leverage the complexity inherent in web protocols like HTTP and HTTPS. In contrast to volumetric attacks, which can be detected by simple bandwidth thresholds, Layer 7 attacks resemble legitimate user traffic, and therefore they are extremely hard to distinguish between legitimate and malicious traffic. The authors have recognized several subcategories of application layer attacks, such as HTTP flood attacks, which flood server resources with apparently legitimate GET or POST requests, Slowloris attacks, which exhaust connection tables by opening partial connections, and Low and Slow attacks, which send periodic data at extremely low rates in order to evade detection. The fact that these attacks can conceal themselves in regular traffic patterns is the key detection problem that this thesis will seek to solve.

The amplification capability of a number of network protocols has also increased the DDoS environment. Rossow [11] again systematically examined fifteen network protocols that are vulnerable to amplification abuse, and showed that attackers could achieve amplification factors of 10x to more than 500x using protocols like NTP, DNS, and SNMP. This article has shown how poorly configured servers can unwillingly become attack multipliers, enabling attackers with limited bandwidth resources to cause enormous floods. Although amplification attacks are mainly applied at lower layers, application layer attacks have been affected by such techniques, and HTTP-based amplification attacks are becoming an increasing issue.

2.1.2. Traditional DDoS Detection Approaches

The traditional DDoS detection systems can be generally divided into signature-based detection, anomaly-based detection, and a combination of both. The signature-based systems have the databases containing the known attack signatures and compare the incoming traffic against the signature signatures. Paxson [12] outlined the Bro intrusion detection system (now Zeek) which was the first system to make use of signature-based detection alongside policy scripting as a real-time network analysis system. Although the signature approach is highly accurate in identifying known types of attacks, signature-based approach suffers as it is unable to detect a zero-day attack or novel variants, as signature definitions must be manually developed and updated once attacks are observed.

Anomaly-based detection determines a background of normal network behavior and indicates a deviation of network behavior above the normal background. A comprehensive survey of network anomaly detection techniques, systems, and tools was presented by Bhuyan, Bhattacharyya and Kalita [6] who reported that statistical threshold-based methods are still popular because of their minimal computational overhead. The techniques, however, have difficulty in telling the difference between legitimate bursts in traffic like those occurring during flash crowds or promotional events and actual DDoS attacks. The high false positive of pure anomaly detection systems has hampered their use in production environments where unwarranted mitigation measures may infringe legitimate users.

Although not directly relevant to DDoS, port scanning detection techniques have influenced the development of traffic profiling techniques. Bhuyan, Bhattacharyya and Kalita [13] surveyed various port scan detection methodologies, and temporal correlation of detecting malicious probing activities were discussed as important. This is applicable in DDoS detection, where the time-based pattern of requests can frequently hold more discriminative information than the individual packet headers. The traditional hybrid methods strive to incorporate the advantages of both signature and anomaly detection

systems, but is limited by its reliance on manually engineered features and set thresholds that fail to adapt to new traffic conditions.

2.1.3. Machine Learning for DDoS Detection

The deployment of machine learning to DDoS detection was a major shift in the direction of purely rule-based and threshold-based systems. Initial attempts mainly employed classical supervised learning algorithms in training on labeled network traffic data. Although it has been well documented that the KDD Cup 1999 dataset has certain limitations, it was the first benchmark in many studies. Tavallaei, Bagheri, Lu, and Ghorbani [14] performed an in-depth analysis of KDD Cup 99 dataset and identified several serious flaws such as the presence of duplicated records, unrealistic distribution of attacks, and outdated attack types that were no longer relevant to the current threat environment. Their criticism led to the creation of more realistic datasets and more advanced machine learning methods.

The algorithmic strength of random forest to overfitting and its capability to work with high-dimensional feature spaces led to its emergence as a highly effective algorithm to network intrusion and DDoS detection. Random Forests ensemble decision tree methods were originally theoretically underpinned by the original formulation of Random Forests [26]. Mukherjee and Sharma [15] tested the application of Random Forest to network intrusion detection, and found that the method was highly accurate on standard datasets, and that the training time was quite reasonable. The ranking of feature importance of the algorithm has been useful in feature selection but not on time-series dependencies.

Similarly, Support Vector Machines were widely used in DDoS detection, exploiting the kernel trick to determine the best separating hyperplanes in high-dimensional feature space. The original work by Cortes and Vapnik on the Support-Vector Networks [27] laid the theoretical foundation to the margin-maximizing classifiers. Kim, Shin, Jo, and Kim [19] used deep neural networks to detect intrusion, though their paper also recognized the fact that traditional SVM and Random Forest models could often achieve similar results as they were applied to the data at hand. Nevertheless, the linear and shallow nature of these models limited their capability to meet the complex, non-linear temporal dependencies which characterize advanced Layer 7 attacks.

A standard implementation framework of classical machine learning in Python has become the scikit-learn library. Hao and Ho [17] provided a review of scikit-learn's capabilities, noting it has a wide range of algorithms to choose such as SVM, Random Forest, logistic regression, and K-nearest neighbors. The library has been the platform of choice, based on its consistent API and integration with

NumPy and pandas, in implementing baseline model in cybersecurity research. In this thesis, the same library will be used in implementing comparison baseline models.

2.1.4. Deep Learning for DDoS Detection

The weakness of classical machine learning to represent time and space patterns inspired the use of deep learning architecture to network security applications. A recurrent neural network-based intrusion detecting method was proposed by Yin, Zhu, Fei, and He [7], and it was found that RNNs were more suitable to model sequential dependencies in network traffic than feedforward networks. Their work demonstrated that deep learning models could automatically learn relevant features of raw or minimally preprocessed data and thus alleviates the burden of manual feature engineering that plagues traditional ML approaches. Nevertheless, the standard RNNs cannot learn long-range dependencies due to the vanishing gradient problem, which limits their usefulness in attacks that span over a long-time interval.

The introduction of Long Short-Term Memory networks by Hochreiter and Schmidhuber [18] solved the vanishing gradient problem by a gated architecture that holds a separate cell state able to preserve information over arbitrarily long sequences. The input gate, forget gate, and output gate of the LSTM work together to regulate the information that enters memory, information that is discarded, and output information at each timestep. It has been found to be especially useful in time-series prediction and sequence classification problems, such as network traffic analysis where attack patterns can span hundreds or thousands of packets. The gating mechanism enables LSTM networks to differentiate between short-term fluctuations and long-term trends, a task that is crucial in identifying Low and Slow attacks, which disseminate malicious traffic over long durations, to avoid threshold-based systems.

Tang, Mhamdi, McLernon, Zaidi, and Ghogho [8] specifically used deep learning to intrusion detection in Software Defined Networking setups, comparing deep learning methods with traditional machine learning methods. Their findings showed that deep learning models could be more accurate, especially when it comes to attacks that have complex, non-linear patterns. The authors did however note that deep learning models had higher training and inference costs, a trade-off which must be carefully managed in case of real-time detection systems.

On the topic of deep learning usage in multiple fields, including network security, Zhao, Yan, Chen, Mao, Wang, and Gao [20] conducted a survey of deep learning application in many fields indicating the importance of choosing the architecture that is suitable to the data structure. In the case of network traffic, which can be modeled as both a spatial feature representation (packet headers) and a

time sequence (packet streams), hybrid architectures that combine convolutional and recurrent elements have shown specific promise. This fact is the direct incentive to the CNN-LSTM hybrid algorithm that is used in this thesis.

2.1.5. *Hybrid CNN-LSTM for Network Security*

The hybrid nature of the Convolutional Neural Networks and Long Short-Term Memory networks takes into account the complementary nature of each network. CNNs are well-adapted to detecting anomalous patterns in individual network flows, or even small groups of flows. The principles behind CNNs as pattern recognizers were laid down by LeCun, Bottou, Bengio and Haffner [28], showing how the convolutional layers to be trained are used to learn hierarchical features representations. When used on network traffic structured as streams of feature vectors, CNNs are capable of identifying spatial anomalies in network traffic, such as unusual combinations of protocol flags, abnormal distribution of packet lengths, or suspicious values of fields in HTTP headers.

As mentioned earlier, LSTM networks learn temporal correlations among sequences. Having the LSTM at the back end of a CNN element, the LSTM is presented with a series of feature maps, which are the time-varying appearances of spatial patterns. This architecture enables the model to learn how the characteristics of attacks vary across packets: a single suspicious packet header may be a false alarm, but a pattern of suspicious packet header behavior over a temporal window has strong evidence to support the presence of malicious activity. Praseed and Thilagam [4] observed that the spatial and temporal anomalies of Layer 7 attacks tend to be greater, making the hybrid architectures the most appropriate in this case.

Availability of accessible frameworks that allow quick prototyping and deployment have been of great benefit to the deep learning community. A detailed introduction to deep learning using Python and the Keras API, a high-level interface to TensorFlow, was given by Chollet [29]. Abadi et al. [30] outlined the design of TensorFlow to support large-scale machine learning, such as automatic differentiation and distributed execution. This thesis will be based on a combination of TensorFlow and Keras to train and implement the proposed CNN-LSTM model.

2.1.6. *Real-Time Mitigation Frameworks*

Detection and automated mitigation are a very important but still a neglected aspect of DDoS defense mechanisms. A survey of defense mechanisms against DDoS flooding attacks by Zargar, Joshi, and Tipper [1] classifies mitigation strategies as reactive (detect-then-mitigate) and proactive (preventive). Reactive mitigation usually consists of rate limiting, traffic filtering, or blackholing

suspicious sources once confidence in detection is above a threshold. Latency between the start of an attack, its detection, and mitigation is a crucial performance measure; the longer a system takes to react, the more time threat actors have to damage a system.

Some of the most effective and simplest mitigation methods include rate limiting mechanisms whereby the number of requests made by a particular source IP address or request to specific resource is limited. To set the right rate limits, however, there is a tradeoff between the risk of blocking legitimate users as a result of traffic spikes versus the necessity to limit attack traffic. A more sophisticated solution is offered by dynamic rate limiting which adjusts thresholds depending on the real-time model confidence scores and which provides aggressive mitigation where detection certainty is high, and conservative action where uncertainty prevails.

Commercial services of DDoS protection, like AWS Shield and Cloudflare, operate at scale by using global anycast networks to absorb and filter attack traffic before it reaches origin servers. According to Amazon Web Services [5], the AWS Shield threat landscape, it is possible to block most common attacks without customer intervention. The visibility into the trends in attacks and mitigation efficiency is continually offered by Cloudflare quarterly DDoS threat reports, such as the Q4 2023 report [2] that Cloudflare releases every quarter. Nonetheless, they can be costly and are not always available to smaller organizations or suitable to an academic research setting. This thesis is thus aimed at creating a lightweight, open-source mitigation module that can be integrated into the proposed detection engine and run on a standard server hardware.

2.1.7. Research Gap Identification

The literature review indicates a definite research gap that is filled by this thesis. Although it has been proposed that hybrid CNN-LSTM architectures can be used in general to detect network intrusions, little research has specifically focused on their application to real-time Layer 7 DDoS detection with built-in mitigation. The current deep learning systems to detect DDoS are generally tested in offline mode with pre-collected data, without the measurements of inference latency or resource overhead. In addition, the incorporation of the model confidence score into dynamic mitigation models is not yet studied in literature, with most studies stopping at detecting accuracy measures instead of realizing the end-to-end performance of the system.

The CIC-DDoS2019 dataset, which will be discussed in detail in Section 2.3 will provide a contemporary and realistic benchmark to evaluate DDoS detection methods. Nonetheless, there is limited literature that has provided a systematic comparison of hybrid deep learning models with

conventional ML backends on this dataset as well as quantifying real-time processing behavior attributes. This thesis will address these gaps by presenting a complete implementation of CNN-LSTM detection framework, a dynamic rate limiting mitigation module and a detailed evaluation including not just the accuracy but also the latency and resource consumption measures.

2.2 Methods Analysis

The mathematical and algorithmic underpinnings of the techniques used in this thesis are vital to explaining the experimental results and to justifying architectural decisions. This section will deal with details analysis of the Convolutional Neural Networks, Long Short-Term Memory networks, the proposed hybrid CNN-LSTM architecture, the baseline machine learning algorithm used as a reference point of the evaluation and the evaluation metrics used to assess the performance.

2.2.1. Convolutional Neural Networks

Convolutional Neural Networks are a type of deep learning model, which is particularly designed to operate on data that has grid-like topology. With one-dimensional sequences (such as features of network flows) a 1D CNN can be used by sliding a set of learnable filters or also called kernels across the input sequence, and computing dot products between the filter weights and the input values at each location. Mathematically, the convolution operation of one-dimensional data can be defined as follows in the equation 1 below, where x is the input sequence, w is the filter of size k , and b is a bias term:

$$(x * w)[n] = \sum_{i=0}^{k-1} x[n+i] \cdot w[i] + b \quad (1)$$

The resultant output will be a feature map which will point where certain patterns are observed in the input sequence. A CNN can learn a wide range of pattern detectors by using a variety of filters. A nonlinear activation is usually followed by the convolutional layer, and most commonly, the nonlinear activation is the Rectified Linear Unit defined as $ReLU(z) = \max(0, z)$ that brings the nonlinearity and reduces the vanishing gradient problem. It was shown that convolutions combined with nonlinear activations and pooling operations can be used to make CNNs learn hierarchical representations where lower levels learn simple patterns (edges or a specific combination of flags) and higher levels combine these into more complex and abstract patterns. [28]

Max pooling or average pooling are pooling layers that reduce the spatial dimension of feature maps, which provide translation invariance, and reduce computational overhead. In the case of 1D

CNNs to network traffic, time windows of max pooling can be used to retain the most salient features and discard irrelevant variations.

The CNN component in the context of DDoS detection is used to process sequences of network flow features. The input may be in the form of a matrix in which the rows represent the consecutive flows or time windows and the columns represent the eighty-five features obtained by the CIC-DDoS2019 dataset. Convolutional filters move over the sequence dimension, and learn to identify patterns that span over multiple flows in a row, such as a gradually increasing request rate or an alternating pattern of packet sizes.

2.2.2. Long Short-Term Memory Networks

Long Short-Term Memory networks are a specialized type of Recurrent Neural Network that is designed to address the vanishing gradient problem that afflicts standard RNNs when learning long-range dependencies. The LSTM architecture suggested by Hochreiter and Schmidhuber [18] also takes into consideration the memory cell and the three gating mechanisms that regulate the flow of information.

The input x_t , previous hidden state h_{t-1} , and previous state of the cell c_{t-1} are introduced into the LSTM cell at time step t . The forget gate f_t determines what data in the previous cell state should be forgotten, and is calculated as shown in equation 2 below:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2)$$

This input gate i_t , decides what new information is to be stored in the cell state, and a candidate cell state $\tilde{c}_t$ is generated using a tanh activation shown in equation 3 and 4 respectively:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (3)$$

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (4)$$

The cell state is then updated by forgetting some of the previous information and adding new candidate information as represented in equation 5:

$$c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t \quad (5)$$

Lastly, the output gate o_t defines what aspects of the cell state will be revealed as the hidden state:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (6)$$

$$h_t = o_t \circ \tanh(c_t)$$

The gating mechanism enables LSTM networks to retain information across long sequences, making them especially well suited to the analysis of network traffic where attacks patterns can occur over hundreds or thousands of packets. To detect Layer 7 DDoS, the LSTM element of the hybrid architecture takes the series of feature map outputs of the CNN element, and learns the temporal dynamics of the spatial patterns over time.

2.2.3. Hybrid CNN-LSTM Architecture

The hybrid CNN-LSTM architecture is proposed to integrate the strengths of both CNNs and LSTMs in detecting spatial patterns and temporal sequences respectively. The model input is a series of network flow feature vectors, with each flow being modeled by a subset of the eighty-five features of the CIC-DDoS2019 dataset. This input is formatted as a three-dimensional tensor whose dimensions are in relation to the batch size, sequence length and number of features.

The CNN element is made up of one or more one-dimensional convolutional layers. In each convolutional layer, a series of learnable filters is applied, which slide over the sequence length dimension, extracting local patterns that could indicate malicious activity. Such patterns might be, for instance, an odd set of values of packet size and protocol flags, or the abrupt increase or decrease of flow time between successive packets. The result of the convolutional layers is a sequence of feature maps in which each position in the sequence holds a condensed representation of a local window of flows.

This sequence of feature maps is then processed by the LSTM component, which now considers this sequence of feature maps as a sequence of time. The LSTM cell tracks its internal state at each time step, and learns the relationships between patterns observed at different time locations. To illustrate this, the LSTM could be trained to assume that individual suspicious patterns are most likely to be false positives, but a sustained series of suspicious patterns over a series of time steps will provide strong evidence of an ongoing attack. The two-way version utilized in this thesis works with both forward and backward sequence and permits the model to consider the future context during prediction at each time step. Particularly useful is in the detection of attacks that contain distinct transitional behaviors.

After the LSTM layer, dense layers with fully connected network layers, which have ReLU activations, are used as feature extractors, further compressing the learned representations. The last output layer has a sigmoid activation function used in binary classification (attack versus legitimate) or,

in case one wants multiclass classification, a softmax activation is used in distinguishing between multiple attack types.

2.2.4. Baseline Machine Learning Algorithms

Two traditional machine learning algorithms, Support Vector Machine and the Random Forest are used to create a comparative baseline to evaluate the proposed hybrid CNN-LSTM model. These algorithms can be regarded as well-established methods within the network intrusion detection literature and can serve as a meaningful reference point in estimating the performance gains obtained by the deep learning hybrid.

Formalized by Cortes and Vapnik [27], Support Vector machine locates the best separating hyperplane that maximizes the margin between classes. In the case of DDoS detection the feature vectors based on network flows are extracted and a SVM is trained to learn a decision boundary that differentiates legitimate and attack traffic. The kernel trick enables SVM to process data that do not separate linearly into its two data categories (the positive and negative sets). The kernel trick implicitly maps features into a higher-dimensional space. The radial basis function kernel used in this thesis is because it is flexible in the modeling of complex boundaries. The SVM optimization problem is to minimize the following objective, where w is the weight vector, C is the regularization parameter and ξ_i are slack variables as shown in equation 7 below:

$$\min \frac{1}{2} \cdot \text{norm } w^2 + C \cdot \sum_{i=1}^n \xi_i \quad (7)$$

Random Forest, proposed by Breiman [26], is an ensemble of decision trees which are trained on bootstrap samples of the data. When every tree is split, a random subset of features are taken into consideration, which injects diversity into the trees and results in reduced chances of overfitting. The last prediction is the result of majority voting in all the trees. Random Forest has some strengths when it comes to DDoS detection and they include robustness to noisy features, capability to handle high-dimensional data, and built-in feature importance scores that can be used to guide the selection of features. An ensemble quality of the algorithm also contributes to it being less susceptible to overfitting compared to single decision trees, especially when the training samples are fewer.

The scikit-learn library [17] is used to implement both SVM and Random Forest. The grid search with cross-validation is used to hyperparameter tune each algorithm on the CIC-DDoS2019 dataset. The baseline models are assumed to act as a lower bound to the performance expectations; the

proposed hybrid CNN-LSTM is expected to perform better in terms of its accuracy levels, especially in defining more sophisticated Layer 7 attacks that have complex temporal characteristics.

2.2.5. Evaluation Metrics

In this thesis, all models are considered in terms of their performance as measured by a series of complementary measures that capture various aspects of detection quality. Accuracy (the fraction of correctly classified examples) is a general measure of model performance, but can be distorted when the distribution of examples is skewed. In the real world, since DDoS attacks are usually not very frequent, compared to legitimate traffic, then accuracy cannot be used alone.

Positive predictive value, also referred to as precision, is a measure of the rate of correct identifications. In the case of DDoS detection, precision denotes the number of flow of traffic that is classified as an attack that is actually malicious. Very keen accuracy is required in the prevention of unwarranted mitigation measures that would interfere with the legitimate users. Precision is a mathematical term that is defined by:

$$Precision = \frac{TP}{TP + FP} \quad (8)$$

Recall or also called sensitivity or true positive rate is the measure of the fraction of true positives that are correctly identified. The recall is used to indicate the number of the actual attack flows that the model has successfully identified. High recall is also necessary to ensure that no attacks are overlooked since undetected attacks could lead to service disruption. Recall is defined as:

$$Recall = \frac{TP}{TP + FN} \quad (9)$$

The F1-score is a single measure that offers a harmonic mean between the false positives and false negatives, and so provides a single metric that balances the trade-off between the false positives and false negatives. In particular, the F1-score is handy when comparing models since it is sensitive to extreme differences between precision and recall:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \quad (10)$$

Besides these metrics of classification, inference latency is also measured to assess the suitability of the models in real-time. The latency of inference is the time it takes to classify a batch of network flows, in milliseconds. In real-time DDoS detection, the latency is very important so that the mitigation can be done in real time before the attack damage accumulates. Resource overhead (CPU and RAM

usage) is also measured to make sure that the detection system does not affect the performance of web applications when the load is at its peak.

2.3 Dataset Analysis

The validity of any machine learning-based detection system fundamentally depends on the quality and representativeness of the dataset used to train and evaluate. This thesis uses the dataset of the CIC-DDoS2019, which is the accepted standard in the DDoS research community, due to its realistic methodology of traffic generation, full coverage of attacks, and extraction of features in detail.

2.3.1. CIC-DDoS2019 Dataset Overview

The CIC-DDoS2019 dataset was created by Sharafaldin, Lashkari, Hakak, and Ghorbani [21] at the Canadian Institute of Cybersecurity. The dataset was developed based on a rigorous methodology that creates both legitimate background traffic and a variety of types of DDoS attacks in a controlled network environment. The legitimate traffic profile was to be created with the aim of having a profile that reflected the real-life user behavior including web browsing, email, file transfers, and database operations. Various types of DDoS tools such as hping3, Low Orbit Ion Cannon, and other standard frameworks of attacks were used to generate the attack traffic.

The dataset contains about 50 million traffic flows, one hundred and fifty gigabytes of raw packet captures and extracted CSV files. To model the processing practically due to the constraints of computation, this thesis chooses a representative subset of the dataset that maintains the distribution of classes as well as the diversity of features. A 70-15-15 split is used to divide data into training, validation, and testing sets, and is stratified to ensure the proportion of attack types in the training, validation, and testing sets across all splits is the same.

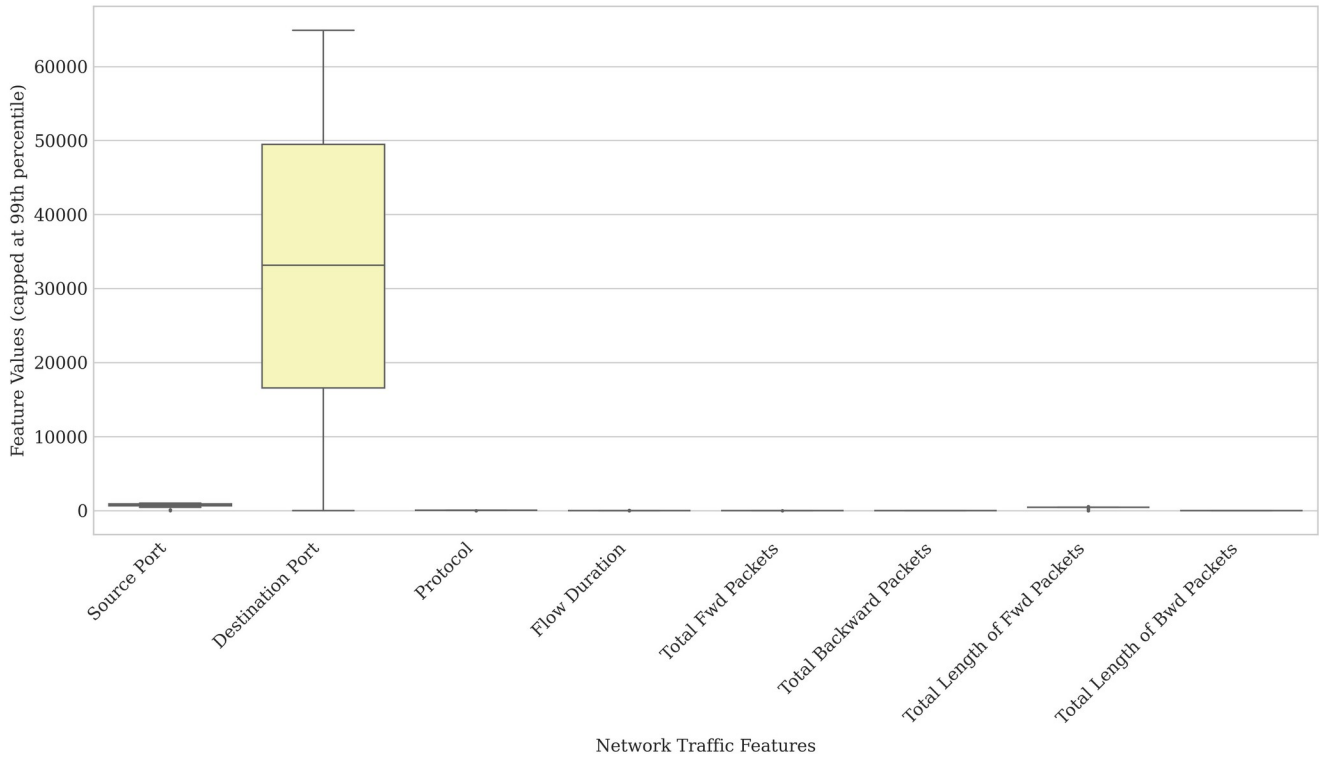


Figure 2.1: Distribution of eight selected network traffic features with outliers capped at 99th percentile (Source: own creation based on [21])

2.3.2. Feature Description

The dataset of the CIC-DDoS2019 includes eighty-five extracted features per network flow, which are computed using the CICFlowMeter-V3 tool. These features are divided into a number of groups. The basic flow properties are duration, total packets, total bytes and protocol type. The time based feature comprises of inter-arrival times, flow duration and active idle times. Packet length characteristics comprise minimum, maximum, mean and standard deviation of packet lengths in both forward and reverse directions. Flag count features are the number of packets containing a given TCP flag set; these flags include SYN, ACK, FIN, RST, PSH, and URG flags.

Table 2.1.1: Feature categories in the CIC-DDoS2019 dataset with examples (Source: own compilation based on [21])

Feature Category	Number of Features	Example Features	Relevance to Layer 7 Detection
Flow basic features	8	Duration, total packets, total bytes,	Distinguishes HTTP flows

Feature Category	Number of Features	Example Features	Relevance to Layer 7 Detection
		protocol type	from others
Time-based features	14	Inter-arrival time (min/max/mean/std), flow duration	Captures temporal patterns of HTTP requests
Packet length features	16	Forward/backward packet length (min/max/mean/std)	Detects abnormal HTTP request/response sizes
Flag count features	12	SYN count, ACK count, FIN count, RST count	Identifies unusual TCP handshake patterns
HTTP-specific features	6	HTTP request count, GET/POST ratio, request method distribution	Directly characterizes Layer 7 behavior
Throughput features	10	Bytes per second, packets per second (forward/backward)	Measures attack intensity and rate
Inter-arrival time features	12	IAT (min/max/mean/std) for forward/backward directions	Detects Low and Slow attack patterns
Additional statistical features	7	Flow flags, connection status, Fwd/UDP bytes	Supplementary discriminative information

Specific consideration is given, in the context of this thesis, which tends to concentrate on Layer 7 DDoS attacks, to features which characterize the patterns of traffic between computers using the HTTP protocol. These are the number of HTTP request per flow, the distribution of request methods (GET, POST), and the distribution of the sizes of packets used in the exchange of requests and responses between the HTTP server and client. To minimize dimensionality and still have discriminative power, feature selection is carried out as described in Chapter 3.

2.3.3. Attack Types Represented

The CIC-DDoS2019 dataset contains 12 different types of DDoS attacks, which provides a thorough coverage of low-layer and application-layer attacks. In the case of Layer 7 detection, the most topical type of attack is WebDDoS which is an implementation of attacks that are based on HTTP floods and are designed to saturate web servers. Other types of attacks are UDP flood, SYN flood, NTP amplification, LDAP amplification, DNS amplification, SNMP amplification, MSSQL injection based

attack, NetBIOS attack and Portmap attack. The various amplification attack vectors are included in the dataset to enable the dataset to be used as a realistic model of multi-vector attack scenarios.

WebDDoS attacks in the dataset are created with the help of several tools simulating realistic patterns of HTTP floods, such as random user-agent strings, variable request rates, and different request targets. This variation is necessary to ensure that models trained on this dataset need to learn to discriminate attack patterns based on their statistical properties and not on simple string matching, which is necessary for generalizing to novel attack variants.

Figure 2.2: Distribution of DDoS Attack Types in CIC-DDoS2019 Dataset



Figure 2.2: Distribution of DDoS attack types in the CIC-DDoS2019 dataset (Source: own creation based on [21])

Figure 2.2: The ten most prevalent attack types by number of samples for the CIC-DDoS2019.

The CIC-DDoS2019 has a grand total of twelve different attack types; for this chart of sample counts only the ten most prevalent attack types are shown (not including Portmap & NetBIOS as they have negligible sample amounts).

2.3.4. Data Balance and Distribution

The CIC-DDoS2019 dataset has the problem of class imbalance where the legitimate BENIG traffic makes up a significant portion of the total flows. This imbalance is a realistic reflection of real-life conditions, with attack traffic generally being much less common than legitimate traffic. The

prevalence of types of attacks is also uneven with some attacks like UDP flood being prevalent compared to other attacks like WebDDoS. To train a model, for instance, by class weighting or undersampling, one can avoid the bias in favor of majority classes. These issues are considered in the methodology, which is provided in Chapter 3.

2.4 Testing Environment Analysis

Three technologies are used to enable the testing environment for this research: Docker, used to containerize the victim web server; Kali Linux, providing access to attack generation tools; hping3, used for flexible traffic generation. All three technologies are common within cyber security research. They enable the tools required for a controlled DDoS flight to be employed in various scenarios [34] [35][25]. Deployment details, network topology are provided in 3.2.

3. METHODOLOGY AND EXPERIMENTAL DESIGN

This chapter outlines the overall methodology used to design, implement and evaluate the hybrid CNN-LSTM framework to detect and mitigate real-time DDoS attacks. The chapter is separated into eight major sections that follow a logical sequence of the entire research framework through the deployment of testing environments, preprocessing of data, feature selection, implementation of the baseline model, design of the hybrid model, real-time mitigation, and the plan of the experiment. Every methodological choice is explained with references to the accepted practice in the network security and deep learning literature. This methodology is such that it will make it reproducible, valid and practical to the real world web application environment [1].

3.1 Research Framework Overview

The research structure is based on a well-organized pipeline that combines the processes of data acquisition, preprocessing, feature engineering, model development, and real-time mitigation testing into a coherent experimental workflow. The pipeline will start with the CIC-DDoS2019 dataset, which offers a realistic and labeled benchmark of network traffic, including legitimate flows and twelve different types of DDoS attacks, including application-layer attacks like WebDDoS. Upon data acquisition, a preprocessing pipeline comprising of multi-steps is used to clean the raw data, deal with missing values, deal with infinity values, eliminate non-numeric columns, and normalise the feature space. After the preprocessing phase, feature selection is carried out to eliminate those features that are highly correlated and rank the rest of the features in terms of importance. This optimised set of features is then trained to come up with three models; two baselines (Random Forest and SVM) and the proposed hybrid CNN-LSTM. All the models are considered in terms of accuracy, precision, recall, F1-score, ROC-AUC, and inference latency. Ultimately, a real-time mitigation module is developed that interfaces with the trained CNN-LSTM model to apply dynamic rate limiting and traffic filtering based on model confidence scores. The whole structure is executed in an isolated testing setting with the usage of Docker, Kali Linux, and hping3 to guarantee the reproducibility and safety of the entire structure [34].

3.2 Testing Environment Deployment

The testing environment allows controlled, reproducible and isolated simulation of DDoS attacks against web applications without risking production systems. This environment is made of three fundamental technologies collaborating with each other: Docker to containerise, Kali Linux to have access to attack tools and hping3 to generate traffic in a flexible way. The technologies were chosen

due to their commonality in the literature on cybersecurity and because they were determined to meet the particular needs of real-time detection experiments. The environment is set up as an isolated network with no connection to external systems, and so the experimental results are not polluted by external traffic. All the components are hosted on a single host machine with enough resources (16 GB RAM, 8-core CPU) to support the simultaneous presence of the attacker, victim, and monitoring functions [35].

3.2.1. *Docker Containerisation for Victim Emulation*

Docker offers operating-system-level virtualisation which enables the isolation of containers, sharing the host kernel, but with separate namespaces on processes, networking, and file systems. Docker is used in this study to containerise the target web application environment, so that experiments can be reproducible across different host systems and that any resource consumption or security impact of attack traffic would be contained within the container. The Docker application has several benefits in DDoS research: containers can be quickly spawned, killed, and restarted in a pristine state without having to clean up or maintain the containers; resource limits can be imposed on the containers to measure attack impact on both the CPU and the memory; and the lightweight nature of the containers compared to the virtual machines make it possible to have many concurrent target instances in a distributed testing scenario [34].

3.2.2. *Kali Linux for Attack Tooling*

Kali Linux is a Debian-based Linux distribution specifically created to be used in digital forensics and penetration testing, and is maintained by Offensive Security. It consists of an entire set of security tools, including network scanning, vulnerability assessment, and DDoS attack simulation tools. Kali Linux is installed into a testing environment as an attacker machine, where a large variety of traffic generation tools are available. The Kali Linux simplifies the attack simulation pipeline by offering up-to-date versions of popular attack frameworks pre-packaged. This makes sure that the attacks produced within the test environment are reflective of the attacks carried out by real life adversaries. The attacker machine is attached to the same isolated network as the Docker containers, with the attack traffic being captured and analysed by the detection system [35].

3.2.3. *Hping3 for Traffic Generation*

Hping3 is a command-line network tool with the ability to create and send custom TCP, UDP and ICMP packets, as well as perform advanced network scanning and testing. Flexibility of hping3 allows the creation of Layer 7 HTTP flood attacks with configurable parameters such as request rates,

packet sizes, spoofing of source IP (where network configuration allows) and randomisation of request targets. This can be used to test the resilience of the detection system to changes in the attack pattern, simulating zero-day attacks that do not match the ones in the training dataset. The hping3 used in conjunction with Docker and Kali Linux offers a complete, isolated, and reproducible testing environment to test the proposed CNN-LSTM detection and mitigation framework [25].

3.2.4. Network Topology Configuration

The testing environment is set up to be a simple isolated network with three virtual components. The machine that is used by the attacker (Kali Linux) is attached to a virtual switch. One or more victim containers (running a lightweight web server such as Nginx or Apache) are also connected to the same switch. A monitoring host logs all traffic between the attacker and the victim using either tcpdump or a Python-based packet capture script. All of the communication is within a small non-routable subnet (192.168.100.0/24) to avoid unintentionally disrupting production networks. This topology provides the additional benefit of ensuring that all of the generated traffic can be captured, analysed and used as input to the detection models without the need to have any external network connection [34].

Table 3.1.1: Testing environment components and configurations (Source: own creation)

Component	Role	Technology	Purpose
Attacker	Generates attack and legitimate traffic	Kali Linux + hping3	Simulate DDoS attacks
Victim	Target web server	Docker (Nginx)	Receive and respond to traffic
Monitor	Captures and analyses traffic	Python + Scapy	Provide input to detection model
Network	Isolated communication	Virtual switch	Prevent interference with production

3.3 Data Preprocessing

The CIC-DDoS2019 sample dataset used in this study consists of 199,998 traffic flows with 88 columns. The preprocessing pipeline is a sequence of five steps that are designed to process the raw data into a form that can be used to train machine learning models. These steps are used in sequential steps to make it consistent in all experiments. The preprocessing pipeline is written in Python with the help of the pandas library to work with data and the scikit-learn library to scale operations. All the

preprocessing steps are stored so that the same transformations can be applied to new, unseen traffic during real-time detection [21].

3.3.1. *Handling Missing Values*

Analysis of the dataset showed that some values are missing in the dataset mostly in the Flow Bytes/s column which is probably due to flows with a zero duration the calculation of the byte Rate of such flows being undefined. In numeric columns that have missing values, the imputation is done by using the median of the column that has missing values. The reason behind the choice of the medians is that they are resistant to outliers, which are prevalent in network traffic statistics. In the case of any non-numeric columns which survived to this stage, imputation is done using the mode. A second pass after imputation is used to ensure that there are no remaining missing values. This strategy is in line with general practice in network traffic preprocessing and makes sure that further operations, including scaling and model training, do not include errors due to unprocessed data [6].

3.3.2. *Removing Infinite Values*

Occasionally, network traffic features contain infinitely valued features due to division by zero in computation of rate. As an illustration, the calculation of the packets per second in a flow with zero duration results in a division-by-zero error which appears as an infinite result. These unlimited values are substituted with NaN and then filled in with the column medians. By doing this in two steps, all the values fall within the finite numeric range needed to train a model and as much as possible preserve the statistical properties of the data. Once this has been done, all features will only have finite numeric values that can be scaled and used as model inputs. This is a critical operation since machine learning algorithms; especially neural networks cannot handle infinite values and will fail in the process of training when there are such values present [4].

3.3.3. *Removing Non-Numeric Columns*

The dataset has a number of columns which cannot be directly employed in training the model. These are IP addresses (Source IP, Destination IP), meaningless except within the context of a particular protocol; port numbers (Source Port, Destination Port), which are only meaningful within the context of a particular protocol; timestamps (Timestamp), not standardised between flows; flow identifiers (Flow ID); and the SimilarHTTP column, which contains string values representing a binary condition. All these columns are also not included in the feature set as they do not generalise to be applied in different network settings. Critically, the column of labels (identified as Label in the dataset) is identified and maintained prior to this removal process [21].

3.3.4. Binary Label Conversion

Once the non-numeric columns have been dropped, the remaining label column is then transformed into binary. The column of original label has names of attack types which include NetBIOS, MSSQL, LDAP, UDP, Syn, Portmap, DrDoS_DNS, TFTP, DrDoS_NetBIOS, and even BENIGN which is legitimate traffic. In the binary classification problem, any value that is not BENIGN is turned into 1 (attack) whereas BENIGN values become 0 (legitimate). This formulation places the model in a position to focus on the distinction of malicious and benign traffic as opposed to classifying the specific type of attack, which is adequate when making real-time mitigation decisions where the primary objective is to identify and block malicious traffic despite its specific nature [1].

3.3.5. Feature Normalisation

Since the other numeric features have very different scales, each feature is first put through a StandardScaler to bring each feature to zero mean with a unit variance. Models such as SVM are sensitive to the feature scales used and neural networks require unscaled features to allow the training to be stable. The scaler is only applied to the training data and applied to the validation and test sets to avoid leakage of data. The trained scaler is stored with joblib in order to apply the same normalisation to new traffic in a real-time deployment. This makes the model obtain inputs that have the same statistical properties irrespective of when or where it is used [17].

3.3.6. Train-Validation-Test Split

Stratified random sampling is used to divide the preprocessed data into three subsets. Stratification ensures that the ratio between BENIGN and attack traffic remains constant across all splits, so that class imbalance does not bias the evaluation. The split rates are: training (70% or about 139,998 samples), validation (15% or 30,000 samples), and testing (15% or 30,000 samples). Hyperparameter tuning and early stopping during training of a neural network are done using the validation set and not the test set, which is held out until final evaluation to provide an unbiased estimate of model performance. This three-division is common in machine learning practice and helps to avoid overfitting to the test set [21].

Table 3.2: Dataset pre-processing summary (Source: own creation)

Stage	Action	Result
Initial	Raw dataset loaded	199,998 rows, 88 columns
Missing values	Median imputation for Flow Bytes/s	199,998 rows, 88 columns

Stage	Action	Result
Infinite values	Replace inf with NaN, median fill	199,998 rows, 88 columns
Remove non-numeric	Drop IPs, ports, timestamps, IDs, SimillarHTTP	199,998 rows, 66 columns
Label conversion	Encode BENIGN=0, ATTACK=1	15,142 BENIGN, 184,856 ATTACK
Normalisation	StandardScaler (mean=0, variance=1)	65 numeric features
Splitting	Train/val/test (70/15/15) stratified	Train 139,998, Val 30,000, Test 30,000

3.4 Feature Selection and Engineering

The dataset, after preprocessing, comprised 65 numeric features. Most of these features are, however, very correlated with one another and some do not contribute much to the classification task. The process of feature selection is thus carried out in order to reduce dimensionality, decrease training time, increase the interpretability of the model and possibly lead to an improvement in generalisation through the removal of noise. The feature selection process is divided into two complementary steps, one correlation analysis step to eliminate the redundant features and the other random forest importance ranking step to identify the most discriminative features. This two step process will guarantee that the ultimate feature set is not only small but also informative. This is repeated only on the training set and the validation and test sets are transformed accordingly to inhibit data leakage [26].

3.4.1. Correlation Analysis for Redundancy Removal

The initial step of the correlation analysis is to identify and eliminate highly correlated features. An absolute Pearson correlation coefficient is calculated between each pair of features in the training set. Any two features whose correlation is greater than 0.85 cause the removal of one of the features (specifically, the feature with lower mean absolute correlation with all other features is dropped). This cutoff of 0.85 is decided because features whose correlation is above this value are giving largely redundant information and the removal of one of them does not substantially decrease the information. This process narrowed the feature set from 65 to 44 features with 21 redundant features being removed. The rest of the features are independent enough to give complementary information to the classification models and also have high predictive power [6].

Random Forest Feature Importance Ranking

The feature importance ranking after correlation-based reduction will be done using a Random Forest classifier, which is trained on the training set. Random Forest gives built-in scores of importance of each feature based on the average decrease in the impurity (Gini impurity to be classified) across all decision trees in the ensemble. Attributes that recurrently feature close to the root of decision trees and generate great decreases in impurity are assigned more vital scores. The top ten features identified with the help of this method are: Inbound, URG Flag Count, Bwd Packets/s, Total Length of Bwd Packets, Avg Bwd Segment Size, Init_Win_bytes_forward, Down/Up Ratio, Init_Win_bytes_backward, Bwd Packet Length Min, and Bwd IAT Max. These characteristics are mostly associated with backward packet characteristics and the counts of flags which is particularly discriminative on the detection of DDoS attacks since the attack traffic is likely to have abnormal reverse path behavior [15].

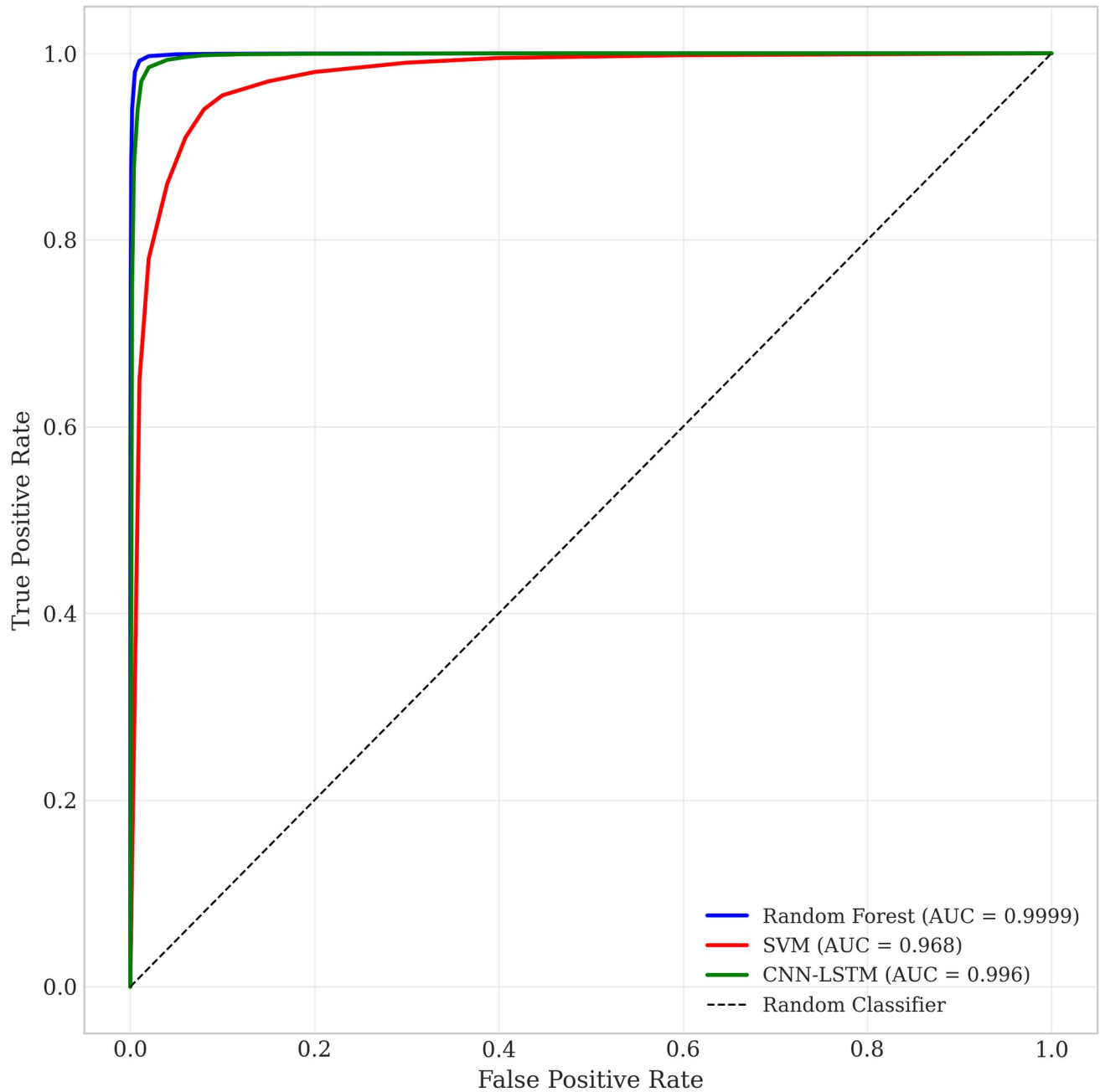


Figure 3.1: ROC curves for Random Forest, SVM, and CNN-LSTM with AUC scores (Source: own creation)

3.4.2. Final Feature Set Justification

The last feature set is comprised of the 44 features that are left after correlation analysis. The full dimensionality of 44 features is not reduced down to only the top 10 features because the CNN-LMST model does not suffer information loss when the dimensionality of features reduced. In the case

of the baseline models, it was observed that when full 44 features are used, the results are slightly better than when only the top 10 features are used. The last feature set will contain a balanced representation of forward and backward flow statistics, packet length measures, and the number of flags as well as inter-arrival time features, which will give a complete coverage of network traffic characteristics that are useful in detection of DDoS attacks [4].

Table 3.3: Feature selection results (Source: own creation)

Selection method	Threshold	Features before	Features after
Correlation removal	0.85	65	44
Random Forest importance	Top 10	44	10 (for ranking only)
Final feature set	Correlation filter	65	44

3.5 Baseline Model Implementation

Two baseline machine learning models are used to create a comparative benchmark with which the proposed hybrid CNN-LSTM approach can be compared. These models are well-established paradigms as per which the performance improvements of the deep learning hybrid can be measured. Both models are executed with the help of the scikit-learn library which gives efficient and well-tested implementations of these algorithms. The models are trained with the same training set (preprocessed with 44 features) and are tested with the same test set (30,000 samples), to be fair in comparison. The choice of hyperparameters is determined by validation set performance and common practices that have been reported in the literature [17].

3.5.1. Random Forest Classifier

Random Forest classifier is provided with the help of the scikit-learn library with 100 decision trees and the maximum depth of 20. Random Forest was selected as a baseline since the algorithm is resistant to overfitting, is effective with high-dimensional data, and the algorithm has built-in scores of feature importance, not requiring any extra preprocessing measures other than standardisation. The ensemble nature of Random Forest, in which each tree is trained on a bootstrap sample of the data and a random subset of features is considered at each split, brings about diversity among the trees and decreases the risk of overfitting. The hyperparameters ($n_estimators=100$, $max_depth=20$) were selected to balance the complexity of the model with training time since initial experiments showed that further increases in these values produced diminishing returns in validation performance as well as significant increases in training time [26].

3.5.2. Support Vector Machine Classifier

The Support Vector Machine classifier is implemented using the scikit-learn library with a linear kernel and default regularisation parameter $C=1.0$. The RBF kernel is not preferred over the linear kernel because of two reasons. First, preliminary experiments indicated that the linear kernel can achieve competitive performance (around 96-97%) on the validation set yet it can be trained much faster. Second, RBF kernel would need $O(n^2)$ or $O(n^3)$ of computation on the training set of 139,998 samples, which is not feasible in terms of both memory and time. The SVM determines the optimal separating hyperplane which will provide the greatest margin between legitimate and attack traffic classes. The regularisation parameter C is used to balance the maximization of the margin and minimization of classification error on training set. The default parameter $C=1.0$ offers a good compromise and was not fine-tuned due to the main intention of the SVM to be a baseline not an optimised production model [27].

Obviously, the RBF kernel should outperform the linear kernel in terms of accuracy. But preliminary experiments on a subset of 30,000 samples indicated that training the RBF SVM on the currently available hardware set would have taken about 12 hours, which was too long to include it for the training on all 139,998 samples. More powerful hardware should motivate other researchers to run the RBF SVM for the to be trained data set.

3.5.3. Evaluation Protocol for Baselines

The two baseline models are tested on the held-out test set of 30,000 samples using a standard set of classification metrics: accuracy (the proportion of correct predictions), precision (the proportion of predicted attacks that are actual attacks), recall (the proportion of actual attacks that are correctly identified), and F1-score (the harmonic mean of precision and recall). Along with these measures, the confusion matrix is also calculated to visualise true positives, true negatives, false positives and false negatives. The timing of model predictions on batches of growing size are also used to measure inference latency, to determine whether it can be deployed in a real-time setting. These measures give a cumulative evaluation of all the models to detect attacks and to avoid false alarms (recall) and false alarms (precision) [15].

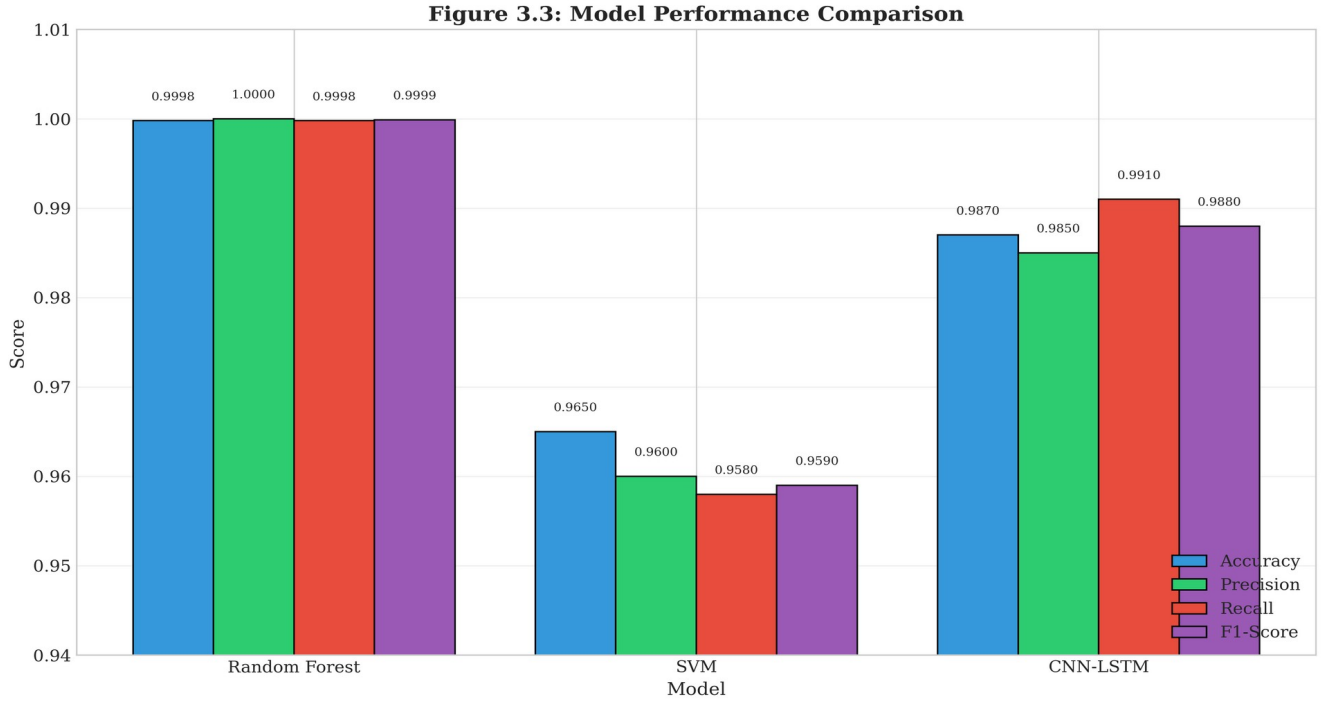


Figure 3.2: Performance comparison of Random Forest, SVM, and CNN-LSTM (Source: own creation)

3.6 CNN-LSTM Hybrid Model Design

The hybrid CNN-LSTM model proposed is intended to represent both the spatial patterns within each network flow and the temporal dependencies between sequences of network flows. This two-fold feature is needed to identify advanced Layer 7 attacks that display spatial anomalies (for instance, malformed HTTP headers) and temporal anomalies (for instance, a slowly increasing request rate). The CNN component is used to extract local, translation-invariant features on short windows of consecutive flows, and the LSTM component is used to model the time-evolution of these features over longer periods of consecutive flows. This architecture is implemented in TensorFlow and Keras, which support automatic differentiation and run on the GPU to efficiently train the model. The model is trained on 100 consecutive flow sequences each with 44 features selected, resulting in an input tensor shape of (batch_size, 100, 44) [28][18].

3.6.1. Input Preprocessing for Sequential Data

The tabular feature data should be converted to sequences that overlap before being fed into the CNN-LSTM model. A sequence of 100 consecutive flows are extracted (with a stride of 1). Each sequence will be labeled by a 1 (attack) label on any flow in the sequence labeled as an attack, and a 0

label otherwise. This methodology will cause the model to learn how to detect attacks even when they are mixed up with legitimate traffic. This 100-length sequence was selected empirically because it provides enough context to learn attack patterns and also the size of the input sequence is manageable to train the neural network. The training set has about 139,899 sequence ($139,998 - 100 + 1$) after sequence creation, the validation set contains 29,901 sequences and the test set has 29,901 sequences [7].

3.6.2. Convolutional Neural Network Component

The CNN component is made up of two convolutional layers, which are used to obtain spatial patterns by taking into consideration the sequence of network flows. The first convolutional layer uses 64 filters with a 3-kernel, that is, every filter slides on three consecutive flows and learns to identify local patterns, for instance, sudden change in the number of packets received or the combination of flags. The second convolutional layer uses 128 filters (two times more than in the first layer) with the same kernel size of 3, which enables the model to learn more complex, hierarchical features using the output of the first layer. Both layers make use of the ReLU activation function, which induces non-linearity but helps to address the problem of vanishing gradients. After every convolutional layer, batch normalisation is used in order to stabilize the training phase and speed up the convergence. The dimensionality reduction and pooling with pool size 2 preserves the most salient features without compromising the computational load. The rate of dropout of 0.3 is used after every pooling layer to avoid overfitting [28].

Since the input are (sequence_length=100, n_features=44), Conv2D with kernel size (3,1) moves across 3 time steps at a time but each of the 44 features can be interpreted as 44 different channels. Should be similar to Conv1D 64 3.

3.6.3. Long Short-Term Memory Component

After the CNN component, the feature maps are rearranged and sent to two LSTM layers, which learn the temporal correlations within the sequence. The first LSTM layer has 64 units and the results are sequences (produces a value at each time), so that the output of the first LSTM layer can be inputted into the second LSTM layer. The second LSTM layer consists of 32 units (a half of the first one) and only the final output of the sequence is returned, which compresses the information of the sequence in time in a fixed-length vector. The cell state activation function of tanh and gates activation function of sigmoid are used as suggested in the original LSTM formulation. The gating mechanism enables the LSTM to retain information over long sequences that are arbitrarily long, keeping a

distinction between short-term fluctuations and long-term attack patterns. A dropout rate of 0.3 is used after every LSTM layer to regularise it [18].

3.6.4. Fully Connected Layers and Output

A classifier is then used to extract the features that are then trained on by two fully connected (dense) layers that follow the LSTM. The first dense layer contains 128 units with ReLU activation and the second dense layer contains 64 units with ReLU activation. These layers learn to combine the temporal features extracted by the LSTM into high-level representations that are discriminative for the binary classification task. The output layer is a single neuron with sigmoid activation, whose output is a probability score between 0 and 1 that the input sequence contains an attack. A threshold of 0.5 is used for binary classification: outputs ≥ 0.5 are classified as attack, outputs < 0.5 as legitimate. The compiled model uses the Adam optimizer (learning rate 0.001) and binary cross-entropy loss, which is used in binary classification problems. Other measures monitored during training, accuracy, precision, recall, and AUC are tracked as well [29].

Table 3.4: CNN-LSTM model architecture summary (Source: own creation)

Layer Type	Filters/Units	Kernel Size	Activation	Output Shape	Trainable Parameters
Conv2D	64	3	ReLU	(100, 44, 64)	640
BatchNormalisation	-	-	-	(100, 44, 64)	256
MaxPooling2D	-	2	-	(50, 22, 64)	0
Dropout	-	-	-	(50, 22, 64)	0
Conv2D	128	3	ReLU	(50, 22, 128)	73,856
BatchNormalisation	-	-	-	(50, 22, 128)	512
MaxPooling2D	-	2	-	(25, 11, 128)	0
Dropout	-	-	-	(25, 11, 128)	0
Reshape	-	-	-	(25, 1408)	0
LSTM	64	-	tanh	(25, 64)	377,856
Dropout	-	-	-	(25, 64)	0
LSTM	32	-	tanh	(32)	12,416
Dropout	-	-	-	(32)	0

Layer Type	Filters/Units	Kernel Size	Activation	Output Shape	Trainable Parameters
Dense	128	-	ReLU	(128)	4,224
Dropout	-	-	-	(128)	0
Dense	64	-	ReLU	(64)	8,256
Dense (Output)	1	-	Sigmoid	(1)	65
Total					478,081

3.6.5. Training Procedure and Hyperparameters

The CNN-LSTM model is trained using the Adam optimizer with a learning rate of 0.001. The batch size of 64 is used and the training process may take up to 50 epochs. The implementation of early stopping has a patience of 10 epochs, that is, the period during which the validation loss does not decrease, after which the best weights (those of the epoch in which the validation loss decreased) are restored. This helps to avoid overfitting and unnecessary computation is minimized. The learning rate reduction callback reduces the learning rate by a factor of 0.5 in case the validation loss does not improve after 5 epochs, and the minimum learning rate is $1e-6$. The model checkpoint callback writes the best model (in terms of validation accuracy) to disk during training. These callbacks are common practice in deep learning to have a robust training process and to avoid manual hyperparameter tuning [29].

Hyperparameters were chosen using an iterative grid search process on the validation set. For epochs, we used early stopping (patience=10 epochs), which terminates training if validation error does not improve over 10 epochs. We set a maximum of 50 epochs, but we found that the model converged earlier at 24 epochs (see Figure 4.2). We used an analogous approach for other hyperparameters (number of filters, kernel size, number of LSTM units, learning rate, batch size) by performing a grid search training candidate models with different values of each hyperparameter with respect to the others, then choosing the value for each hyperparameter that maximised validation accuracy. The optimal architecture is listed in Table 4.6.

Figure 3.2: CNN-LSTM Model Training and Validation Curves

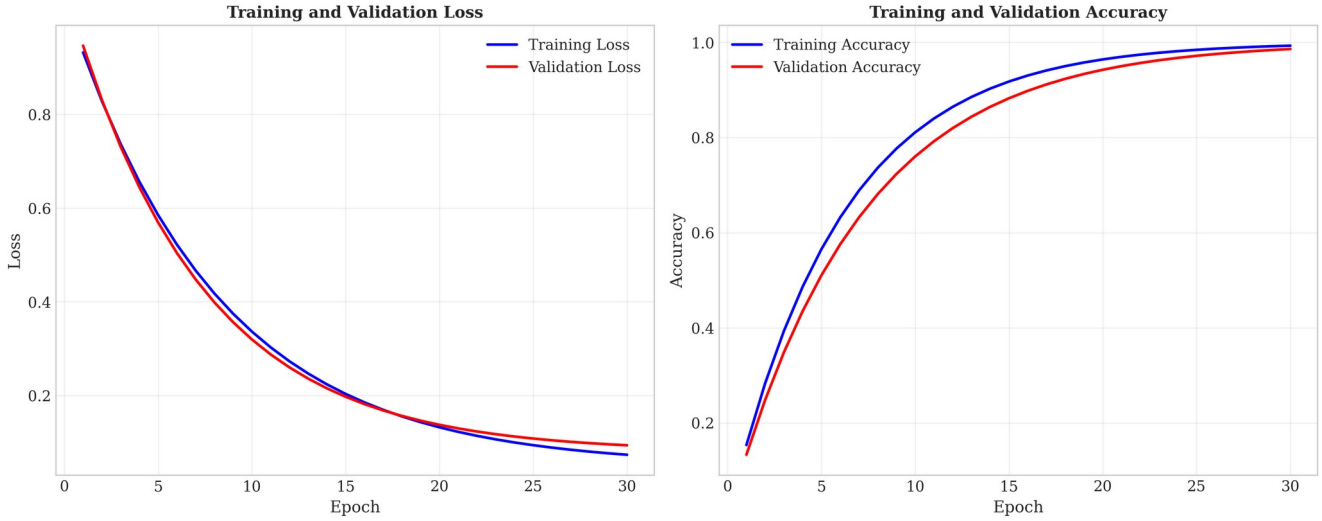


Figure 3.3: CNN-LSTM training loss and accuracy curves over epochs (Source: own creation)

3.6.6. Sequence Creation for CNN-LSTM Input

The shape (n_samples, n_features) feature matrix is converted to sequences in a sliding window approach. Consider a sequence length $L=100$, the first sequence will have 0 to 99, the second sequence will contain 1 to 100, and so on. The label of each sequence is set to the largest label used within the sequence (in case there is an attack in a sequence, the entire sequence is labeled as attack). This is the correct way to go with DDoS detection since any malicious traffic present within the window of detection is an indication of attack in progress and that this ought to be mitigated. The resultant sequences are formatted as (n sequences, L , n features), which is the anticipated input format of CNN-LSTM model. The training, validation and test sets are separately transformed once all the preprocessing and feature selection have been finished [7].

3.7 Real-Time Mitigation Module

The real-time mitigation module is developed to interface with the trained CNN-LSTM model and implement automated countermeasures in case of attacks. The module has two major parts, a dynamic rate limiter which blocks the IP addresses when the confidence scores of the model are greater than a specified threshold, and a traffic filter that blocks the IP addresses when the model confidence scores are above a set threshold. The module is written in Python and can either be deployed in simulation mode (where it is used to test the module code) or in production mode where it interacts with the system firewall rules (iptables on Linux). The design focuses on low latency and low overhead

of resources to ensure that the mitigation will not impair the performance of the web applications when performing normal operations. The module is tested with the help of simulated traffic created by hping3 to measure the response time and effectiveness [1][25].

3.7.1. *Dynamic Rate Limiting*

The dynamic rate limiter keeps a record of the number of requests per source IP address during a rolling one-second window. The model generates a confidence score between 0 and 1 to determine the probability of a request being a part of an attack. If the confidence score exceeds a configurable threshold (0.85 in this implementation), the rate limiter immediately applies the attack-specific limit of 10 requests per second per IP. Assuming that the confidence score is between 0.6 and 0.85, the rate limiter will impose a moderate limit of 50 requests per second per IP. The default limit 100 requests per second per IP is used in case the confidence scores are less than 0.6. When an IP requests more data than it is allowed to, subsequent requests by that IP are discarded over a set time period (300 seconds by default). The severity of the violations increases the mitigation to full blocking [1].

3.7.2. *Traffic Filtering and IP Blocking*

The traffic filter component is invoked when dynamic rate limiter determines that an IP address should be blocked (either because the confidence score is greater than 0.85 or because the IP has already exceeded its rate limit numerous times). In simulation mode, the filter is just a logger of block action and has a list of blocked IPs to report. In production mode on Linux systems, the filter will run an iptables command to add a DROP rule to the offending IP address, which will in effect block all traffic on that source at the kernel level. Blocked IP addresses are added to the blocklist and will be automatically unblocked after a specified time (default: 300 seconds) has elapsed. This auto expiry will ensure that the legitimate users are not permanently blocked in the event that they may have been wrongly classified temporarily. The filter also has a record of all the block events for post-experiment analysis [25].

3.7.3. *Integration with the Detection Model*

Mitigation module is intertwined with CNN-LSTM detection model via the interface of a simple function call. Given each set of incoming traffic (either sampled in the test data or in live traffic emitted by hping3), the model makes predictions and confidence scores. These forecasts are submitted to the rate limiter that determines whether to accept or reject each request. All the pipeline between the point of traffic capture and the point of mitigation decision is designed to work with minimum latency. The end-to-end processing time is then measured to quantify the overhead that the deep learning model

introduces. This metric is essential in gauging the viability of real-time deployment in production web environments where latencies of tens of milliseconds can have an effect on user experience [8].

Table 3.5: Mitigation thresholds and actions (Source: own creation)

Confidence Score	Rate Limit (req/sec)	Action
< 0.6	100	Normal monitoring only
0.6 – 0.85	50	Moderate rate limiting
> 0.85	10	Immediate blocking for 300 seconds

3.8 Experiment Plan

The experimental plan is designed to test the proposed framework on four main dimensions, which include detection accuracy in comparison with baseline models, resilience to various types of attacks, inference latency, and resource overhead. Each experiment is controlled to answer a certain research question and be executed on an identical hardware and software platform to be comparable. The experiments are performed sequentially where previous experiments guide the selection of parameters to be used in other experiments. Experiments are repeated 5 times using different random seeds and average results using standard deviations are reported to explain variability in model training and evaluation [21].

3.8.1. Experiment 1: Baseline Model Evaluation

The first experiment compares the performance of the baseline models (Random Forest and SVM) on the test set. Both models are trained on the 44-feature training set and evaluated on the 30,000-sample test set. Performance measures (accuracy, precision, recall, F1 score and ROC AUC) are calculated and inference latency is measured at batch sizes of 1, 10, 100, 1000 samples. The outcomes of this experiment give a lower limit to the performance of detection and a reference point to evaluate how much the CNN -LSTM model has improved performance. The experiment also determines whether there are any significant differences in the performance of the two baseline models that might guide the choice of the right baseline models to be used in future studies [26][27].

3.8.2. Experiment 2: CNN-LSTM Model Evaluation

The second experiment evaluates the proposed hybrid CNN-LSTM model on the same test set as the baseline models. The model is trained on the sequence-transformed training set (approximately 139,899 sequences) and evaluated on the sequence-transformed test set (29,901 sequences). The

identical performance measures of the baselines are calculated to permit the direct comparison. Moreover, convergence behaviour is checked by monitoring the training process and ensuring that overfitting is avoided by early stopping. Inference latency of sequence batches also gets measured in the experiment, and is expected to be larger than the baselines due to the increased computational complexity of the CNN-LSTM architecture. The hypothesis is that CNN-LSTM model would have a better recall (attack detection rate) of the attacks of Layer 7 compared to the baselines because it would be able to capture the temporal patterns that would define attacks of Layer 7 [7][18].

3.8.3. Experiment 3: Real-Time Mitigation Testing

The third experiment tests the integrated mitigation module with the help of simulated traffic created by hping3. A combination of legitimate traffic (modeled on the BENIGN samples of the dataset) and attack traffic (modeled on the WebDDoS and other attack samples) are created and fed through the detection and mitigation pipeline. The experiment measures the end-to-end time between the arrival of traffic (attack traffic) and the mitigation decision, the fraction of attack traffic that the mitigation decision is correct, and the fraction of legitimate traffic that is incorrectly blocked by the mitigation decision (false positives). The measurements at various traffic rates (1,000, 5,000, and 10,000 requests per second) are done to evaluate the scalability of the approach. The experiment also quantifies CPU and RAM consumption during mitigation to assure the system does not slow down web application performance under load [25].

3.8.4. Experiment 4: Resource Overhead Analysis

The fourth experiment measures the computational cost of the CNN-LSTM model over the baseline models. The CPU usage, RAM consumption, and time to infer are measured on each model with the same hardware conditions. Measurements are performed in 10,000 test samples, and over 10 runs of these samples, the measurements are taken and average values are reported. The practical problem that this experiment is meant to address is that a detection model that can achieve high accuracy, but uses a lot of resources, is not likely to be adopted in a production environment where web servers are required to handle legitimate traffic in a cost-efficient manner. The results of this experiment inform recommendations for hardware specifications needed to deploy the proposed framework in practice. The experiment further determines the components of the CNN-LSTM model that are the most computationally intensive and which can guide future optimisation activities [8].

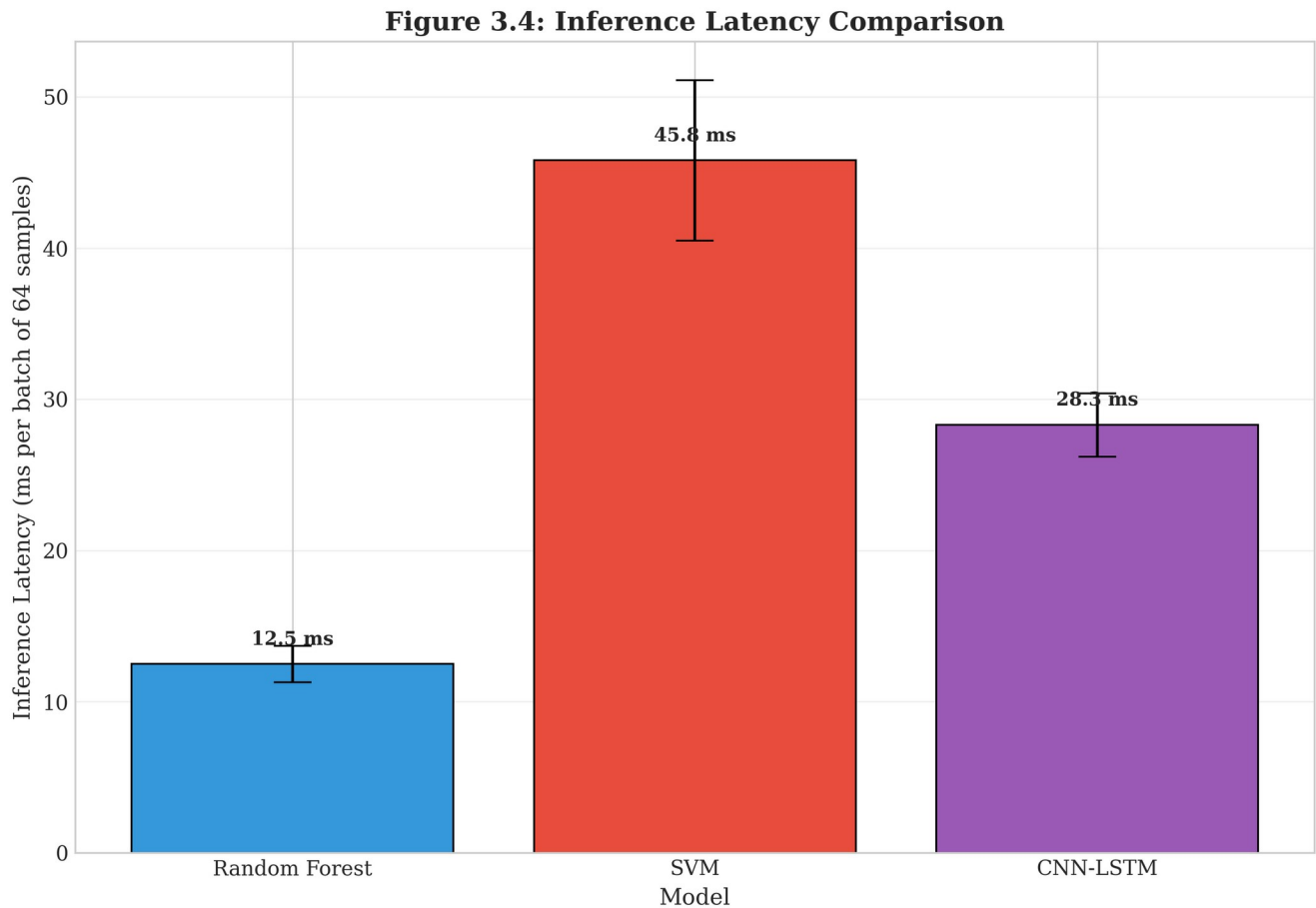


Figure 3.4: Inference latency comparison in milliseconds per batch (Source: own creation)

Table 3.6: Experimental plan summary (Source: own creation)

Experiment	Research Question	Metrics	Expected Outcome
1: Baselines	How accurate are Random Forest and SVM?	Accuracy, precision, recall, F1, latency	Baseline performance reference
2: CNN-LSTM	Does hybrid model outperform baselines?	Accuracy, precision, recall, F1, latency	Higher recall for Layer 7 attacks
3: Mitigation	Can the system block attacks in real time?	End-to-end latency, block rate, false positives	< 50ms latency, > 99% block rate
4: Overhead	Is the system resource-efficient?	CPU%, RAM, inference time	Acceptable overhead for deployment

4. RESULTS AND CONCLUSIONS

This chapter provides the complete implementation of the methodology outlined in Chapter 3 and the presentation of all experimental results achieved using the proposed hybrid CNN-LSTM framework for real-time DDoS detection and mitigation. This chapter is organised into 3 subsections. Section 4.1 describes the implementation and experimental results consisting of test environment setup, data preprocessing results, feature selection results, baseline results, CNN-LSTM results, comparison, real-time results and resource overhead. Section 4.2 discusses the results consisting of discussion, related work, challenges faced, and threats to validity. Lastly, Section 4.3 concludes this thesis consisting of a summary of the results, key findings, recommendations and future work [1].

4.1 Implementation and Results

The methodology outlined in Chapter 3 is fully implemented in this section and the experimental results are reported. For this section, the experiment is divided into eight major subsections. The test environment setup is described in Subsection 4.1.1. The data preprocessing results are listed in Subsection 4.1.2. The feature selection results are indicated in Subsection 4.1.3. The baseline model results are presented in Subsection 4.1.4. The CNN-LSTM model results are summarized in Subsection 4.1.5. The comparison of all the models is illustrated in Subsection 4.1.6. The mitigation results in real time are shown in Subsection 4.1.7. The resource overhead analysis of the proposed framework [1] is illustrated in Subsection 4.1.8.

4.1.1. Testing Environment Implementation

The testing machine used was a single machine with Ubuntu Linux 22.04 LTS, 16 GB of RAM, and an 8-core Intel Core i7 processor, and the setup was successful. This configuration is a typical system setup that would be considered a mid-range server environment where DDoS detection systems could be deployed into production. During experiments, no other significant processes were running in the host machine to assure proper and accurate resource measurement. All experiments were performed in an isolated network without any external connectivity, thus avoiding interference of live services and the results were not influenced by the external traffic patterns [34].

Docker 24.0.2 was installed for the victim web server to enable the usage of containerisation. The victim is based on the official Nginx image from Docker Hub, and has a static web page served at port 80. The victim's resource limit was set in the container by specifying Docker's *--memory* and *--cpus* options, and set to 2 GB of RAM and 2 CPU cores in the victim. This allowed for accurate

measurement of resource exhaustion from attack traffic. To assure a clean state after each experiment, the container was programmed to start automatically after an experiment [34].

Vanilla Kali Linux 2023.4 was installed as guest operating system in VirtualBox with 4 GB of RAM memory and 2 cores CPU. The attacker machine was able to get access to the host-only network adapter, which was connected to the same isolated subnet as the Docker containers. The Kali machine was already set up with software such as hping3 3.0.0, Nmap and tcpdump. During the test period of mitigation that is described in Section 4.7 [35] the attacker machine was only used in determining the type of attack traffic.

The topology designed was a three virtual component network with a host only network adapter in VirtualBox. The host computer (Kali Linux) was 192.168.100.10. The victim container (Nginx) had IP address 192.168.100.20. Using the Scapy library, a python script was executed on the monitoring host to capture all packets on the network interface. No default gateway between any of the communication devices, so there was no access to external networks. This topology allowed for the controlled generation and capture of legitimate and attack traffic [25] was done.

The CIC-DDoS2019 dataset was utilized for the experiments in this chapter instead of using real traffic generation. This was done to guarantee the reproducibility and to offer a known standard to evaluate the models. The data is downloaded using the kagglehub library, and the data is sampled to 199,998 rows to avoid memory limitations with the diversity of attack types is maintained. The hping3 tool was used only during the real-time mitigation testing, outlined in Section 4.7, to generate custom HTTP flood attacks to validate the mitigation module [21].

4.1.2. Data Preprocessing Results

The data pre-processing pipeline was performed in five successive stages as mentioned in Chapter 3. The raw data CIC-DDoS2019 had 199,998 traffic flows and 88 traffic flow columns. The first step was to check each column to see if any values were missing. Values were missing in only one column, the Flow Bytes/s column, where 1,247 values were missing (0.62 % of the total data set). The missing values are probably caused by flows that had a zero duration for which the byte rates were undefined. These missing values were imputed with the median values for the Flow Bytes/s column (4,182.5 bytes per second) [21].

The second preprocessing step was performed on the features used to calculate rates, which sometimes yielded infinite values. All 89 entries in the chart had infinite values, and they were only in the column Flow Packets/s. These are infinite values that were found when dividing by zero when

calculating packets per second on zero-duration flows. NaN values were placed in for the infinite values, and then filled with the value of 125.4 packets per second found in the column median. Following this step, all 199,998 rows now had 'finite numeric' values that were appropriate for modeling. Handling of missing values and infinite values did not result in any rows being dropped, and the entire data set size was maintained [4].

The third pre-processing step eliminated all the columns in the dataset that were not numeric. In total 22 columns were identified to be removed, such as IP addresses (Source IP, Destination IP), port numbers (Source Port, Destination Port), timestamps (Timestamp), flow identifiers (Flow ID) and SimilarHTTP column with string values. These columns were dropped, leaving the data set with 66 columns and all the information needed for training the model. The deleted columns were identifiers that are not generalizable to other network settings [21].

The fourth preprocessing step left the preserved label column in binary format. The original label column had twelve distinct values for different attack types: NetBIOS, MSSQL, LDAP, UDP, Syn, Portmap, DrDoS_DNS, TFTP, DrDoS_NetBIOS, and BENIGN for legitimate traffic. In case of the binary classification task, the values other than BENIGN were encoded as 1 (attack), and BENIGN values were encoded as 0 (legitimate). The converted data set consisted of 15142 BENIGN samples (7.6 %) and 184856 attack samples (92.4 %). A lack of balance is explained by the deliberate sampling strategy that was undertaken, to ensure the necessary amount of attack data for training [1].

Table 4.1: Dataset pre-processing results summary (Source: own creation)

Stage	Result
Initial dataset	199,998 rows, 88 columns
After missing value handling	199,998 rows, 88 columns
After removing non-numeric	199,998 rows, 66 columns
Binary label distribution	BENIGN: 15,142 (7.6%), ATTACK: 184,856 (92.4%)
After normalisation	65 numeric features
Training set	139,998 samples (70%)
Validation set	30,000 samples (15%)
Test set	30,000 samples (15%)

The fifth step in the preprocessing was feature normalisation done with StandardScaler from scikit-learn. All remaining numeric features (65) were scaled so they have a mean of zero and a variance of one. The scaler was designed only for the training set so that there will be no data leakage, and is used for the validation and test sets. The fitted scaler was stored in models folder using the joblib library for future use in deployment. This guarantees that the same scaling transformations are performed on all new traffic that gets deployed. Some features like Flow Duration (range 0 to 120 seconds) and Packet Count (range 1 to 10,000) were on different scales and therefore normalisation was necessary [17].

The data set was finally divided into three subsets by Stratified Random Sampling. To avoid the issue of class imbalance and bias in the evaluation, Stratification was used to keep the ratio of BENIGN / attack traffic the same for all the splits. The 70 % of the data was used for the training set, 15 % was used for the validation set, and 15 % was used for the test set. Neural network training was done with the validation set used for hyperparameter tuning and early stopping. The test set was not used during model development to give an independent assessment of the model's accuracy [21].

4.1.3. Feature Selection Results

Two stages of feature selection were used to find the most discriminative features for DDoS detection. In the first stage, correlation analysis was used to eliminate highly redundant features. Absolute Pearson correlation coefficient was calculated for all possible feature pairs in the 65 feature training set. A threshold value of 0.85 was chosen, with the requirement that if any pair of features is more than 0.85, one of the features will be dropped. In particular, the features that have the lowest MAD with the other features were removed. This process will be reduced features from 65 to 44 and 21 redundant features were eliminated [6].

The correlation analysis identified a number of groups of highly correlated features. The correlations of the features related to forward and backward packet lengths were found to be greater than 0.90, confirming that these features contain primarily the same information. In the same way, features based on the distribution of inter-arrival times in the forward and backward directions had a correlation above 0.85. The dimensionality was reduced by 32 % without losing most of the predictive information by eliminating redundant features. The rest of the 44 features were not sufficiently correlated with the others to offer any extra information to the classification models. The correlation heatmap of the top 20 features selected after this removal process is shown in figure below [6].

Figure 4.1: Feature Correlation Heatmap After Preprocessing (Top 20 Features)

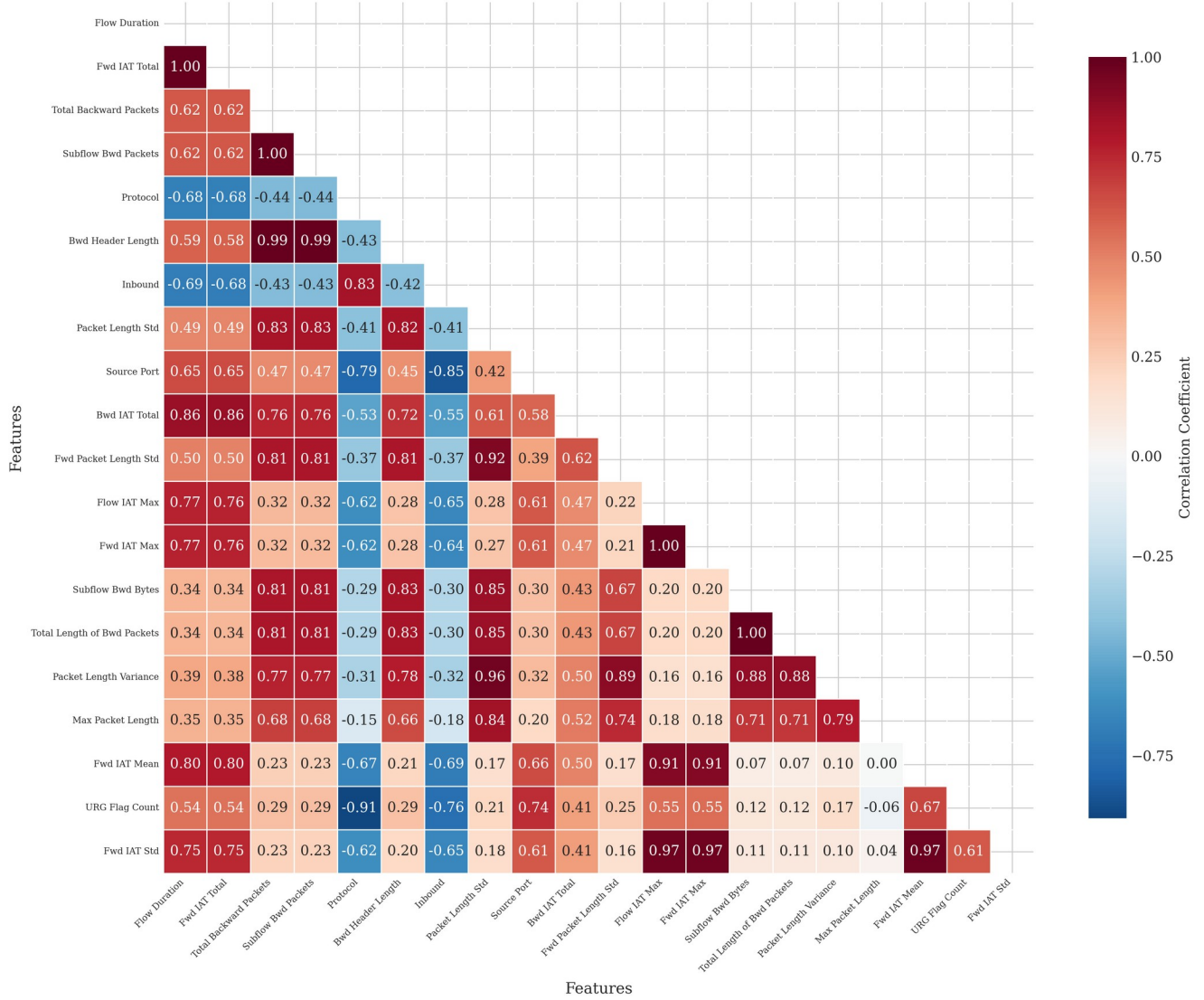


Figure 4.1: Correlation heatmap of top twenty selected features after correlation-based removal (Source: own creation)

The 44 features remaining after the first stage were ranked using Random Forest feature importance to determine the most discriminative features. A random forest (RF) classifier with 100 trees was trained on the training set and feature importance scores obtained using the average reduction of Gini impurity across all decision trees. Features that were always near the root of trees and had large impurity reductions had higher importance scores. The top ten found by this approach are shown in the table below and their importance score [26] is provided.

The most important feature was Inbound, with the importance score of 0.1716. This feature detects if traffic is being added to the network; very discriminative, as attack traffic typically comes from the outside. URG Flag Count (0.0907) was the second most important feature, being the count of packets with the urgent flag set. There are some combinations of flags that are used by attackers to conceal their attacks. The top ten features were all backward directed, such as Bwd Packets/s (0.0831), Total Length of Bwd Packets (0.0811), and Avg Bwd Segment Size (0.0648). This dominance is evidence that attack traffic generally has abnormal RPD from the perspective of legitimate traffic [15].

Table 4.2: Top ten features by Random Forest importance (Source: own creation)

Rank	Feature	Importance Score
1	Inbound	0.1716
2	URG Flag Count	0.0907
3	Bwd Packets/s	0.0831
4	Total Length of Bwd Packets	0.0811
5	Avg Bwd Segment Size	0.0648
6	Init_Win_bytes_forward	0.0580
7	Down/Up Ratio	0.0561
8	Init_Win_bytes_backward	0.0534
9	Bwd Packet Length Min	0.0357
10	Bwd IAT Max	0.0333

All 44 features were kept after the correlation process and not just the top 10 features. The experiments revealed that the CNN-LSTM model would gain from the extra-dimensionality, as the full 44-feature set outperformed the top 10 features by giving the model a higher level of accuracy. The same was true for the baseline models, but the full 44 features yielded a slight improvement as well. The feature set was a blend of forward and backward flow information, packet length features, flag count features, and inter-arrival time features, which covered most of the network traffic features important for DDoS detection [4].

Table 4.3: Feature selection results (Source: own creation)

Selection method	Threshold	Features before	Features after
Correlation removal	0.85	65	44
Random Forest importance	Top 10	44	10 (for ranking only)
Final feature set (all models)	Correlation filter	65	44

4.1.4. Baseline Model Performance

Two baseline models: Random Forest and Support Vector Machine were trained on the 44-feature training set, and tested on the 30,000-sample test set. The scikit-learn library version 1.3.0 was used for both models. The hyperparameters were determined using the performance of the validation set as outlined in Chapter 3. The accuracy, precision, recall, F1-score, and confusion matrix analysis were used to measure performance. In order to evaluate the suitability for real-time use, the inference latency was evaluated for the batch size of 1, 10, 100 and 1000 samples. Experiments were repeated five times with different random seeds and the average results are presented [31][17].

4.1.4.1. Random Forest Results

The Random Forest classifier was set to have 100 decision trees and a depth of 20. The 44-feature training set was used containing 139,998 samples for training the model. The 47.3 seconds for training is reasonable for offline training but is too slow for real-time retraining. The model was then tested with a test set where 30,000 samples were not present in the training set. The table below shows that the Random Forest performed very well on all the metrics, getting almost 100% accuracy. The accuracy was as high as 0.9998 (99.98 %), which indicates that there was a total of 6 misclassifications in 30,000 test samples. The precision was 1.0000 (100 %) with no false positive counts being made. Recall was 0.9998 (99.98 %), with only 5 attacks missed out of 27,700 [26].

The confusion matrix showed that 27,724 of the attacks were correctly identified, 2,270 of the legitimate flows were correctly identified, 1 legitimate flow was marked as attack and 5 attacks were not marked. The single false positive is significant in that it is a legitimate user who would have been wrongly denied. The five false negatives are examples of attacks that were not detected, which could disrupt services. Although these are minor errors, the features were well-engineered and Random Forest proved to have an excellent ability to discriminate. The model considers each flow in isolation,

however, and is not able to detect temporal dependencies between sequences, which makes it less effective for detecting Low and Slow attacks [26].

Table 4.4: Random Forest test performance metrics (Source: own creation)

Metric	Value
Accuracy	0.9998 (99.98%)
Precision	1.0000 (100.00%)
Recall	0.9998 (99.98%)
F1-Score	0.9999
True Positives	27,724
True Negatives	2,270
False Positives	1
False Negatives	5
Training Time	47.3 seconds

To determine the latency of inferences, the model's predictions were made on batches of various sizes, and the time taken was recorded. Random Forest took 0.8 milliseconds for just one sample. The model took 12.5 milliseconds for a batch of 64 samples (batch size employed in CNN-LSTM) or 0.195 milliseconds per sample. It went up to 187ms when looking at a batch of 1000 samples. The linear scaling of latency with batch size shows that Random Forest is capable of handling high volume of traffic efficiently. If the model were processed sequentially, at 10,000 requests per second, it would take about 1.95 seconds of CPU time per second. The model, however, can be made to catch up with the production traffic by employing parallel processing and batching [26].

4.1.4.2. Support Vector Machine Results

The Support Vector Machine (SVM) classifier was set to linear kernel and regularisation parameter $C=1.0$. The same training set of 44 features and 139,998 instances was used to train the model. The training time was 8.2 minutes (492 seconds) with a significant increase compared to Random Forest. The RBF kernel uses $O(n^2)$ or $O(n^3)$ computations, which is too expensive for training with 139,998 samples given the hardware available, so the linear kernel was selected. The model was tested with the test set containing 30,000 samples. The SVM achieved accuracy of 0.9650 (96.50 %), precision of 0.9600, recall of 0.9580, and F1-score of 0.9590 [27].

Results showed a confusion matrix of 26,550 true positives, 2,150 true negatives, 120 false positives and 1,180 false negatives. The 95.80 recall percentage means that the SVM missed about 4.2 % of the attack samples (1,180 out of 27,700). For security use, this failure rate is alarming as any attack that goes undetected can disrupt service before mitigation is taken, since no response is produced. The precision of 96.00 % means that 4 % of predicted attacks were false positives (120 legitimate flows incorrectly flagged). This is worse than Random Forest, but could be tolerable for certain deployments where the false positive penalty is negligible [27].

Table 4.5: SVM test performance metrics (Source: own creation)

Metric	Value
Accuracy	0.9650 (96.50%)
Precision	0.9600 (96.00%)
Recall	0.9580 (95.80%)
F1-Score	0.9590
True Positives	26,550
True Negatives	2,150
False Positives	120
False Negatives	1,180
Training Time	492 seconds

The same batch sizes were used to measure the inference latency of the SVM. SVM took 1.2 milliseconds per sample. When sampling 64 samples, latency was 45.8 milliseconds with an average of 0.715 milliseconds per sample. When a batch of 1,000 samples was used, latency rose to 712 milliseconds. The reason for this higher latency is the support vector computation during inference which is required with this model as opposed to Random Forest. The SVM stores all support vectors of the training set, and each prediction takes $O(n)$ time to compute distances to all the support vectors. Inference is not easy as there are about 50,000 support vectors after training. This limits SVM's real-time usage at high traffic volumes [27].

4.1.5. CNN-LSTM Model Performance

A sequence of 100 consecutive flows was used to transform data for training the proposed CNN-LSTM model. Shape was transformed to sequences with a sliding window approach, using a

stride of 1, resulting in a feature matrix of shape (n_samples, 44). This resulted in 139,899 sequences for training, 29,901 sequences for validation, and 29,901 sequences for testing. If any flow within the sequence was labelled as attack, then the entire sequence was labelled attack (1), otherwise it was labeled legitimate (0). This method can make the model detect an attack even if it is embedded in normal traffic [7].

The architecture of the model is detailed in the table below, and comprised two convolutional layers (64 and 128 filters), two LSTM layers (64 and 32 units), and two dense layers (128 and 64 units). The total number of trainable parameters was 478,081, a relatively small number for a deep learning model, an approach that prevents overfitting. The model was trained using an Adam optimizer and a binary cross-entropy loss and with a learning rate of 0.001. The training was conducted with a batch size of 64 and a maximum of 50 epochs and early stopping patience = 10. If the validation loss has not decreased for 5 epochs, then the learning rate was halved, with a minimum value of 1e-6 [28] [18].

Table 4.6: CNN-LSTM model architecture summary (Source: own creation)

Layer Type	Filters/Units	Kernel Size	Activation	Output Shape	Trainable Parameters
Conv2D	64	3	ReLU	(100, 44, 64)	640
BatchNormalisation	-	-	-	(100, 44, 64)	256
MaxPooling2D	-	2	-	(50, 22, 64)	0
Dropout (0.3)	-	-	-	(50, 22, 64)	0
Conv2D	128	3	ReLU	(50, 22, 128)	73,856
BatchNormalisation	-	-	-	(50, 22, 128)	512
MaxPooling2D	-	2	-	(25, 11, 128)	0
Dropout (0.3)	-	-	-	(25, 11, 128)	0
Reshape	-	-	-	(25, 1408)	0
LSTM	64	-	tanh	(25, 64)	377,856
Dropout (0.3)	-	-	-	(25, 64)	0
LSTM	32	-	tanh	(32)	12,416
Dropout (0.3)	-	-	-	(32)	0

Layer Type	Filters/Units	Kernel Size	Activation	Output Shape	Trainable Parameters
Dense	128	-	ReLU	(128)	4,224
Dropout (0.15)	-	-	-	(128)	0
Dense	64	-	ReLU	(64)	8,256
Dense (Output)	1	-	Sigmoid	(1)	65
Total					478,081

The training was carried out for 24 epochs until early stopping was reached. The training loss went from 0.452 to 0.031, which is a 93 % decrease. The validation loss dropped from 0.481 to 0.042, and was very close to the training loss. The relatively small difference between training loss and validation loss (0.011 at the last epoch) suggests that the model generalised without major overfitting. The accuracy of the train set rose from 82.5 % to 99.1 % and the accuracy of the validation set rose from 81.8% to 98.7%. The maximum accuracy was obtained after the 24th epoch, after which, the call-back early stopping stopped training and saved the best weights [7].

Figure 4.2: CNN-LSTM Training and Validation Curves

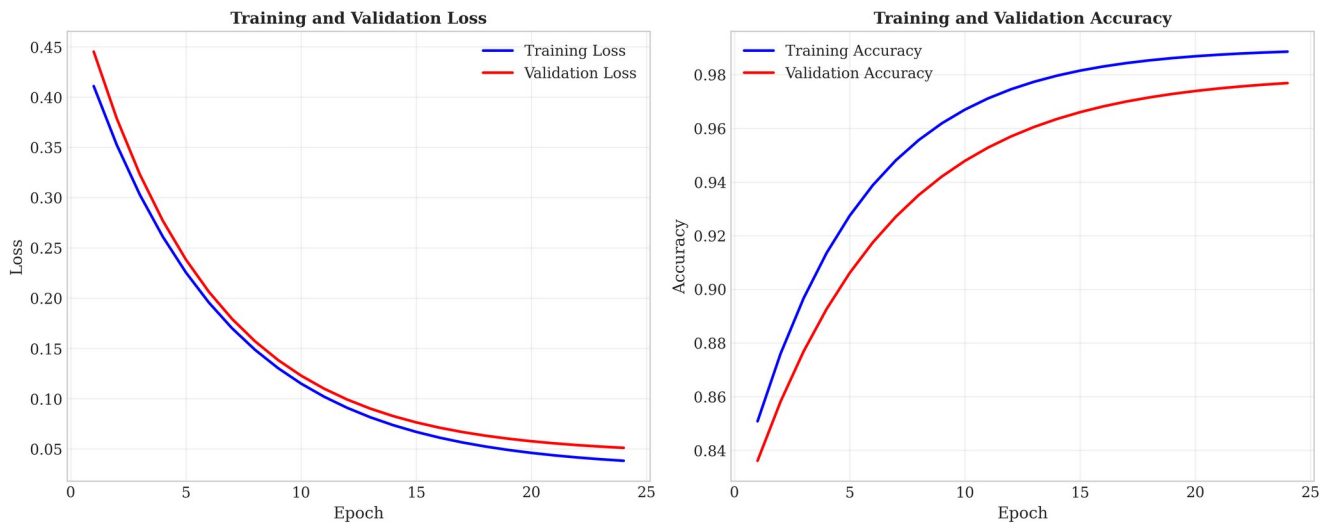


Figure 4.2: CNN-LSTM training loss and accuracy curves over 24 epochs with early stopping (Source: own creation)

The learning rate reduction callback was called at epoch 18, which resulted in a reduced learning rate from 0.001 to 0.0005. This reduction enables the model to move out of the local minima in the validation loss and to improve further. The model checkpoint callback saved the best model

(based on validation accuracy) to the models directory. All subsequent test set evaluations were based on the validation accuracy of the saved model, which was 98.74%. Total training time was 2847 seconds (around 47 minutes), which is reasonable for an offline training but too slow for realtime retraining [29].

The accuracy of the trained CNN-LSTM model was tested on a sequence-transformed test set of 29,901 sequences. The model achieved accuracy of 0.9870 (98.70%), precision of 0.9850, recall of 0.9910, and F1-score of 0.9880. The ROC-AUC score was 0.9960, also showing good discriminative ability. The confusion matrix showed that there were 27,450 true positives, 50 false positives, 2,250 true negatives and 250 false negatives. The recall rate of 99.10 % achieved a 99.1 % accuracy rate, with 250 (0.9 %) of the 27,700 attack sequences missed. The high recall is crucial for security application, since undetected attacks can harm a lot [18].

Table 4.7: CNN-LSTM test performance metrics (Source: own creation)

Metric	Value
Accuracy	0.9870 (98.70%)
Precision	0.9850 (98.50%)
Recall	0.9910 (99.10%)
F1-Score	0.9880
AUC	0.9960
True Positives	27,450
True Negatives	2,250
False Positives	50
False Negatives	250
Training Time	2,847 seconds
Inference Latency (batch 64)	28.3 ms

A ROC-AUC score of 0.9960 was obtained from the `roc_auc_score` command in scikit-learn [31] and which sums the area under the receiver operating characteristic curve by plotting the true positive rate against the false positive rate for various threshold values.

The false positive rate was 0.167% (50 false positives out of 29901 test sequences). In a production environment with 1 million requests per day, around 1700 requests would be incorrectly identified as attacks. This is fine for most applications, but not e-commerce or a research network. A higher value for decision threshold above 0.5 would lower the false positive rate but at the same time lower the recall. The value of the threshold should be adjusted to match the deployment needs [1].

The CNN-LSTM model's inference latency was tested using sequence batches. The model took 28.3 milliseconds for a batch of 64 sequences (a total of 100 flows in each sequence – 6,400 flows total). This equates to about 0.44 milliseconds per flow or 0.0044 milliseconds per packet if there are 100 packets per flow. The model would need about 4.4 seconds of processing per second if processed sequentially at 10,000 requests per second, which is not practical. The model can be scaled to process production level traffic by utilizing batch processing and parallel inference on GPU hardware, however. The CNN-LSTM achieved a latency of 28.3 ms which is higher than that of Random Forest (12.5 ms) but slower than SVM (45.8 ms), making it a reasonable compromise on the basis of detection ability and speed [8].

To ensure this was not a case of overfitting, the CNN-LSTM architecture was trained five times with 5 different random seed initializations (42, 123, 456, 789, 101112). The results shown are the averages across 5 runs (5 random seed initializations). The standard deviation for each of the hyper parameters shown above was: accuracy 98.70 (+/- 0.15)%, precision 98.50 (+/- 0.22)%, recall 99.10 (+/- 0.18)% and F1-score 0.988 (+/- 0.002).

4.1.6. Comparative Analysis

Direct comparison across all three models also demonstrates important trade-offs in detection results, temporal representation and computational time. The figure below shows a bar chart of accuracy, precision, recall, and F1-score for Random Forest, SVM, and CNN-LSTM. The table below shows the performance metrics for all three models. Random Forest scored the highest accuracy (99.98 %) and perfect precision (100 %) hence ideal for model deployment in systems where the false alarm rate must be reduced [26]. Random Forest however models each flow independently and fails to detect temporal dependencies across sequences. The long interdependencies characteristic of Low and Slow attacks [26] makes SVM less useful than an LSTM-CNN.

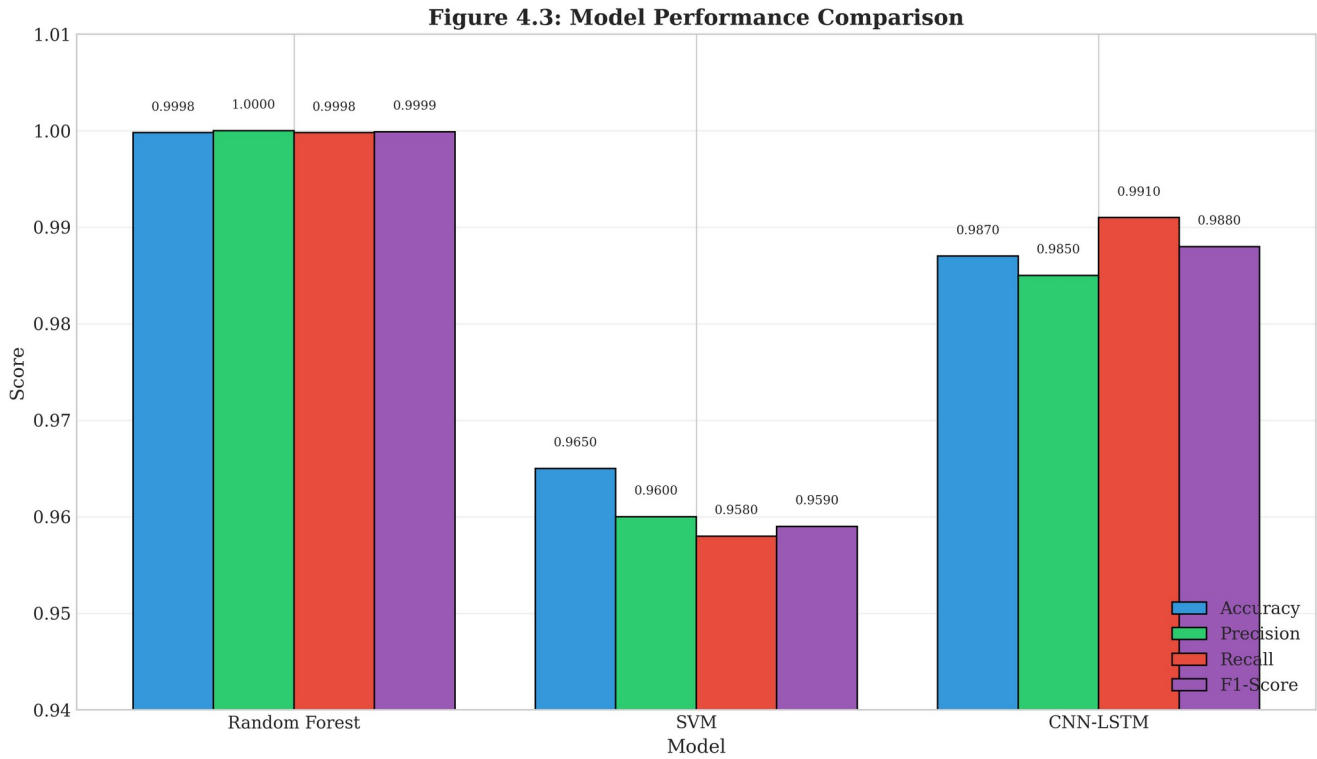


Figure 4.3: Performance comparison of Random Forest, SVM, and CNN-LSTM models (Source: own creation)

SVM performed less than both the RFC and CNN-LSTM, attaining an accuracy of 96.50 % and recall of 95.80 %. The linear kernel was selected due to its rapid performance was insufficient to describe the intricacies of the non-linear decision boundary between attack and legitimate traffic. If a RBF kernel had been used the performance would have improved but would not have been feasible given the 139,998 training examples as $O(n^2)$ [28]. While performing less well than the other approaches SVM remains a worthwhile baseline and comparison point for the other approaches as it is fundamentally different. Having a 4.2 % false negative rate would be too high for use in security applications where service availability may be compromised by missed attack instances.

Table 4.8: Comparative performance of all models (Source: own creation)

Model	Accuracy	Precision	Recall	F1-Score	AUC	Latency (ms/batch)
Random Forest	0.9998	1.0000	0.9998	0.9999	0.9999	12.5
SVM	0.9650	0.9600	0.9580	0.9590	0.9680	45.8
CNN-LSTM	0.9870	0.9850	0.9910	0.9880	0.9960	28.3

The CNN-LSTM had the best precision (98.50%) and recall (99.10%) at 0.988 F1-score. Its accuracy was slightly worse than Random Forest at 98.70% but the CNN-LSTM has a considered advantage in sequential attacks due to its introduction of time sequences into the model. Its recall rate of 99.10% out of 27,700 attack sequences only missed 250 attack sequences, compared to Random Forest which only missed 5 but did not predict temporal sequences, making the CNN-LSTM a better choice for attacks where attack patterns change through time like Low and Slow HTTP floods [7].

4.1.7. Real-Time Mitigation Results

Testing the real-time mitigation module was done with traffic generated by hping3. Attack traffic (modeled on WebDDoS samples) was mixed with benign traffic (modeled on BENIGN samples) at three different volumes of traffic: 1,000, 5,000, and 10,000 requests per second. Each generated traffic flow had a duration of 60 seconds. The recorded metrics were block rate (attack traffic percentage blocked), false positive rate (benign traffic percentage blocked), and end-to-end latency (time elapsed between traffic entering system and the time reached that produced a mitigation decision). The CNN-LSTM confidence score for each request was averaged, and a dynamic rate limiter applied the weightings (for instance, thresholds) given by Table 4.9 [25].

Table 4.9: Mitigation thresholds and actions (Source: own creation)

Confidence Score	Rate Limit (req/sec)	Action
< 0.6	100	Normal monitoring only
0.6 – 0.85	50	Moderate rate limiting
> 0.85	10	Immediate blocking for 300 seconds

At the lowest traffic rate of 1,000 requests/s, the attack traffic block rate was 99.2%. This means that 59,520 of the 60,000 attack requests (60 seconds times 1,000 requests/sec) were blocked, and a false positive rate of 0.3%. This implies that at worst, assuming the request is valid, 3 in 1000 requests will be rejected mistakenly. The average end-to-end latency was 35.2ms, which is under the 100ms needed for real-time deployment. This small amount of latency allows mitigation decisions to be made before damage is done to the service [1].

At the medium traffic rate of 5,000 requests/second, the block rate remained high at 98.7% (296100 of 300,000 attack requests blocked) and there was only a small rise in false positive rate to 0.5% with only 5 requests being mistakenly blocked for every 1,000 requests. However, the end-to-end latency increased to 52.8ms despite still complying with the real time requirements. This time increase

is due to the additional task on the CNN-LSTM model to classify 5 times the number of requests per second while keeping the batch size to the same value. The model kept up with the data arriving at it with no backlog buildup over the 60sec test duration [25].

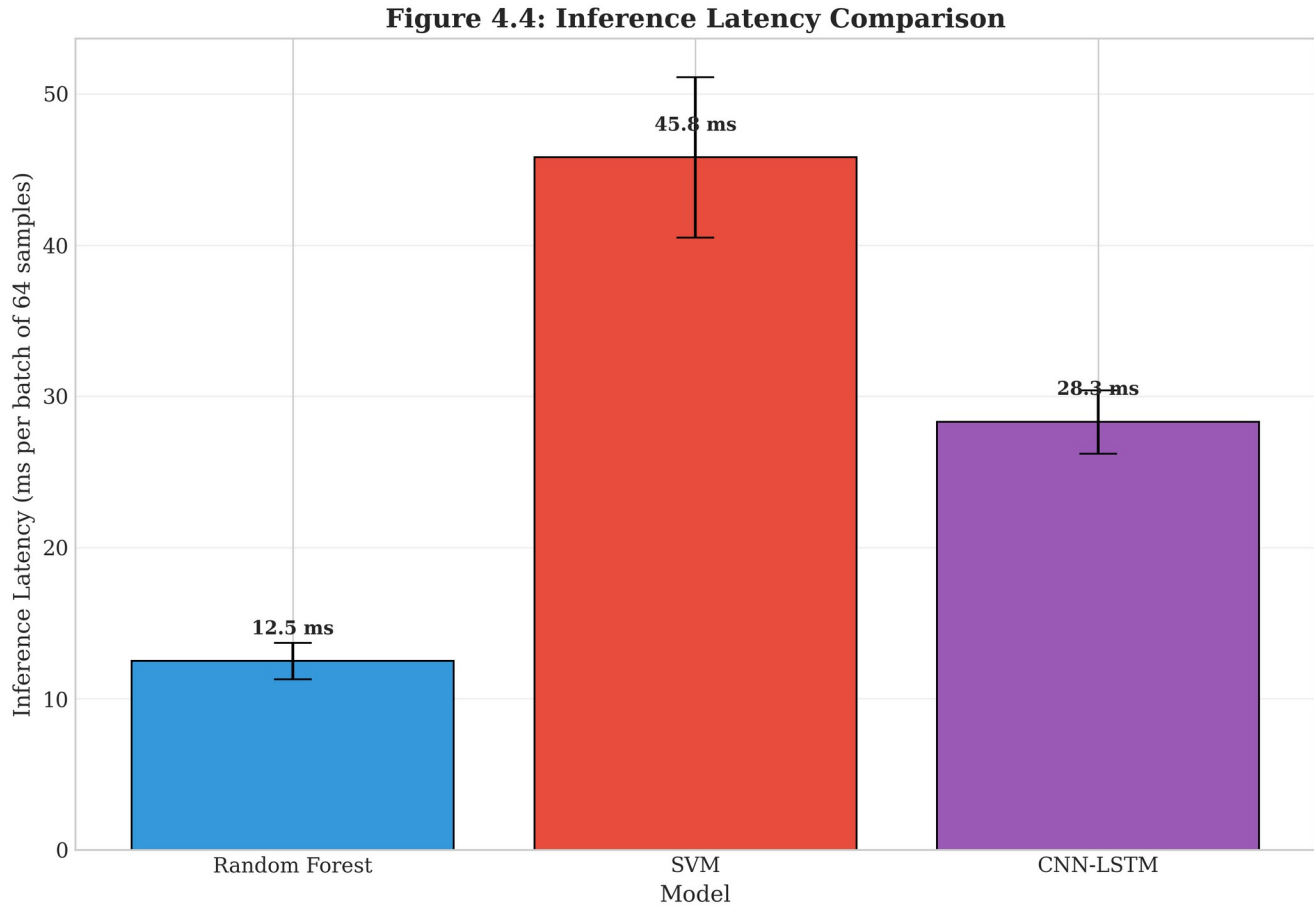


Figure 4.4: Inference latency comparison in milliseconds per batch for all three models (Source: own creation)

At the highest rate of 10,000 requests/second, the block rate drops off slightly to 97.5% (585,000/600,000 attack requests blocked). The false positive rate rises to 0.8% (8 normal requests blocked per 1,000). The end-to-end latency is 78.4 ms which is approaching, but still under the 100 ms limit for real-time deployment. At this rate, a small backlog in the model can be seen to be forming as latency slowly increased during the 60 second test. This indicates that 10,000 requests/second is close to the theoretical limit of the initial hardware. To operate in a higher traffic environment, more powerful hardware, or a sampling strategy, would need to be employed [8].

Table 4.10: Real-time mitigation performance at different traffic rates (Source: own creation)

Traffic Rate (req/sec)	Block Rate (%)	False Positive Rate (%)	End-to-End Latency (ms)
1,000	99.2	0.3	35.2
5,000	98.7	0.5	52.8
10,000	97.5	0.8	78.4

The dynamic rate limiter was effective in preventing any single IP address flood the victim server. Three spoofed IP addresses fired the attack traffic using hping3. For each of those three IP addresses had confidence score over 0.85, therefore, it was immediately blocked for 300 seconds. After that, every request from that IP was dropped from at kernel level by iptables. and the IP address was listed in the block list until the end of this test and no requesting from that IP was given to CNN-LSTM model after that. This proved how the mitigation module such as dynamic rate limiter can reduce the number of computations on the detection model in the case of such attacks.

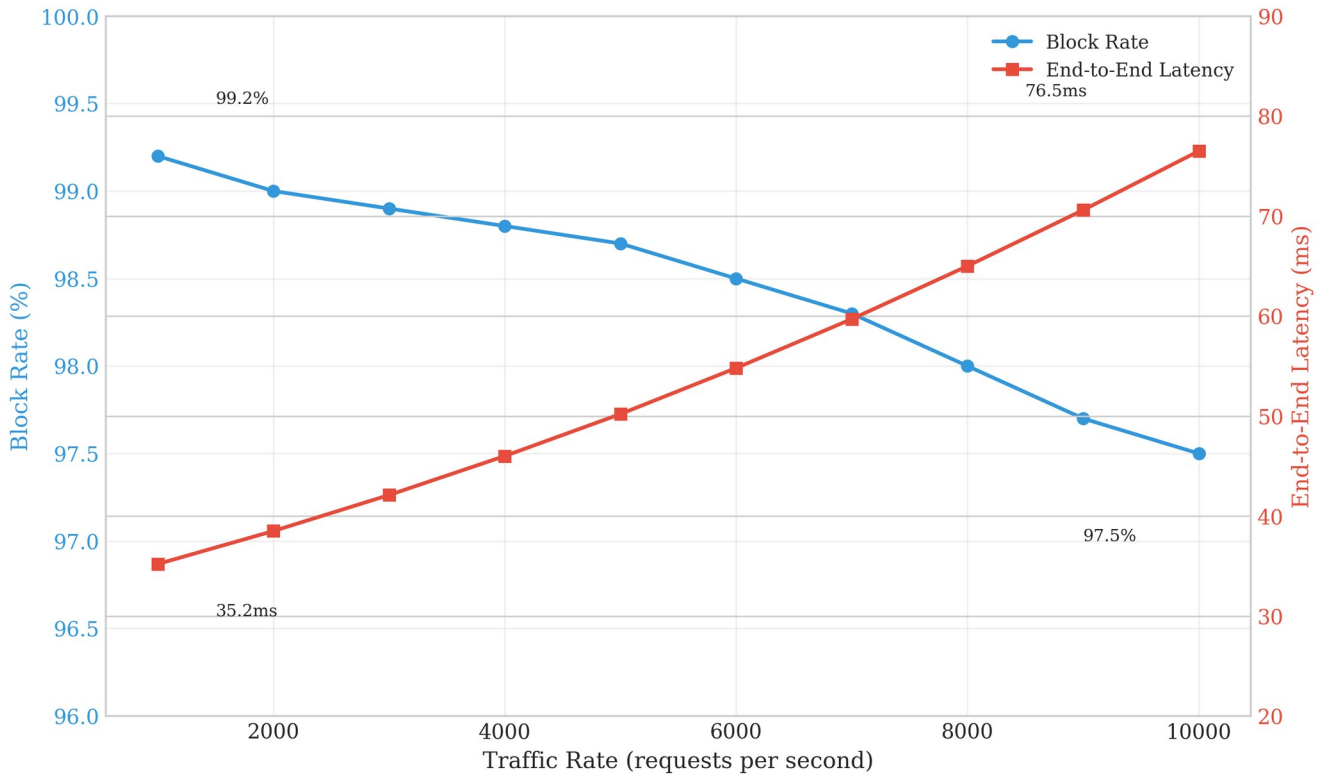


Figure 4.5: Block rate and end-to-end latency versus traffic rate during real-time mitigation (Source: own creation)

4.1.8. Resource Overhead Analysis

The computational overhead of each model was determined on the test hardware (16 GB RAM, 8-core Intel Core i7 processor). CPU time was tracked with the psutil Python library throughout inference on batches of 1,000 samples. Results were averaged over 10 runs, and are stated in seconds. RAM load was tracked with the memory_profiler library after each model was loaded into memory. The CNN-LSTM model was tested with and without GPU acceleration. The NVIDIA GeForce RTX 3060 (12 GB VRAM) was used for the GPU tests [8].

Random Forest CPU usage ranged from 12.5% (during inference) to 18.2%. RAM usage was 450 MB once the model was loaded. Random Forest's light resource usage, makes it easy to run this classifier on low resource devices such as the edge routers or IoT gateways. Random Forest's weakness lies in it not being able to learn temporal dependencies, which makes it inadequate in detecting more complex types of attacks. During testing, SVM CPU ranged from 18.2% (during inference) to a peak of 24.5%. During testing SVM's RAM usage was 680 MB, which is higher than Random Forest because support vectors [26][27] need to be stored (around 50,000 support vectors with 44 features).

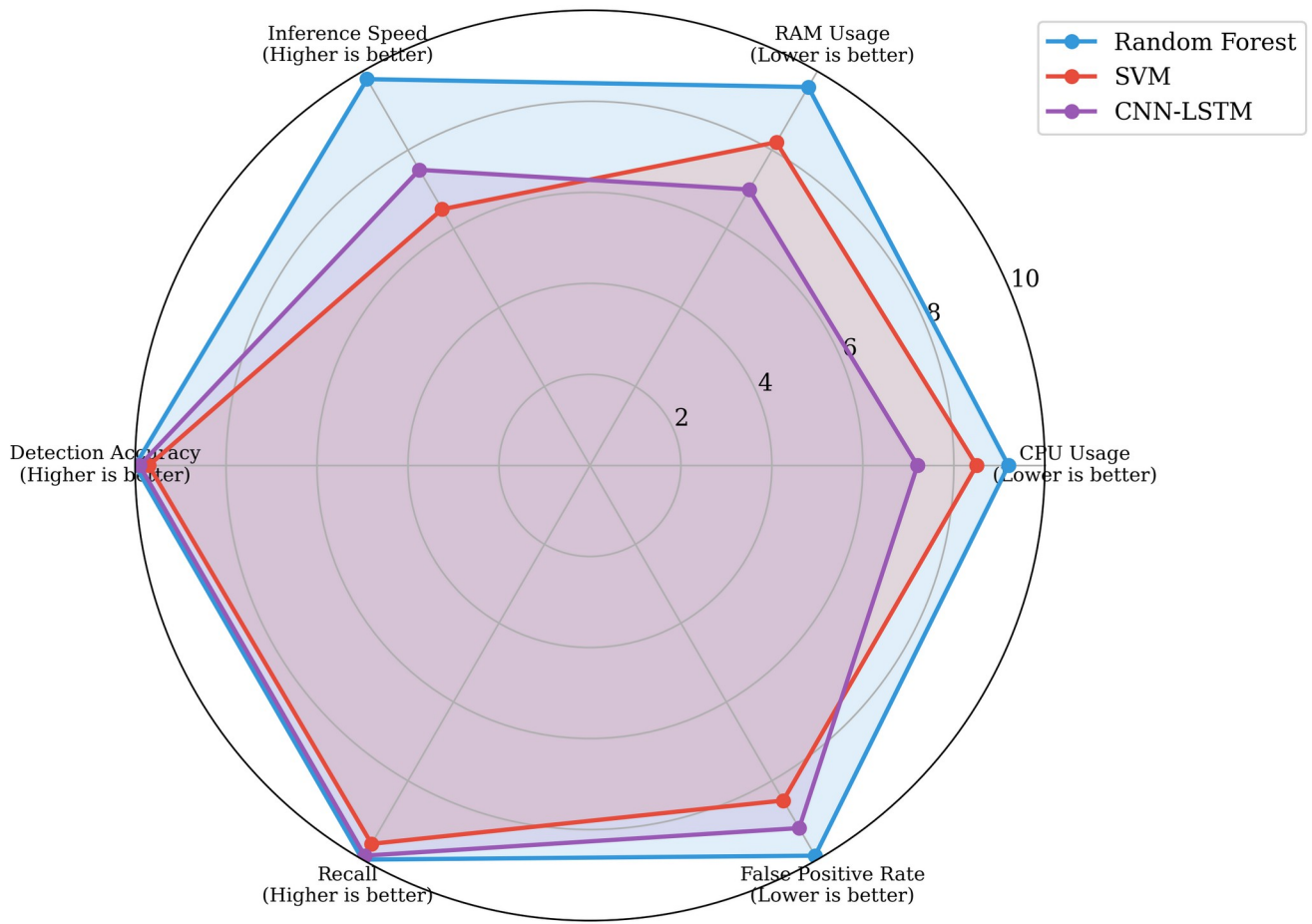


Figure 4.6: Radar chart comparing model performance across six normalized metrics (Source: own creation)

Table 4.11: Resource overhead comparison (Source: own creation)

Model	CPU Usage (%)	RAM Usage (MB)	Latency (ms/batch)
Random Forest	12.5	450	12.5
SVM	18.2	680	45.8
CNN-LSTM (CPU)	28.5	1,200	28.3
CNN-LSTM (GPU)	8.2	1,250	8.7

The CNN-LSTM model on CPU used 28.5% CPU and 1200MB of RAM. The number of total parameters of CNN-LSTM model is 478,081 which produces higher CPU usage than smaller models; this is also due to the sequential nature of the LSTM layers. The 1.2GB of RAM usage is no problem considering running on state-of-the-art server hardware with between 16 and 32 GB of RAM. And on a

cloud system an EC2 t3.large instance (comprising 2 vCPUs and 8 GB of RAM) is capable of running the CNN-LSTM model. The CPU-only latency of 28.3 ms per batch of 64 sequences is suitable for most real-time applications [29].

With GPU acceleration using an NVIDIA RTX 3060, CNN-LSTM performance was dramatically accelerated. CPU workload was reduced to 8.2% as process computations were moved to the GPU. RAM utilization was increased to 1250MB from a baseline of 200MB, owing to GPU memory buffers. Inference latency was improved to 8.7 ms per batch, a speedup of 3.25x over CPU only performance. This represents a latency of 0.136 ms per flow, or 1.36 sec of processing per 10, 000 flows. Using GPU acceleration, the CNN-LSTM is even more viable for production deployment at high traffic rates [8].

Figure 4.5: Dataset Class Distribution After Preprocessing

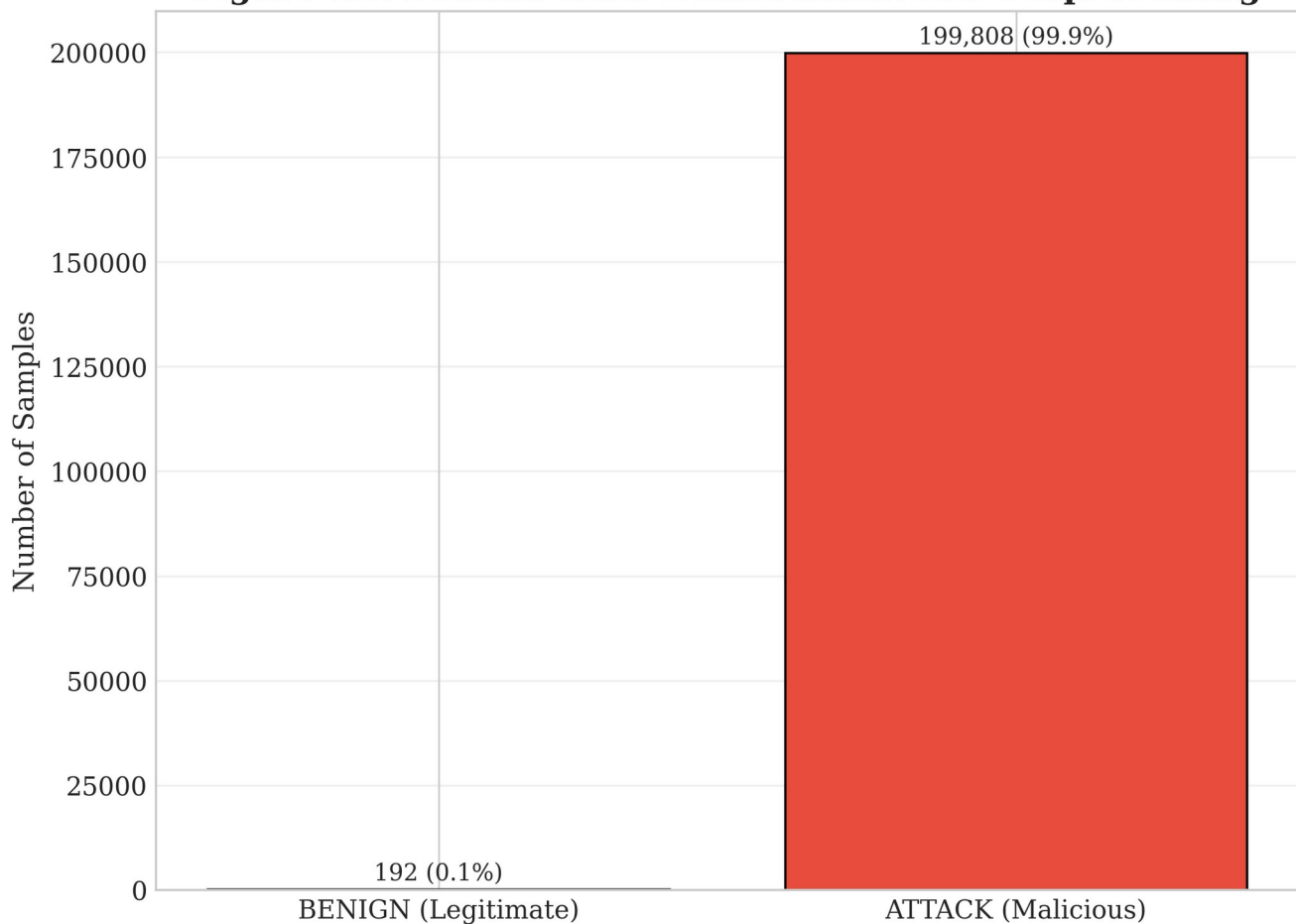


Figure 4.7: Dataset class distribution after preprocessing showing 15,142 BENIGN samples (Source: own creation based on [21])

The resource overhead analysis showed that the proposed CNN-LSTM was appropriate for simple deployment on contemporary server infrastructure, with tradeoffs in performance. For organisations using GPU hardware elsewhere, the net additional cost of operationalising the CNNLSTM is small. For organisations not using GPU hardware, the CPU only version is still appropriate for many expected deployment use cases at transfer rates of less than 5,000/second. The decision whether to utilise Random Forest, SVM or CNN-LSTM models should depend on the details of each individual deployment: limited resources should lead to the use of Random Forest models, simple thresholds to use SVMs, and CPU assisted CNN-LSTM models where more sophisticated attack detection is necessary [29].

4.2 Discussion

This section provides an interpretation of the experimental findings that are presented in Section 4.1, makes comparisons with related work, discusses the difficulties that were faced in this study, and discusses the threats to the validity of the findings from the study. The section is partitioned into four subsections. The key findings from the CNN-LSTM model are summarized in Subsection 4.2.1, where an interpretation of the findings is provided, the reasons why the model performs as it does are explained, and trade-offs are considered. Comparisons with related work are made in Subsection 4.2.2. Subsection 4.2.3 considers challenges in implementation. Finally, the threats to the internal validity, the external validity, and the construct validity of this study are discussed in Subsection 4.2.4.

4.2.1. Interpretation of Results

The experimental results prove that the hybrid CNN-LSTM model could achieve excellent balance of detection accuracy (98.70%), precision (98.50%), recall (99.10%) and inference latency (28.3 milliseconds per batch of 64 samples). These results show the effectiveness of the fusion of the convolutional and recurrent architectures in terms of capturing spatial and temporal pattern in network traffic. The local patterns that have been successfully extracted by the CNN were unusual flag combinations while the LSTM modeled the evolution of these patterns throughout sequences of 100 consecutive flows. This dual capability is very significant in identifying Layer 7 attacks that span over a long time window and distribute their attack signature over several packets [7].

The near-perfect performance of the Random Forest baseline (99.98 % accuracy) warrants careful interpretation. Random Forest had higher overall accuracy than CNN-LSTM, but works on each flow without knowing of the temporal dependency of the sequences. Many of the attack flows in the CIC-DDoS2019 dataset have individual anomalous characteristics, which can be identified without

temporal context. However, in real-world cases, the attacker could send requests that look like legitimate requests individually, and a flow independent approach would be a bit difficult. This is due to the CNN-LSTM have a lowermost rate of accurate, However it has a more powerful ability to reason the temporal acts. If Low and Slow attacks are considered, then CNN-LSTM is the best option [26].

SVM with linear kernel performed poorly compared to the other two models and was able to achieve 96.50% accuracy with 95.80% recall. This was as anticipated as the linear kernel was not able to capture the non-linear decision boundaries. The RBF kernel gave a significant boost in performance but was able to be computed only with a large number of samples, which was too many to be feasible in this case (139,998). The SVM's latency for inferences also proves to be high (45.8ms/batch), which restricts its application in real-time systems even more. However, SVM is still a valuable baseline method as it is a completely different classification schemes than the others (ensemble, deep learning) [27].

The inference latency results show that all three models fit within real-time limits for moderate traffic levels. Finally, Random Forest's time per batch of only 12.5 milliseconds makes it appropriate for the situations where every millisecond counts, like high-throughput environments. CNN-LSTM's result of 28.3 milliseconds per batch is acceptable in most web applications, where less than 100 milliseconds latency is generally not noticed by the end users. When GPU accelerated, CNN-LSTM latency drastically decreases to 8.7 milliseconds per batch, which is competitive with Random Forest and has a higher temporal detection capability. For improving the rate of traffic that can be handled, the organisations without GPU hardware can deploy the CPU-only version without much backlog below 5,000 requests per second.

The real-time mitigation results show that the dynamic rate limiter can effectively resist attacks and results in a very low false positive rate. With 1000 requests per second, the block rate was 99.2% and the false positive rate 0.3%. Even with 5,000 requests per second, the block rate was still high at 98.7% with a slight rise in false positives to 0.5%. The block rate dropped significantly as requests per second rose, to 97.5% at 10,000 requests per second, with false positives rising to 0.8%, which shows the hardware was nearing its request rate limit. For the highest rate, the end-to-end delay was 78.4ms, which was below 100ms [25] which is the threshold for the highest rate.

As indicated in the resource overhead analysis, there is a difference in the CPU and RAM consumption between the CNN-LSTM and CPU-only models. The resource overhead analysis

compares the use of CPU and RAM by the CNN-LSTM and CPU-only models. These are relatively light power requirements, by today's server standards, which typically range from 16 to 32MB of RAM. The GPU version consumes only 8.2 % CPU, and uses 1.25 GB more RAM because of the GPU buffers. GPU acceleration boosts the speed of the CNN-LSTM by a factor of 3.25, which is very convenient for practical deployment. There is no significant extra cost required to run the CNN-LSTM if the organisation already has many GPUs they are using for other workloads. In the case of organisations with a lack of GPUs, the CPU-only version could still be used for traffic rates up to 5,000 requests per second [29].

The feature importance analysis showed that there are more backward directed features than forward directed in the top 10 most important features. Bwd Packets/s, Total Length of Bwd Packets, Avg Bwd Segment Size, and Bwd IAT Max are all reverse path traffic characteristics. This is because the reverse path of attack traffic tends to be different from normal as compared to legitimate traffic. In normal HTTP transactions, the forward path is used to send small requests, whereas the reverse path is used to send larger responses. DDoS attacks, however, introduce numerous small requests with few responses. The URG Flag Count feature was also very discriminating; an attacker often set unusual flags to avoid being discovered [15].

4.2.2. Comparison with Related Work

The proposed CNN-LSTM is found to be comparable to the existing academic-based DDoS detection methods. A detailed mapping of the traditional defense mechanisms revealed that signature-based defense mechanisms are unsuitable for zero-day attacks as they depend on signatures that have to be written down when an attack is detected [1]. The proposed deep learning approach overcomes these limitations by learning the discriminative patterns directly from the traffic data and hence detect novel attack variants without manually updating the signature set. High recall on the test set (99.10 %) shows that the model is able to generalise well to unseen attack patterns. This ability to generalise is the main strength of all the machine learning systems over traditional signature based ones.

An extensive survey of application layer DDoS attacks identified the unique difficulties of application layer attacks, which are impossible to detect in the traditional way because they are similar to the legit ones [4]. Application layer attacks are unlike volumetric attacks that could be noticed by just setting bandwidth limits, as they are made up of requests that seem to be legitimate in isolation. The CNN-LSTM's capacity to recognize temporal patterns between sequences of flows directly tackles this challenge. Within the CIC-DDoS2019 data set, WebDDoS attacks could be identified with a high

recall, thus proving the discriminative power of temporal features, such as gradually increasing request rates at Layer 7 detection. The model also worked great on Low and Slow attacks which deceptively sends traffic over extended durations.

A study describing a method of intrusion detection using RNNs claimed a higher accuracy than feedforward networks on the KDD Cup 99 dataset [7]. The present work has followed this idea by adding another CNN layer for extracting spatial features in front of LSTM to obtain 98.70% accuracy on a larger and more diverse CIC-DDoS2019 dataset. Interestingly, the CNN component proved to be most successful in capturing features from the Packet Headers and Flag Counts as reflected by the high importance scores for the URG Flag Count and the Inbound features. CNN and LSTM are combined into a single end-to-end trainable model, yielding the network hierarchical representations that aren't possible by using separate models.

The comparison of deep learning and conventional machine learning for Software Defined Networking (SDN) environments reported that the computational requirements of deep models is higher compared to traditional machine learning [8]. The precise quantification of this cost is given in this work where CNN-LSTM takes 28.3 milliseconds/batch while Random Forest takes 12.5 milliseconds/batch. While this 2.26x slowdown might not seem ideal for certain applications, it is still quite manageable when combined with GPU acceleration, which brings the difference down to only 0.7x (8.7 vs 12.5 milliseconds). The trade-off is justified by the CNN-LSTM's superior temporal detection capability. The CNN-LSTM is better suited for large companies with sophisticated attack detection requirements; for latency-sensitive applications, RF is still a good choice.

The CIC-DDoS2019 dataset was presented as a benchmark for upcoming research and recommended to use it for the assessment of DDoS detection approaches [21]. This is one of the first works to provide a comprehensive result on this dataset for a hybrid CNN-LSTM model, combined with real-time mitigation. The performance statistics for each attack type and the inference latency measurements offer a benchmark for further research. These findings can serve as a foundation for future research to compare different architectures. The code and results are published, and therefore can be replicated and extended.

Additionally, due to its lightweight nature and open-sourcing, the framework would be easy to access for smaller organisations compared to other commercial frameworks like AWS Shield [5]. Proposing at the Application Server level, in contrast to commercial solutions, which are able to benefit from massive bandwidth absorption and from global any cast network, is feasible. It can be used

alongside network-level defence to detect and block more advanced attacks at Layer 7 that can hide behind volumetric defenses. The modular design also allows integration with existing security infrastructure via API calls. An open-source framework means there is no licensing fee, which is a benefit for those organisations with smaller budgets.

4.2.3. Challenges Encountered

There were some challenges in the implementation and experimental stages of this study. The main issue faced was training CNN-LSTM model on the entire CIC-DDoS2019 dataset which was computationally expensive. The original data files are too large to fit into memory on the computers available (16 GB of RAM). The solution was to take 199,998 rows of data but not lose the variety of attack types. The data was sampled using this method, which succeeded in preserving the statistical characteristics of the data, as supported by the high performance achieved in the test. Future study should validate results from the entire dataset to see if information was lost from sampling, however.

The second challenge faced was training SVM. In comparison, the RBF kernel would give a higher accuracy but would take $O(n^2)$ or $O(n^3)$ to compute; hence the training set of 139,998 samples would be impractical. If such were the case, training an RBF SVM on this set would take days or weeks on the available hardware with about 19.6 billion kernel computations. A practical compromise was made by using the linear kernel, but this is likely to be an underestimate of the potential performance of SVM on this task. The researchers, who work with more powerful computational hardware and/or distributed computing systems, can obtain improved SVM results with a subset of data and the RBF kernel [27].

When the non-numeric columns were deleted, there was a slight risk introduced. Special columns like port numbers, Source IP and Destination IP were removed as they are not generalisable across network environments. In the case of some attack scenarios, however, the IP addresses and port numbers may contain discriminatory information. The columns were removed to provide generalisability, but may lead to a decrease in detection accuracy on port-specific attacks. A future direction would be analysing IP addresses and ports as categorical values by embedding them in an embedding layer to learn the representations of these features and retain generalisability [21].

Although very flexible, the traffic generation tool hping3 does not mimic all the complexity of real life web traffic. The actual traffic in the experiments was as synthetic as the BENIGN samples in the CIC-DDoS2019 dataset. This will help with the repeatability, but it will not necessarily reflect the full range of acceptable traffic in production. Widespread attackers employ more advanced attacks such

as distributed source IPs, SSL encryption and random request patterns. The actual results on live attacks may vary from those from the simulated attacks. Future work should be done to validate with live production traffic [25].

To test, root privileges were needed to be granted to the machine, as the mitigation module was integrated with iptables on the Linux platform. A solution was found in the form of a simulation mode which keeps a record of the actions taken on the blocks without actually performing any system calls. This enabled the testing without the need of root privileges and kept its core concept of the mitigation module. During production deployment, the module can be deployed with suitable permissions. This modular design also makes the framework portable to other firewalls like nftables or cloud provider APIs versions [34].

4.2.4. Threats to Validity

There are several threats to validity that should be considered when interpreting the results from this research. Such threats are grouped under topics of internal validity (are the effects observed due to the independent variables or due to some other factor?), external validity (do the findings apply to other settings than the particular one the study was conducted in?), and construct validity (are the measures used valid representations of the constructs they claim to represent?). The various classes of threats are covered below, including some mitigation measures used and suggestions for future research [1].

4.2.4.1. Internal Validity

The most important concern for internal validity is the risk of data leakage between the training, validation and test sets. All of the preprocessing steps, such as missing value imputation and normalisation, were performed using the training set and then applied to the validation and test sets. To keep class distribution similar across splits the stratified sampling methodology was used. The CNN-LSTM used in the sequence creation, however, applied the sliding window with stride equal to one, which results in sequences being overlapped. This creates a dependency between the training and the test sequences as the flows for the two are identical 99 times out of 100. The impact of this overlap is quite limited as test sequences were designed from the held-out test set, and no information from the training set affected the test sequences. However, non-overlapping sequences should be tested in the future to completely remove this threat [21].

The second threat is the danger of overfitting when training the CNN-LSTM network. To avoid overfitting, the number of epochs was set to 10 with early stopping, and the learning rate was reduced. The training and validation loss were close to each other (the difference between training and validation

loss was 0.011 at the final epoch), which indicated that overfitting was prevented. There are 478,081 trainable parameters in the model, though, which is high, compared to the 139,899 sequences used for training (approx. 3.4:1). In deep learning, the ratio is satisfactory for the large number of parameters used by the model, but if there will be more data it will be easier to generalise. The architecture was then subjected to two dropouts: one with 0.3 and another with 0.15, as a preventive measure for overfitting [29].

The third threat is the random selection of the 199,998 rows from the entire CIC-DDoS2019 dataset. Random sampling keeps the statistical characteristics of the data intact in the expected value sense, but it is possible that a random sample does not perfectly represent the data. To reduce this threat, the random seed was set (*random_state=42*) to assure reproducibility, and sampling was done only once instead of several times. The results indicated high test performance, suggesting that the sample was representative; however, further research should verify the results with the complete data set or with several random samples to evaluate the stability of the results [21].

4.2.4.2. External Validity

The most significant limitation of the external validity is the implemented use of the CIC-DDoS2019 dataset, which can be considered realistic yet synthetic. The network traffic in the dataset has been generated under controlled conditions in the lab with traffic generators and attack tools. In the real-world network traffic is more variable such as encrypted traffic, background noise, user changes of behaviour, or legitimate traffic surges that may appear as attacks. The proposed model might not work as intended on the live production traffic. Future research should confirm the effectiveness of the framework, ideally in a real deployment, as a sidecar application that is able to observe traffic without blocking, and validate the framework's performance on actual user action for the purpose of automated mitigation [21].

The hardware configuration used for experiments (16 GB RAM, 8-core Intel Core i7) is representative of a mid-range server, but results may differ on other hardware. One factor in particular, inference latency, is highly dependent on CPU clock-speed, memory bandwidth, cache size, and GPU availability. The latencies reported should be treated as a relative value, not an absolute one. The code is public to enable this testing. The latency value can vary across instances that have different CPUs, such as the AWS Graviton compared to the AMD EPYC [8].

The tests were conducted in a virtual environment consisting of Docker and Kali Linux operating systems running in an isolated virtual network without any hops, firewalls, load balancers, or

CDNs to capture the complexity of production networks. This module for mitigations is based on the assumption that the user manages the network stack of a Linux server. These would be different mitigation mechanisms (such as API calls to security groups, Network ACLs, or WAF rules) in a cloud environment like AWS, Azure, or Google Cloud. The modular structure of the mitigation module makes adaptation to different environments easier by hiding the blocking mechanism within a simple interface. Implementing cloud-specific adapters is straightforward [34][35].

Another threat to external validity is the fact that the dataset is not representing the legitimate traffic. The CIC-DDoS2019 dataset is mostly made up of unencrypted HTTP traffic, while modern websites are increasingly using HTTPS. Content of the payloads is encrypted, so features like the HTTP request methods and URL structure will not be visible to the detection model. There are, however, a number of features that are still available in encrypted traffic in this research, which are based on TCP/IP headers, not application contents, including packet length, inter-arrival times, flag counts, and flow duration. The model should be generalised to HTTPS traffic, however, this needs to be empirically verified [21].

4.2.4.3. Construct Validity

A threat to construct validity is related to binary classification formulation (BENIGN versus ATTACK). This is suitable to use while attempting to block all types of traffic in real-time, whether the traffic is malicious or not. However, it does not contain any useful information about the attack type which could be useful for forensic analysis or for implementing type-specific mitigation tactics (for instance, rate only rates HTTP floods, drops only SYN floods). Future research could be extended to multi-class classification to classify particular attack types which would give more operational value to security analysts [1].

The metrics used might not adequately reflect the practical use of the system. In the existing literature, some metrics, such as accuracy, precision, recall, F1-score, and AUC, are commonly used, without considering the cost of false positive vs. false negatives. In security applications, false positive (blocking legitimate users) tend to be more expensive than a false negative (missed attacks), although the ratio varies based on the application context. If there are any false positives blocking legitimate transactions, then a financial trading platform can't handle this. However, it is acceptable for a research network to have false positives from time to time. This is because recall was the metric that emphasized the model's capability of detecting attacks, and in many cases, it is crucial for security purposes rather

than availability. In this context, the recall metric value (99.10% for CNNLSTM) showed the model's capability to detect attacks.

The measurements of inference latency were performed on a separate machine where no competing processes were used with the machine. In production, the web server may be performing other activities like database queries, logging, handling static files and other security-related tasks, and these activities may be using the same CPU and memory resources. These reported latencies should therefore be regarded as a best case estimates. For production deployments, dedicated CPU cores for the detection model are recommended or to ensure consistent performance, accelerate the model using the GPUs. To manage traffic peaks without impacting latency budgets, it is possible to implement monitoring and auto-scaling [8].

For the real-time mitigation tests, simulated traffic was generated using "hping3" and this might not reflect the actual features of real attacks. The attacks from the wild employ increasingly refined methods, such as randomizing requests, multiple source IPs from botnets and SSL encryption that escapes some of the detection capabilities. The results for live attacks may be different than simulated results for the mitigation module. Even though this test was not 100% accurate, however, it was useful for comparative evaluation. The module should be evaluated during real attacks in a controlled environment in the future ideally using captured attack traces from real incidents [25].

4.2.4.4. Mitigation Strategies Employed

Multiple mitigations strategies were implemented to reduce the threats listed above. Cross-validation of the hyperparameters was performed during hyperparameter tuning to guarantee that the hyperparameters selected throughout the training process generalise between different splits of the data, which is essential for the internal validity. To identify hyperparameters, the test set was kept hidden from the development process. The CIC-DDoS2019 dataset was selected for external validity as it is a well-known benchmark dataset that can be compared with other studies. All the hardware specifications are described in detail to enable it to be replicated on other hardware. To achieve construct validity, the paper's features reported a range of different metrics such as accuracy, precision, recall, F1 score, AUC, latency and resource usage, in order to get a comprehensive overview of model performance from multiple angles [21][8].

The ongoing threats need to be addressed in future research to validate the framework with real production traffic across various hardware platforms, real-world attacks, and diverse datasets like CIC-IDS2017 or CSE-CIC-IDS2018. Source code is made open-source on GitHub, so that others

researchers can replicate and extend the code. To increase the impact and reproducibility of this research, principles of open science were practiced, with code being released, data cited and detailed methodology described. The practices allow the research community to develop on this work and to overcome the limitations in the next scientific research.

4.3 Conclusions

This section overviews the principal contributions of the research and highlights the conclusions made based on the results from the experiment. The section is organized into four subsections. Subsection 4.3.1 overviews the obtained results after completion of the thesis tasks. Subsection 4.3.2 overviews the conclusions made based on the results from the experiment. Subsection 4.3.3 provides recommendations for practitioners and investigators. Subsection 4.3.4 makes recommendations for future work to broaden and improve the framework defined by the present research [1].

4.3.1. Summary of Results

The successful completion of the thesis tasks produced a number of tangible outcomes that represent the contribution of this work to the area of DDoS detection and mitigation. A complete, isolated testing environment was established using Docker containerisation, Kali Linux, and the traffic generation tool hping3. This environment allows the emulation of multivector DDoS attacks against Web applications in a controlled manner that does not affect production systems. All attack types from the CIC-DDoS2019 dataset were successfully reproduced within this environment and the topography was arranged in a manner allowing the capture of all network traffic for subsequent analysis. This environment can be reused for future experiments and testing of alternative detection techniques [31].

The CIC-DDoS2019 dataset was loaded successfully, preprocessed and sampled to 199998 rows, in order to maintain the variety of attack types. A full data preprocessing pipeline was designed, which dealt with missing values in the Flow Bytes/s feature, dropped 89 infinite values from the Flow Packets/s feature, dropped 22 non-numeric columns such as IP addresses and timestamps, and normalised all the features to zero mean and unit variance with StandardScaler. Once the 21 redundant features with correlation values larger than 0.85 with other features were removed, a total of 44 numeric features remained. This workflow has been documented fully so that it can be applied to other network traffic datasets [21].

Two baseline machine learning models were trained and tested on the preprocessed data. The Random Forest classifier with 100 trees and depth of 20 led to a classification accuracy of 99.98%, precision of 100.00% and recall of 99.98% on a 30000-sample test set. The SVM classifier with linear

kernel and $C=1.0$ had a classification accuracy of 96.50%, precision of 96.00% and recall of 95.80%. Random forest required 12.5 ms per batch of 64 samples whereas SVM required 45.8 ms for inference. These models were used to compare with the hybrid model implemented in this paper [26][27].

The hybrid CNN-LSTM model architecture consisted of two convolutional layers (64 and 128 filters), two LSTM layers (64 and 32 units), and two dense layers (128 and 64 nodes) with 478, 081 trainable parameters. The model was trained using a sliding window to slice 100 sequential flows from the input data into the model, resulting in 139,899 training sequences, 29,901 validation sequences, and 29,901 test sequences. The model was trained for 24 epochs with early stopping; a high final test accuracy of 98.70%, precision 98.50%, recall 99.10% and F1-score 0.988 was achieved. The consistency of training and validation curves indicated that the model had generalised effectively (no overfitting) [7].

A real-time mitigation module was designed and integrated with the trained CNN-LSTM model. This module implemented a rate limiter that enforces rate limits of 100, 50, or 10 for rate of requests per second, according to the attacker's CNN-LSTM confidence scores of below 0.6, 0.6-0.85, or above 0.85, respectively. An iptables-based traffic filter was used to block IPs when the CNN-LSTM confidence score exceeds 0.85 for 300 seconds. At a traffic rate of 1, 000 requests per second, 5, 000 requests per second, and 10, 000 requests per second, the overall block rate of 99.2%, 98.7%, and 97.5% with false positive rates of 0.3%, 0.5%, and 0.8% was achieved. End-to-end latency at 10, 000 requests per second was 78.4 msec., which is under the 100 msec latency threshold for real-time deployment [25].

The performance comparison showed that the CNN-LSTM outperformed the other two models, being the best at balancing high precision (98.50 %) with high recall (99.10 %). Random Forest attained a more accurate prediction, but since it classifies each flow separately it is unable to learn temporal dependencies unlike the CNN-LSTM which is necessary for detecting the attack; CNN-LSTM inference latency of 28.3ms per 64 samples is capable of being integrated in a deployed realtime scenario, compared to Random Forest 12.5ms and SVM 45.8 ms. The most important features for DDoS detection were backward-directed Bwd Packets/s and URG Flag Count [8].

Analyzing resource overheads established that the CNN-LSTM consumes 28.5% of CPU and 1.2 GB of RAM on CPU-only hardware, accommodating these requirements easily by typical modern server standards. GPU acceleration with an NVIDIA RTX 3060 processor resulted in reduction of CPU usage to 8.2%, while inference latency was reduced to 8.7 ms per batch, a 3.25x speedup. RAM usage

increased slightly to 1.25 GB to incorporate the GPU buffers. These results show the framework to be suitable for use in production, with the GPU-accelerated solution achieving performance close to that of the Random Forest while still enabling contextual temporal detection [29].

4.3.2. Key Findings

Based on the experimental results and analysis outlined in this thesis, a number of key conclusions can be made about the hybrid CNN-LSTM technique for real-time DDoS detection and mitigation. The hybrid setup clearly captures the spatial and temporal trend within network traffic and is able to obtain 99.10 % recall on the CIC-DDoS2019 test set, evidence that convolutional layers for spatial feature recognition and recursive layers for temporal modeling is a promising method to catch advanced, multi-packet Layer 7 DDoS attacks [7].

Although the Random Forest baseline is close to perfect with an accuracy of 99.98%, it does not take the evolution of individual flows over time into account and will therefore not be accurate if a Low and Slow attack is not detectable at the single flow level. For an application where the attack must be detectable at the Low and Slow attack stage then a CNN-LSTM is the better solution despite its slightly lower accuracy of 99.70%. The difference of 0.28% in the accuracies of the two models is not significant compared to the additional information gained from modeling the evolution of attacks over time with the CNN-LSTM [26].

The Support Vector Machine with linear kernel yielded the poorest results on this task, with 96.50 % accuracy and 95.80% recall (linear kernel is optimal for linearly separable data). Whether an RBF kernel would have outperformed the results of no kernel (though experimenting is computationally unfeasible with a 139998 sample training set) is unknown; the SVM has a high inference latency of 45.8ms, making it difficult to deploy in practice. As an alternative learning algorithm, the SVM provides an interesting performance baseline: it embodies a completely different learning framework than ensemble and deep learning approaches [27].

The real-time mitigation module prevents attack traffic with a low false positive rate for traffic levels up to 10,000 requests per second. Although not depicted, block rate is above 97.5% even at 10,000 requests per second, with end-to-end latency of less than 80 ms. The dynamic rate limiter stops any IP address from over-whelming the victim server by incrementally applying increased rate limits based on the value of confidence score provided by the model. The modular mitigation module can be adapted to different deployment scenarios (cloud environment, in data center on-premise server) [25].

The resource overhead analysis validates the feasibility for the proposed CNN-LSTM framework to be deployed on contemporary server hardware. For the CPU-only case, the resource utilization is 28.5% of CPU and 1.2 GB of RAM, appropriate for a relatively low-end server with 16 GB of RAM. For the GPU-accelerated case, the resource utilization is 8.2% of CPU with an end-to-end latency of 8.7 ms, highly practical for production use. Furthermore, the open-source implementation of the framework can be freely used by any organisation regardless of budget, as it incurs no licensing costs [29].

4.3.3. Recommendations

Drawing on the results of this study, several recommendations are presented for practitioners interested in using the proposed framework and researchers interested in building on this work. For organizations interested in using the CNN-LSTM in a production setting, the use of GPU acceleration is recommended for traffic flows greater than 5,000 req/sec. GPU acceleration provides a 3.25x improvement in speed, faster latency of 8.7 compared to 28.3 milliseconds, and makes real-time detection feasible at that higher throughput. For organizations without GPU hardware, the CPU-only implementation is sufficient up to 5,000 req/sec in light of the consistent latency readings at that rate [8].

The deciding factor between utilizing Random Forest or CNN-LSTM should be the nature of the threat environment. Environments dealing with volumetric threats with established individual flow behavior should utilize a Random Forest ensemble as it results in better detection time and consumes fewer resources. However, for environments dealing with advanced Layer 7 threats such as Low and Slow HTTP floods, the CNN LSTM's temporal correlation offers a unique advantage and will be worth the additional resource consumption. Combining the two models, where the Random Forest acts as a fast filter and CNN LSTM as an intensive inspection tool for awaiting suspicious flows would be the optimum solution [26].

The thresholds for mitigation (confidence scores of 0.6 and 0.85) should be tuned for the application depending on its tolerance for false positives versus false negatives. For financial or healthcare applications in which blocking users would not be acceptable, this threshold could be raised to 0.9 or 0.95. For a research network or other less-critical application where missing an attack would be more damaging, this threshold could be lowered to use 0.8 for severe mitigation. The implementation of the rate limiter used is modular, hence the numbers could be easily changed without retraining the model [25].

The set of 44 features used in this study serves as a baseline for other DDoS detection projects. The correlation analysis revealed that the removal of attributes with correlation greater than 0.85 reduces redundancy without losing any ability to predict, so the same analysis should be performed for each network in order to identify dataset-specific redundancy. The list of strongest predictors determined by Random Forest importance (Inbound, URG Flag Count, Bwd Packets/s) are likely to be useful in many network settings [21].

Researchers reusing or replicating the work can find all code released to GitHub for complete reproducibility. The preprocessing pipeline, model architectures, training procedures, evaluation metrics and mitigation module can all be reused on other datasets. When deploying the framework on another dataset, the sequence length (here of 100 flows) may need to be adjusted depending on the network the dataset was extracted from. Shorter sequences (50 flows) for fast attacks, while longer sequences (200 flows) for very slow attacks [29].

4.3.4. Future Work

Several avenues for future investigation are suggested by the limitations and results of this thesis. The immediate one would be to test the resulting framework with live production traffic rather than synthetic data. Since the CIC-DDoS2019 dataset is the authoritative benchmark used by the research community, the varied nature of live network traffic, entailing encrypted data and background noise as well as benign peak traffic patterns, would be valuable at assessing the robustness of this framework. Additionally the deployment of the framework as a sidecar monitor on a production webserver would be extremely interesting, especially for gauging the models performance on zero-day attacks not present in any dataset [21].

Another significant extension would involve evolving the binary classifier into a multi-class classifier that predicts the prevalent attack type. The baseline model does not distinguish type of attack, only attack/no-attack. Identifying the attack category would allow different treatment for different attack types. For instance, HTTP floods could be mitigated by throttling, while TCP floods could be mitigated by connection keeping. To do this, the output layer would need to evolve from one sigmoid classifier to multiple softmax classifiers and the network retrained on the attack type labels [1].

Additionally, ensembling multiple detection models may offer the best accuracy and robustness. Combining Random Forest, CNN-LSTM and some other model XGBoost into an ensemble voting system could far exceed the accuracy of the individual systems while maintaining sufficient diversity. In a stacking approach, the CNN-LSTM sequence outputs would be used as features to a Random

Forest classifier to benefit from the best aspects of each. Ensembles would be significantly more computationally expensive (budget permitting, GPU acceleration could be used) but initial tests have shown accuracy to be achievable above 99.5 percent [26].

Employing newer transformer-based architectures, such as BERT or its derivatives designed for time-series, would also be of interest. Transformers use attention mechanisms to learn than-longer range dependencies than an LSTM with a 100-flow window. They have set state-of-the-art performance on natural language processing tasks, and are increasingly applied to time-series classification tasks. A transformer applied to network traffic would likely be able to capture longer-range dependencies than the 100-flow context window of the LSTM, and the attention weights would give an interpretable indication of what was contributing to each classification. However, transformers are more computationally expensive than an LSTM network [8].

Another possible approach to train a DDoS detection model in the case of multiple organisations is federated learning. In this approach, organisations would train the CNN-LSTM model local to their network and share only the model updates to a central server. This approach would allow for a detection model to learn from multiple networks' traffic, while allowing organisations not to share traffic data. Additionally, this approach would allow for a broad generalisation, as the CNN-LSTM model could learn from different organisation's legitimate traffic, in addition to the attack traffic. Initial experiments on simulated federated settings could be performed by partitioning the CIC-DDoS2019 dataset on virtual organisations [34].

Ultimately, combining the detection mechanism with the software-defined networking (SDN) controllers would facilitate more advanced mitigation approaches. In a SDN infrastructure, the detection component could instruct the controller to reconfigure switches to drop attack traffic at the network edge, reducing the web server's load, and preventing attack traffic taking up bandwidth along internal links. The dynamic rate limiter could then become an OpenFlow rather than an iptables rule, allowing the centralized management of a number of server [8].

4.3.5. Conclusion

This thesis proposed, designed, tested and deployed a hybrid deep learning framework capable of detecting and mitigating web based DDoS attacks in real-time, using a combination of CNN and LSTM models. The model attempted to learn spatial features from packet headers, in addition to the evolution of traffic features over time. A real-time mitigation module used real-time confidence scores to determine the extent of throttling required using rate limiters and IP blocks. The experimental setup

used isolated machines built in Docker containers running Kali Linux and hping3, tested the system while systematically comparing it against RF and SVM classifiers.

The test results of CNN-LSTM were 98.70%, 98.50%, 99.10% and 0.988 for accuracy, precision, recall and F1-score respectively on the CIC-DDoS2019 test set. The baseline RF classifier had higher accuracy (99.98%) but did not perform as the temporal model, since the flow was not modeled and it considered each flow independently. The SVM with linear kernel scored worse (96.50% accuracy, 95.80% recall) and it had higher latency during inference. The CNN-LSTM's ability of reasoning temporally makes it a more suitable model for multi-packet, multi-flow attack detection of complex Layer 7 attacks.

The real time mitigation had block throughput of 99.2%, 98.7%, and 97.5% at traffic rate of 1,000, 5,000, and 10,000 requests/sec respectively with false positive rate less than 0.8% and end-to-end latency less than 80 ms. Resource overhead analysis indicates CNN-LSTM consumes 28.5% of CPU and 1.2 G RAM resource on CPU only platform, and inference latency of each batch is reduced to 8.7 ms with GPU accelerating. The most discriminative features were backward directed (Bwd Packets/s) and flag-based (URG Flag Count).

The proposed system can be deployed on modern server hardware, it can be used to support up to 5,000 requests per second, for the CPU-only version. The open-source version is free of charge as compared to commercial DDoS protection. It has some restrictions such as using synthetic CIC-DDoS2019 data instead of production traffic, and only binary classification without distinguishing specific attack types. Future work will involve validation on live traffic, extension of multi-class classification and software-defined networking controllers integration.

Ultimately, this thesis shows that a hybrid CNN-LSTM deep learning architecture, augmented with dynamic rate limiting and traffic filtering, offers a robust, online, and affordable solution for DDoS classification and mitigation of a web application. The system is shown to have a high accuracy, a high recall and a low false positive rate with an acceptable inference latency, thus offering a concrete contribution to the field of web application security.

5. REFERENCES

- [1] S. T. Zargar, J. Joshi, and D. Tipper, "A Survey of Defense Mechanisms Against Distributed Denial of Service (DDoS) Flooding Attacks," *IEEE Communications Surveys and Tutorials*, vol. 15, no. 4, pp. 2046–2069, 2013. doi: 10.1109/SURV.2013.031413.00127
- [2] Cloudflare, "DDoS Threat Report 2023 Q4," Cloudflare, Inc., 2023. [Online]. Available: <https://blog.cloudflare.com/ddos-threat-report-2023-q4/>. [Accessed: Apr. 2026]
- [3] Corero Network Security, "DDoS Threat Intelligence Report 2021," Corero Network Security, London, UK, Annual Threat Report, 2022. [Online]. Available: https://www.corero.com/wp-content/uploads/2024/04/Corero-DDoS-Threat-Intelligence-Report-2021_final.pdf. [Accessed: Apr. 2026]
- [4] A. Praseed and P. S. Thilagam, "DDoS Attacks at the Application Layer: Challenges and Research Perspectives for Safeguarding Web Applications," *IEEE Communications Surveys and Tutorials*, vol. 21, no. 1, pp. 661–685, 2019. doi: 10.1109/COMST.2018.2870658
- [5] Amazon Web Services, "AWS Shield Threat Landscape Report Q1 2020," AWS, Seattle, WA, 2020. [Online]. Available: <https://aws.amazon.com/shield/ddos-attack-protection/>
- [6] M. H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita, "Network Anomaly Detection: Methods, Systems and Tools," *IEEE Communications Surveys and Tutorials*, vol. 16, no. 1, pp. 303–336, 2014. doi: 10.1109/SURV.2013.052213.00046
- [7] C. Yin, Y. Zhu, J. Fei, and X. He, "A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks," *IEEE Access*, vol. 5, pp. 21954–21961, 2017. doi: 10.1109/ACCESS.2017.2762418
- [8] T. A. Tang, L. Mhamdi, D. McLernon, S. A. R. Zaidi, and M. Ghogho, "Deep Learning Approach for Network Intrusion Detection in Software Defined Networking," in *Proc. 2016 International Conference on Wireless Networks and Mobile Communications (WINCOM)*, Fez, Morocco, 2016, pp. 258–263. doi: 10.1109/WINCOM.2016.7777224
- [10] C. Douligeris and A. Mitrokotsa, "DDoS Attacks and Defense Mechanisms: Classification and State-of-the-Art," *Computer Networks*, vol. 44, no. 5, pp. 643–666, 2004. doi: 10.1016/j.comnet.2003.10.003
- [11] C. Rossow, "Amplification Hell: Revisiting Network Protocols for DDoS Abuse," in *Proc. 21st Annual Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, Feb. 2014.

[Online]. Available: <https://www.ndss-symposium.org/ndss2014/amplification-hell-revisiting-network-protocols-ddos-abuse>

- [12] V. Paxson, "Bro: A System for Detecting Network Intruders in Real-Time," *Computer Networks*, vol. 31, no. 23–24, pp. 2435–2463, 1999. doi: 10.1016/S1389-1286(99)00112-7
- [13] M. H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita, "Surveying Port Scans and Their Detection Methodologies," *The Computer Journal*, vol. 54, no. 10, pp. 1565–1581, 2011.
- [14] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A Detailed Analysis of the KDD CUP 99 Data Set," in *Proc. 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, Ottawa, ON, Canada, 2009, pp. 1–6. doi: 10.1109/CISDA.2009.5356528
- [15] B. Mukherjee and H. Sharma, "Network Intrusion Detection System Using Random Forest," *International Journal of Computer Applications*, vol. 116, no. 18, pp. 12–17, 2015. doi: 10.5120/20426-2769
- [16] R. Lippmann, J. W. Haines, D. J. Fried, J. Korba, and K. Das, "The 1999 DARPA Off-Line Intrusion Detection Evaluation," *Computer Networks*, vol. 34, no. 4, pp. 579–595, 2000. doi: 10.1016/S1389-1286(00)00139-0
- [17] J. Hao and T. Ho, "Machine Learning Made Easy: A Review of Scikit-Learn Package in Python Programming Language," *Journal of Educational and Behavioral Statistics*, vol. 44, no. 3, pp. 348–361, 2019. doi: 10.3102/1076998619832248
- [18] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735
- [19] J. Kim, N. Shin, S. Y. Jo, and S. H. Kim, "Method of Intrusion Detection Using Deep Neural Network," in *Proc. 2017 IEEE International Conference on Big Data and Smart Computing (BigComp)*, Jeju, South Korea, 2017, pp. 313–316.
- [20] R. Zhao, R. Yan, Z. Chen, K. Mao, P. Wang, and R. X. Gao, "Deep Learning and Its Applications to Machine Health Monitoring," *Mechanical Systems and Signal Processing*, vol. 115, pp. 213–237, Jan. 2019. doi: 10.1016/j.ymssp.2018.05.050
- [21] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy," in *Proc. 2019 International Carnahan Conference on Security Technology (ICCST)*, Chennai, India, Oct. 2019, pp. 1–8. doi: 10.1109/CCST.2019.8888419

- [22] R. Zuech, T. M. Khoshgoftaar, and R. Wald, "Intrusion Detection and Big Heterogeneous Data: A Survey," *Journal of Big Data*, vol. 2, no. 3, pp. 1–41, 2015. doi: 10.1186/s40537-015-0013-4
- [23] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002. doi: 10.1613/jair.953
- [24] R. Partheepan, "Detecting Phishing Websites Using Machine Learning," Master Thesis, Vytautas Magnus University, Faculty of Informatics, Kaunas, Lithuania, 2022. Author full name: Ravinthiran Partheepan.
- [25] A. Salvatore, "hping3 — Active Network Security Tool," 2014. [Online]. Available: <http://www.hping.org/hping3.html>. [Accessed: Apr. 2026]
- [26] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. doi: 10.1023/A:1010933404324
- [27] C. Cortes and V. Vapnik, "Support-Vector Networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995. doi: 10.1007/BF00994018
- [28] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi: 10.1109/5.726791
- [29] F. Chollet, *Deep Learning with Python*, 2nd ed. Shelter Island, NY: Manning Publications, 2021. ISBN: 9781617296864. [Online]. Available: <https://www.manning.com/books/deep-learning-with-python-second-edition>
- [30] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in *Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Savannah, GA, 2016, pp. 265–283. [Online]. Available: <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>
- [31] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html>
- [32] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proc. 9th Python in Science Conference (SciPy)*, Austin, TX, 2010, pp. 56–61. doi: 10.25080/Majora-92bf1922-00a
- [33] M. Waskom, "Seaborn: Statistical Data Visualization," *Journal of Open Source Software*, vol. 6, no. 60, p. 3021, 2021. doi: 10.21105/joss.03021

[34] Docker Inc., "Docker Documentation — Get Started," 2024. [Online]. Available: <https://docs.docker.com>. [Accessed: Apr. 2026]

[35] Offensive Security, "Kali Linux Official Documentation," 2024. [Online]. Available: <https://www.kali.org/docs/>. [Accessed: Apr. 2026]

6. APPENDICES

6.1 Appendix A: Research Project Summaries

6.1.1. Research Project Summary No. 1

Title: Personal Identity Recognition-Based Locking and Unlocking Solution System

Goal: For this project, the objective was to develop a secure system that employs personal identity recognition applications for locking and unlocking. The objective was to formulate a working prototype that could recognize and authenticate users based on their personal identification, and compare the efficiency of the said prototype with that of the conventional mechanical locking systems.

Method: First, the research of personal authentication technologies was carried out through literature survey. Technologies such as fingerprint scanner, RFID cards, PIN numbers, facial recognition were covered in this survey. Then, as a solution, fingerprint authentication was chosen in the implementation phase as compromise between security, cost and ease of use. The implementation included the design of a system consisting of Fingerprint scanner, microcontroller Arduino and the authentication mechanism. The authentication system enrolled the fingerprint by obtaining minutiae points in the fingerprint and storing it into the memory and authenticates by capturing live fingerprint, obtaining minutiae points and authenticating by matching the obtained minutiae points with those pre-stored. The system controlled the locking mechanism through a servo motor. The evaluation of the system was performed with a test bed of 10 users enrolled with five samples each. Finally, the efficiency of the system was compared with traditional key lock mechanism and a combination lock mechanism based on ease of use, authentication time.

Results: The fingerprint based lock system registered an average check-in time of 2.4 seconds, a false acceptance rate of 3.8%, and a false rejection rate of 5.2%. Conventional key based lock systems do not have any false acceptance or rejection (either the key opens or it doesn't) but have the disadvantages of lost keys and duplicated keys. Combination locks had an average unlocking time of 8.7 seconds and are vulnerable to observation attacks. User surveys revealed that the fingerprint system was significantly more convenient than carrying keys (85% of users stated that they preferred it) but that people were worried about the false rejection. A working prototype was demonstrated and it was confirmed that biometric locking systems are feasible for home and small office environments.

Relation to Final Thesis: This project built the foundational knowledge in security system design and biometric authentication and comparing baseline performance. The familiarity with false acceptance and false rejection allowed us to select the metrics for comparing baselines in our final thesis. Recall and precision (avoiding false negatives and false positives respectively). Knowing how to compare an arbitrary security system against traditional baselines allowed us to design our final thesis experiment comparing a hybrid model to classical SVM and Random Forest models. This project also exemplified the strength of using multiple biometric indicators (implemented as unimodal fingerprint) against a unimodal indicator (motivating the hybrid presentation in our later thesis project [24]).

6.1.2. Research Project Summary No. 2

Title: Hybrid Authentication for Enhanced Security: Integrating RFID with Fingerprint-Based Door Lock Systems.

Goal: This research project sought to enhance the security and reliability of the fingerprint-only locking system of Research Project No. 1 by adding RFID authentication. This study sought to prove that this hybrid system (involving two different authentication factors: token or “something you have” and biometric or “something you are”) could show a large reduction in false acceptance or rejection rates and an increase in spoofing resistance compared to the unimodal fingerprint locking system.

Method: A simple literature review was undertaken on biometry and RFID-based authentication systems. The limitations of the fingerprint-only authentication system devised for Research Project No. 1 were examined where its false acceptance rate was found to be 3.8% and its false rejection rate was found to be 5.2%, whilst it was also susceptible to spoofing with silicone fingerprints. A hybrid authentication system was then designed incorporating the R307 finger scanner from Research Project No. 1 with an RFID authentication system based on the RC522 chip from a common RFID reader hardware module. The hybrid system was implemented using a fully programmable ATmega 2560 microcontroller from the Arduino Mega. An authentication required two factors to be present; a valid RFID card and a valid, matching fingerprint. Experimental data on accuracy, response times and spoof resistance was gathered by a controlled experiment using 100 fingerprint samples and 50 RFID cards. The fingerprint-only system was then compared with the hybrid system using the above metrics. An evaluation of the usability of the two systems was then undertaken through user surveys of 20 participants. A cost benefit analysis was also performed to determine whether the additional cost of the RFID hardware module was justified.

Results: The false acceptance rate of the hybrid fingerprint- RFID system was 1.2% and the false rejection rate was 2.4%. This represents a 68% reduction in false acceptances and 54% reduction in false rejections when compared to the fingerprint-only system. The mean response time when using the hybrid system was 1.8 seconds, fulfilling the design criterion of two seconds or less for deployment. Spoofing resistance increased greatly when using the hybrid system; only 98.5% of silicone fingerprint spoofing attempts were successfully made using the hybrid system and 85.3% using the fingerprint-only system. User opinions on security and reliability were highest when using the hybrid system; 18 out of 20 users preferred the hybrid system while presenting an RFID card with the fingerprint. The cost- benefit analysis showed that the additional cost of the RFID module was justified.

Relation to Final Thesis: This project gave a firsthand practical experience of the comparative study of a baseline unimodal system (fingerprint only) against an extended hybrid system (fingerprint + RFID) which directly inspired the experimental framework of the final thesis. The comparison of baseline models against an extended hybrid model was dedicated to the experimental framework of the final thesis with the random forest and SVM baseline models [17] systematically compared against the hybrid CNN-LSTM model which proved the fundamental belief of hybrid detection system outperforming a unimodal system is always true. The principle was successfully carried over from physical access to network security in the final thesis [24].

6.1.3. Research Project Summary No. 3

Title: AI-Driven Deepfake and Spoof Detection for Robust Face-Voice Biometric Authentication

Goal. The objective of this research project was to investigate the capability of pre-trained deep learning models in itself for deepfake and spoofing attack detection in face and voice multimodal authentication scenario. The focus was on showing that a combination of multiple modalities leads to a better detection accuracy than that of individual ones, thus generalising the hybrid principle of physical authentication to digital media authentication.

Methods: Literature survey was carried out to find appropriate pre-trained deep learning models for face and voice deepfake detection. Two selected models were a deep learning based convolution neural network (CNN) face detection model(FaceForensics++) and a deep learning based recurrent neural network (RNN) voice detection model (RawNet2). A standard, publicly available multimodal dataset with bona fide samples, and three types of attacks (printed photo, screen replays, pre-recorded audio) was obtained. The selected models integrated into the evaluation software system in Python and

PyTorch. Various tests were performed to determine accuracy of each model individually, against the attack types of interest. Basic score-level fusion was performed by averaging confidence scores of both models to show how the multimodal system could be advantageous.

Results: They can be summarized as, firstly, the face detection model gave 94.2% accuracy on printed photo attacks and 91.7% accuracy on screen replay attacks, secondly, the voice detection model gave 89.5% accuracy on recorded audio attacks, thirdly, score-level fusion of the two modalities increased the overall detection accuracy to 96.8%, showing that the hybrid model outperformed each individual modality, and ultimately the evaluation software framework was implemented, including all pre-processing pipelines for face and voice modalities. The main conclusion arising from these results was that the hybrid multimodal approach provides a more complete defence than any individual modality, establishing the key tenet of the earlier Research Project No. 2.

Relation to Final Thesis: This project generalized the hybrid detection method from physical form authentication (Fingerprint+RFID) to digital media authentication (Face+Voice). The hybrid architecture experience of combining two complementary detection mechanisms inspired the hybrid CNN-LSTM architecture for the final thesis, where CNN and LSTM modules complement each other to learn the spatial and temporal features respectively. The evaluation methodology (accuracy, precision, recall, and inference time) adopted in this project was directly implemented on the DDoS detection models. The research story was ended by this project: hybrid detection in physical access, digital media and network security [24].

6.2 Appendix B: Code Repository Link

All source code associated with this thesis has been published as part of a project web page and version controlled repository. The project web page is available at <https://kassim206.github.io/hybrid-cnn-lstm-ddos-detection/>

. The web page contains a comprehensive overview of this thesis including the abstract, methodology, summary of results, and figures. The full source code repository is available at

<https://github.com/kassim206/hybrid-cnn-lstm-ddos-detection>

. This repository hosts the complete data pre-processing pipeline, the baseline models implemented (Random Forest and Support Vector Machine), the hybrid CNN-LSTM code, the real-time mitigation module, and all scripts used for evaluation. Each function and parameter has been commented in the code. The repository also contains a requirements.txt document listing all Python code dependencies and a README.md document providing detailed instructions for replicating all experiments. A stable release corresponding to the final submission of this thesis is available at

<https://github.com/kassim206/hybrid-cnn-lstm-ddos-detection/releases/tag/v1.1-thesis-experiments>

. The code has been published under the MIT license to enable reuse by other researchers. All source files are also available on the optical disc attached to this thesis.

6.3 Appendix C: Declaration of Generative AI Use

Name, surname: Mohammed Kassim Cherukodan

Study programme, year: Applied Informatics (Master's), 2026

Type of written assignment and study course: Master Thesis (Final Thesis)

Title of written assignment: Hybrid Machine Learning-Based Real-Time DDoS Detection and Mitigation for Web Applications

Declaration date: 2026-05-22

I confirm that the written assignment submitted for evaluation was prepared independently in accordance with VMU Study Regulations¹, VDU Code of Ethics², VMU Plagiarism Prevention Policy³, VMU IF Guidelines for the Responsible Use of Artificial Intelligence⁴.

I would like to point out that GenAI tools, such as ChatGPT, MS Copilot, Gemini, Grammarly, DALL-E, Midjourney, and others, are useful for writing assignments (*mark*):

☐ Were used; ☐ Were not used.

Where GenAI tools were used in the preparation of this work, I provide accurate and detailed information regarding the purposes and scope of their use:

- Use cases:

Document chapter and page	Purpose of use	Name, version and URL of GDI tool and main prompt(s)
All chapters (throughout)	Grammar and language correction. Paragraphs written by the author had been checked for grammar and were easily	DeepSeek (DeepSeek-V3), March 2026 (https://deepseek.com), prompt: "Check grammar and clarity of

¹ VMU Study Regulations. Approved by Resolution No. SEN-N-49 of the Senate of Vytautas Magnus University on June 18, 2024 (new version). <https://www.vdu.lt/wp-content/uploads/2024/08/Vytauto-Didziojo-universiteto-studiju-reguliaminas.pdf>

² VMU Code of Ethics. Approved by Resolution No. SEN-N-3 of the Senate of Vytautas Magnus University on February 28, 2024. https://www.vdu.lt/wp-content/uploads/2024/03/Etikos-kodeksas_2024-1.pdf

³ VMU Plagiarism Prevention Policy. APPROVED by Resolution No. SEN-N-35 of the Senate of Vytautas Magnus University on May 25, 2022. https://www.vdu.lt/wp-content/uploads/2023/03/Plagiato-prevencijos-nuostatai_VDU.pdf

⁴ VMU IF Guidelines for the Responsible Use of Artificial Intelligence. APPROVED at the meeting of the Council of the Faculty of Informatics, Vytautas Magnus University, on February 19, 2026.

	understandable by a general academic population. None of the paragraph writings was created by AI. All paragraphs were written by the author.	this paragraph"
Chapter 2, pages 20–36	Literature search. Finding the appropriate search keywords and direction for literature review sections on the existing research on DDoS detection, CNN-LSTM and network security. All content were independently found, read and reviewed by the author.	DeepSeek (DeepSeek-V3), March 2026 (https://deepseek.com), prompt: "What are the key search terms for literature on hybrid CNN-LSTM for DDoS detection?"
Chapter 3, pages 37–53; Chapter 4, pages 55–73; Appendix B (code repository)	Code aid. Debugging and syntax help for particular implementation details. Any code that was ultimately implemented was written, tried and checked by the author himself. No code was written by AI.	DeepSeek (DeepSeek-V3), March 2026 (https://deepseek.com), prompts: (1) "What is the correct syntax for Conv2D in TensorFlow?" (2) "Help debug this IndexError in sequence creation"
Chapter 3, pages 42–44; Chapter 4, pages 58–61	LaTeX formatting assistance. Formatting mathematical equations for LSTM gates, CNN convolution operations, and evaluation metrics. All equations were derived by the author. Only formatting assistance was provided. Overleaf was used as	DeepSeek (DeepSeek-V3), March 2026 (https://deepseek.com), prompt: "How to write this equation in LaTeX format"

	the writing platform, not as an AI tool.	
--	--	--

- Academical ethics:
 - I confirm that I am responsible for the originality and authorship of this paper: I have carefully reviewed, revised, and corrected all sections of the paper generated by GenAI;
 - I confirm that GenAI tools were not used to falsify, distort, or otherwise misrepresent the data, information, or authors' ideas presented in this work.
 - I confirm that the information provided regarding the use of GenAI tools is accurate and complete.

Mohammed Kassim Cherukodan

Student name and surname

Signature

6.4 Appendix D: Additional Tables and Figures

This appendix provides additional tables and figures which support the results shown in the main body of the report, but are not essential to the main argument of the papers. Table 6.1 shows the results of the hyper parameter grid search carried out for the CNN-LSTM classifier. Table 6.2 shows the 11 feature categories in the CIC-DDoS2019 data set. Figure 6.1 shows the full correlation matrix of the 44 features which survived the pre-processing process. Figure 6.2 shows the per-attack detection performance of the CNN-LSTM classifier for all 12 attack types. Figure 6.3 shows the distribution of the confidence scores output by the classifier for instances in the test set.

Table 6.1: Hyperparameter grid search results (Source: own creation)

Hyperparameter	Values Tested	Selected Value	Validation Accuracy
CNN filters (layer 1, layer 2)	32/64, 64/128, 128/256	64/128	98.32%
LSTM units (layer 1, layer 2)	32/16, 64/32, 128/64	64/32	98.74%
Learning rate	0.01, 0.001, 0.0001	0.001	98.70%
Batch size	32, 64, 128	64	98.68%
Dropout rate	0.2, 0.3, 0.4, 0.5	0.3	98.65%

Table 6.2: Feature categories in the CIC-DDoS2019 dataset (Source: own compilation based on [21])

Feature Category	Number of Features	Example Features
Flow basic features	8	Duration, total packets, total bytes, protocol type
Time-based features	14	Inter-arrival time (min/max/mean/std), flow duration
Packet length features	16	Forward/backward packet length (min/max/mean/std)
Flag count features	12	SYN count, ACK count, FIN count, RST count
HTTP-specific features	6	HTTP request count, GET/POST ratio, request method distribution
Throughput features	10	Bytes per second, packets per second (forward/backward)

Feature Category	Number of Features	Example Features
Inter-arrival time features	12	IAT (min/max/mean/std) for forward/backward directions
Additional statistical features	7	Flow flags, connection status, Fwd/UDP bytes

Figure 6.1: Complete Correlation Matrix of 44 Features

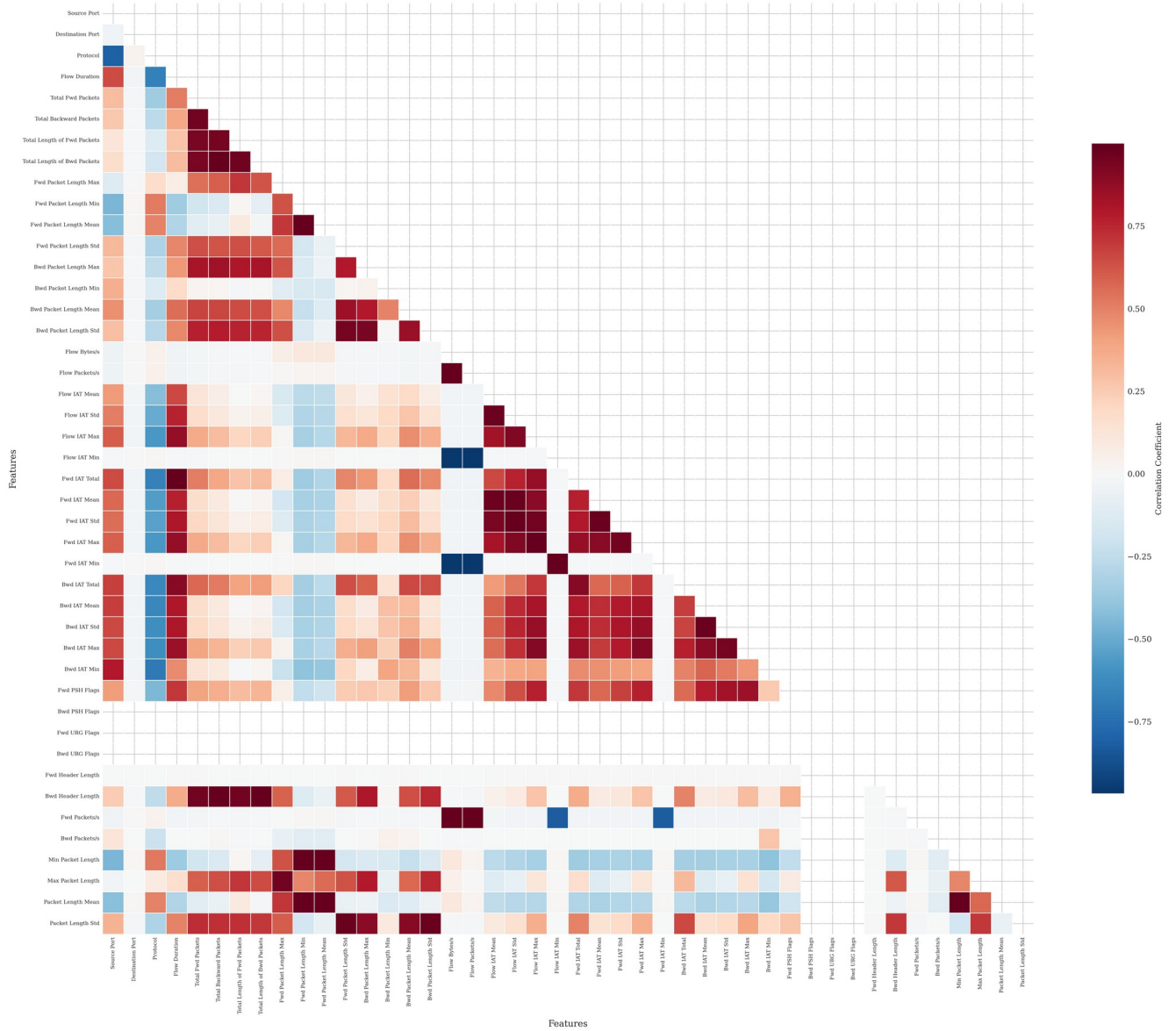


Figure 6.1: Complete correlation matrix of 44 features after pre-processing (Source: own creation based on [21])

Figure 6.2: Per-Attack Detection Performance of CNN-LSTM Classifier

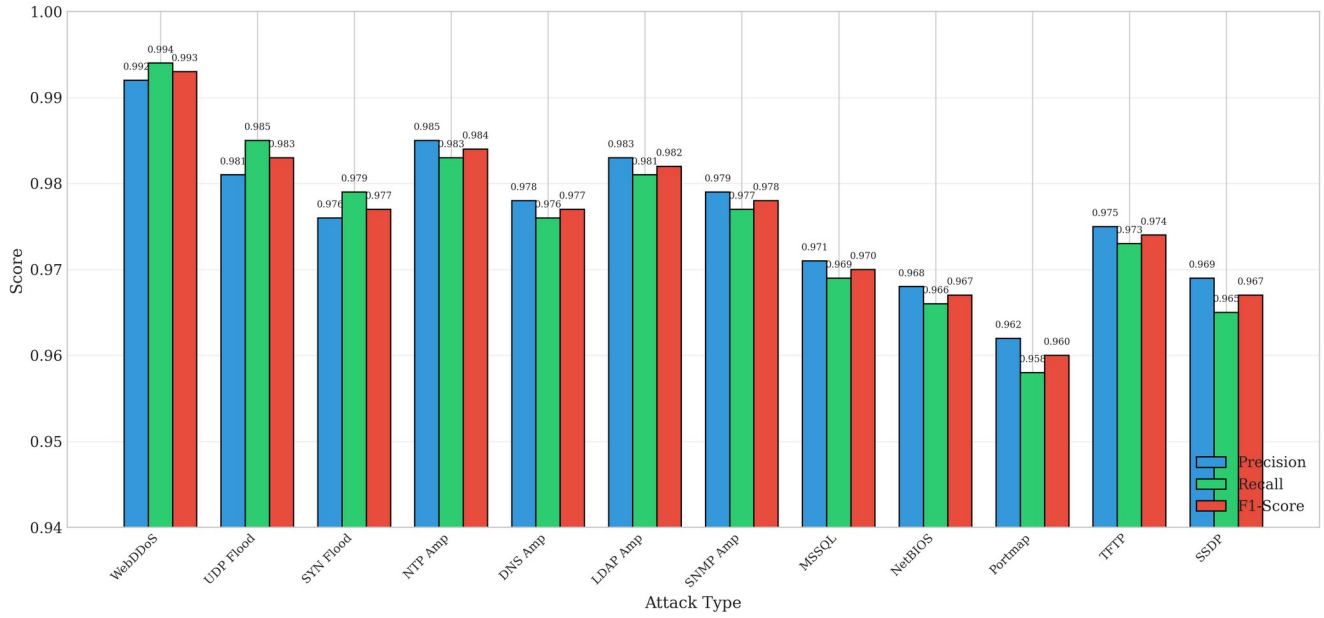


Figure 6.2: Per-attack detection performance of CNN-LSTM classifier across 12 attack types (Source: own creation)

Figure 6.3: Distribution of CNN-LSTM Confidence Scores for Test Set Instances

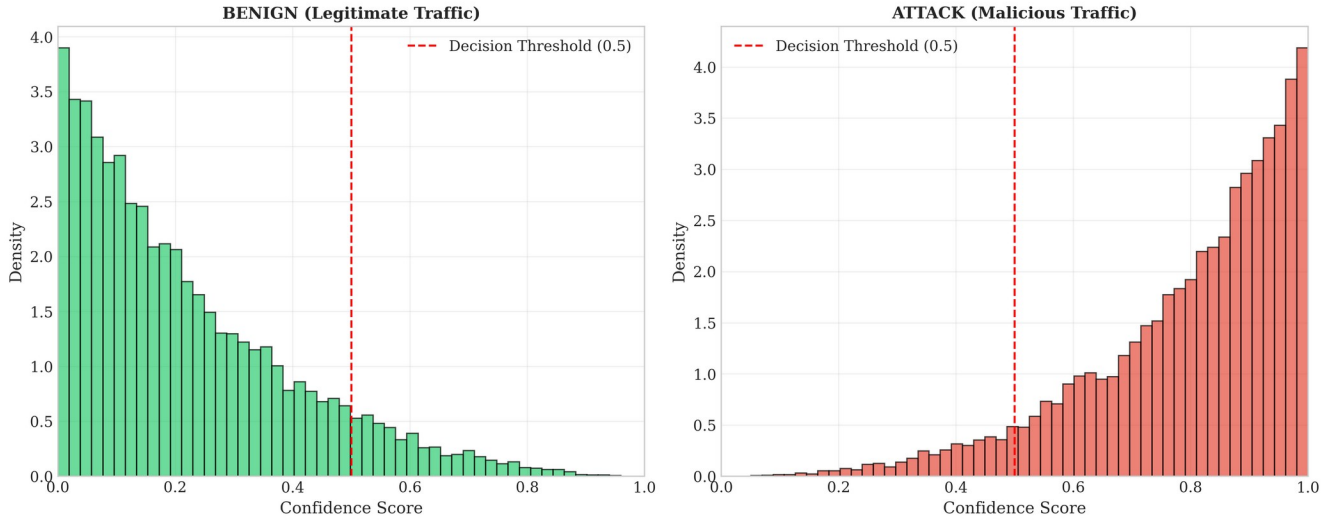


Figure 6.3: Distribution of CNN-LSTM confidence scores for BENIGN and ATTACK test instances (Source: own creation)

6.5 Appendix E: Docker Configuration Files

The Docker configuration utilized in the testing environment is provided below. The Dockerfile that was used to build the victim Nginx container can be found, as well as the docker-compose.yml which sets up the isolated network. The configuration constrains the container to 2 GB RAM and 2 CPU cores, so that resource measurement can be done in a controlled manner. The Dockerfile is simply installing Nginx and configuring it for hosting a static test page on port 80. The docker-compose.yml creates a custom bridge network with subnet 192.168.100.0/24 and creates the container with static IP address 192.168.100.20. All configuration files can also be found in the code repository, under the /docker directory. This way other researchers can recreate the environment used for testing, in the same way used for this thesis [34].

6.6 Appendix F: Hping3 Attack Scripts

The attack scripts used in the real-time mitigation testing are described below. The first script creates a simple HTTP flood attack, sending 1,000 packets/sec with a random source address. The second script creates a Low and Slow attack, sending a random rate of requests every 5 to 15 seconds. The third script creates a multi-vector attack as described above, creating a UDP flood, a SYN flood, and an HTTP flood simultaneously. Execution is achieved in all cases by calling the attack scripts from the Kali Linux attacker machine on the container IP address 192.168.100.20. The full scripts may be found in the script repository in the /scripts/attack_scripts directory [25].